3 Программная реализация

Целью практической части данной дипломной работы является разработка программного модуля для моделирования и оценки криптостойкости функции хэширования ГОСТ Р 34.11–2012 к классической и квантовой herding-атакам. В связи с высокой вычислительной сложностью рассматриваемых атак в программной реализации алмазная структура строится с использованием частей промежуточных хэш-кодов и контрольных сумм функции хэширования ГОСТ Р 34.11–2012.

Программа разработана на языках Python и C++. Для создания графического интерфейса используется библиотека PySide6, для работы с графами – NetworkX, а для визуализации алмазных структур и результатов экспериментальных исследований – Matplotlib. Вычисления функции сжатия и ресурсоемкое моделирование квантового поиска выполняются на языке C++, а Python обеспечивает управление общими алгоритмами, графом и пользовательским интерфейсом.

3.1 Реализация алгоритма Гровера

Для моделирования квантовых этапов herding-атаки разработана программная модель алгоритма Гровера. Алгоритм предназначен для поиска элемента, удовлетворяющего заданному условию, в неструктурированном пространстве поиска. В программном модуле предусмотрены три варианта моделирования квантового поиска: Python-модель с полным массивом амплитуд, C++-реализация алгоритма Гровера с фиксированным числом итераций и адаптивный алгоритм Бойера, Брассара, Хейера и Таппа (BBHT) [21], позволяющий выполнять поиск без предварительного знания количества подходящих элементов. Все варианты используют принципы фазового оракула и оператора диффузии, но отличаются способом представления квантового состояния и стратегией выбора числа итераций. Листинги Python-модели Гровера, C++-модели фиксированного поиска и адаптивного алгоритма BBHT

представлены соответственно в приложениях А, Б и В.

Работа алгоритма начинается с формирования равномерной суперпозиции элементов пространства поиска [22]. Для пространства мощности N соответствующий вектор имеет вид $|\psi_0\rangle = \frac{1}{\sqrt{N}}\sum_{x=0}^{N-1}|x\rangle$, где N – число элементов пространства поиска, $|x\rangle$ – базисное состояние, соответствующее элементу с номером x, а $\frac{1}{\sqrt{N}}$ – его начальная амплитуда. В Python-реализации квантовые вычисления моделируются на классическом компьютере, поэтому квантовый регистр непосредственно не создается. Вместо него используется массив амплитуд $A = (a_0, a_1, \dots, a_{N-1})$, причем перед началом поиска каждой амплитуде присваивается одинаковое значение $a_i = \frac{1}{\sqrt{N}}$. Формирование начального массива амплитуд реализовано в функции simulate_grover_from_marked.

Проверка заданного условия выполняется с помощью фазового оракула. Элементы, удовлетворяющие заданному условию, считаются помеченными. Действие фазового оракула заключается в изменении знака амплитуды каждого помеченного элемента $a_i \to -a_i$. При этом вероятность измерения соответствующего состояния не изменяется.

После применения фазового оракула применяется оператор диффузии. Сначала вычисляется среднее значение амплитуд $\overline{a} = \frac{1}{N}\sum_{i=0}^{N-1} a_i$. Затем каждая амплитуда отражается относительно полученного среднего значения $a_i \to 2\overline{a} - a_i$. Такое преобразование соответствует действию оператора диффузии $D = 2|\psi_0\rangle\langle\psi_0| - I$, где I – единичный оператор [23].

Последовательное применение фазового оракула и оператора диффузии образует одну итерацию алгоритма Гровера [21, 24]. Описанные преобразования реализованы в функции simulate_grover_from_marked.

Пусть в пространстве поиска из N элементов имеется R помеченных элементов. Поскольку в начальной суперпозиции все элементы равновероятны, вероятность получения помеченного элемента равна R/N. Введем угол θ, для

которого $\sin^2\theta = \frac{R}{N}$. Отсюда получаем $\theta = \arcsin\sqrt{\frac{R}{N}}$, определяющий изменение вероятности успеха при последовательном выполнении итераций алгоритма Гровера.

Каждая итерация алгоритма Гровера соответствует повороту вектора состояния на угол 2θ в двумерном пространстве, образованном суперпозициями помеченных и непомеченных элементов. Поэтому после r итераций амплитуда суперпозиции помеченных состояний равна $\sin((2r+1)\theta)$, а вероятность успешного поиска определяется выражением $P(r) = \sin^2((2r+1)\theta)$, при этом увеличение числа итераций не всегда приводит к повышению вероятности успеха, поэтому их количество необходимо выбирать с учетом числа помеченных элементов [21]. В программной реализации вычисление вероятности успеха выполняется функцией grover_success_probability.

Первый максимум вероятности достигается при условии $(2r+1)\theta \approx \frac{\pi}{2}$. Выражая из данного равенства r, получаем $r \approx \frac{\pi}{4\theta} - \frac{1}{2}$.

Поскольку число итераций должно быть целым, в программной реализации используется ближайшее неотрицательное целое значение $r = \max\left(0,\ round\left(\frac{\pi}{4\theta} - \frac{1}{2}\right)\right)$. Вычисление количества итераций реализовано в функции optimal_grover_iterations. При моделировании квантовых этапов herding-атаки в режиме фиксированного поиска количество итераций выбирается исходя из ожидаемого наличия одного подходящего элемента. При этом вероятность успешного результата рассчитывается с учетом фактического количества помеченных элементов.

При наличии одного помеченного элемента $R = 1$ и достаточно большом N справедливо приближение $\theta = \arcsin\frac{1}{\sqrt{N}} \approx \frac{1}{\sqrt{N}}$. Подставляя его в выражение для числа итераций, получаем $r \approx \frac{\pi}{4}\sqrt{N} - \frac{1}{2}$. При больших N слагаемым $\frac{1}{2}$ можно пренебречь, поэтому $r \approx \frac{\pi}{4}\sqrt{N}$.

Вероятность успеха алгоритма Гровера зависит от числа выполненных

итераций и не возрастает монотонно. После достижения первого максимума она начинает уменьшаться, а при дальнейшем увеличении числа итераций вновь возрастает. Поэтому слишком малое количество итераций может не обеспечить достаточно высокой вероятности успеха, а слишком большое – привести к ее снижению. Для правильного выбора числа итераций желательно знать количество помеченных элементов R, которое в реальной задаче поиска заранее может быть неизвестно.

В связи с этим в программном модуле наряду с фиксированным поиском реализован адаптивный алгоритм BBHT, позволяющий выполнять поиск без предварительного знания количества помеченных элементов за счет изменения числа итераций между попытками.

В начале работы BBHT устанавливается параметр $m = 1$, определяющий диапазон выбора числа итераций. Затем случайным образом выбирается целое число r из диапазона от 0 до $\lceil m \rceil - 1$. После этого выполняются r итераций Гровера. Затем моделируется измерение квантового состояния и проверяется соответствие полученного элемента заданному условию. Если подходящий элемент не найден, параметр m увеличивается, и процедура повторяется.

В реализованной модели увеличение параметра выполняется по правилу $m \leftarrow \min\left(\frac{6}{5}m, \sqrt{N}\right)$. То есть, после каждой неудачной попытки значение m увеличивается в $\frac{6}{5}$ раза, но не превышает $\sqrt{N}$, благодаря чему постепенно расширяется диапазон возможного количества итераций. Случайный выбор позволяет выполнять поиск с разным числом итераций и уменьшает зависимость его успешности от заранее принятого предположения о количестве решений.

В квантовой модели при наличии хотя бы одного помеченного элемента BBHT обеспечивает ожидаемую запросную сложность $O\left(\sqrt{\frac{N}{R}}\right)$.

В программной реализации предусмотрены ограничения на количество попыток и запросов к оракулу. Для адаптивного алгоритма BBHT эти ограничения задаются параметрами max_attempts и max_oracle_queries

структуры bbht_adaptive::Options. Если подходящий элемент не найден до достижения установленных ограничений, поиск завершается с признаком неудачи.

После выполнения требуемого числа итераций в полной модели квантового состояния для каждого элемента пространства поиска вычисляется вероятность результата измерения $p_i = |a_i|^2$.

Для проверки корректности вычислений дополнительно контролируется условие нормировки $\sum_{i=0}^{N-1} p_i = 1$. В программном модуле вычисляется также численное отклонение суммы вероятностей от единицы $\varepsilon = \left|\sum_{i=0}^{N-1} p_i - 1\right|$. Данная величина используется как диагностический показатель корректности моделирования квантового состояния.

Результат измерения моделируется случайным выбором индекса в соответствии с рассчитанным распределением вероятностей. Формирование начального состояния, выполнение итераций Гровера, вычисление вероятностей, контроль нормировки и моделирование измерения реализованы в функции simulate_grover_from_marked.

При больших пространствах поиска хранение полного массива амплитуд приводит к существенным затратам памяти. В C++-ядре для малых пространств поиска фиксированный алгоритм Гровера и BBHT используют моделирование полного вектора амплитуд. При превышении установленного порога применяется сокращенная симметричная модель, в которой отдельно представляются амплитуды помеченных и непомеченных состояний. Для полной модели используется simulate_grover_statevector или bbht_dense_statevector::simulate, а для сокращенной – grover_symmetric::simulate.

В общем C++-ядре предусмотрены два варианта квантового поиска. В фиксированном варианте для каждого множества кандидатов выполняется одна попытка поиска с заранее определенным числом итераций. При неудаче поиск может продолжаться на новом множестве кандидатов. В алгоритме BBHT число итераций случайно выбирается из диапазона, расширяемого после неудачных

попыток. В обоих вариантах учитываются обращения к фазовому оракулу и дополнительные проверки результата, а при построении алмазной структуры отдельно учитываются затраты на формирование таблицы qRAM, моделируемой классической структурой данных QuantumQram.

В реализации отдельно учитываются модельные квантовые запросы к оракулу и фактические вычисления функции сжатия, выполненные классическим симулятором. Эти величины не являются взаимозаменяемыми. Для регистрации соответствующих показателей в C++-ядре используются счетчики oracle_queries и simulator_evaluations. Число модельных квантовых запросов используется для сопоставления с теоретической сложностью квантового алгоритма, а число вычислений функции сжатия характеризует вычислительные затраты программной симуляции на классическом компьютере.

В herding-атаке фиксированный и адаптивный варианты алгоритма Гровера применяются при построении квантовой алмазной структуры и при поиске связывающего блока. Реализация этих этапов рассматривается в разделах 3.3 и 3.4 соответственно.

Пусть n – число сохраняемых бит промежуточного хэш-кода, а 2^k – количество начальных состояний алмазной структуры. При предположении о равномерном распределении значений функции сжатия вероятность перехода в одно из начальных состояний при обработке случайного блока составляет $p = \frac{2^k}{2^n} = 2^{k-n}$.

Следовательно, ожидаемое количество классических проверок для нахождения связывающего блока равно $\frac{1}{p} = 2^{n-k}$. В программной реализации эта величина используется для задания размера пространства поиска $S_{link} = 2^{n-k}$. Выбранный размер пространства поиска соответствует ожидаемому одному подходящему элементу, однако в конкретном наборе кандидатов решений может не оказаться. Вычисление данного параметра выполняется функцией algorithm5_link_parameters, а соответствующее количество кандидатов сохраняется в переменной search_messages. При этом фактическое число

подходящих блоков в сформированном пространстве может отличаться от единицы.

При наличии одного подходящего блока в сформированном пространстве количество квантовых запросов для поиска с использованием алгоритма Гровера имеет порядок $O\left(\sqrt{S_{link}}\right) = O\left(2^{\frac{n-k}{2}}\right)$.

В программном модуле выбор метода передается в общее вычислительное ядро через идентификаторы classical, grover_fixed и grover_bbht. Поэтому оба квантовых варианта доступны и на предварительном этапе построения алмазной структуры, и при последующем поиске связывающего блока. Для квантового построения используется функция build_diamond_structure_quantum, а для квантового поиска связывающего блока – find_linking_message_quantum. Метод построения алмазной структуры и метод поиска связывающего блока выбираются независимо.

3.2 Особенности применения herding-атаки к ГОСТ Р 34.11–2012

Теоретическая herding-атака рассматривается для итеративной хэш-функции, в которой переход между промежуточными хэш-кодами выполняется функцией сжатия. При применении данной схемы к ГОСТ Р 34.11–2012 необходимо учитывать, что функция сжатия g_N дополнительно зависит от счетчика длины обработанного сообщения N, а процедура хэширования использует накопительную контрольную сумму Σ, которая участвует в заключительном этапе вычисления хэш-кода [9]. Листинг Python-реализации модели промежуточного хэш-кода с учетом контрольной суммы представлен в приложении Г, а C++-реализация функции сжатия и конструктивного изменения младших бит блока – в приложении Д.

Поскольку проведение herding-атаки для полного 512-битного пространства вычислительно неосуществимо, в программном макете используется редуцированная модель с n-битным промежуточным хэш-кодом [2, 3], то есть переход определяется как $G_n(h, m, N) = Trunc_n\left(g_N(Emb_n(h), m)\right)$,

где Emb_n дополняет n-битный хэш-код нулями до 512 бит, после чего применяется функция сжатия ГОСТ, а из результата сохраняются первые n бит.

Низкоуровневые вычисления функции сжатия в общем C++-ядре выполняются с использованием функций make_context, compress_with_context и служебных процедур. Эти же операции применяются в классическом построении, в фиксированном Гровере и в BBHT.

В текущей реализации дополнительно учитывается часть контрольной суммы. Для этого используется пара $X = (h_n, \Sigma_s),\ \Sigma_s = \Sigma\ mod\ 2^s$, где s – число сохраняемых бит контрольной суммы, а h_n – n-битный промежуточный хэш-код.

После обработки блока m контрольная сумма обновляется независимо от функции сжатия $\Sigma_s' = \left(\Sigma_s + (m\ mod\ 2^s)\right)\ mod\ 2^s$. Чтобы выполнение условия по Σ_s не требовало дополнительного перебора, в генерируемом 512-битном блоке заранее фиксируются младшие s бит. Для перехода от Σ_s к требуемому значению $\Sigma_{s'}$ они выбираются как $m\ mod\ 2^s = (\Sigma_s' - \Sigma_s)\ mod\ 2^s$.

При квантовом построении алмазной структуры вычисление требуемых младших бит блока и их установка выполняются в C++-функциях required_sigma_low64 и set_low_sigma_bits.

При построении алмазной структуры для каждого уровня заранее задается общее значение части контрольной суммы, к которому должны приводить переходы из состояний предыдущего уровня. Значения части контрольной суммы для последовательных уровней алмазной структуры определяются функцией sigma_schedule.

Остальные $512 - s$ бит блока остаются доступными для поиска коллизии по h_n. Поэтому в реализованной модели поиск коллизий и связывающего блока по-прежнему выполняется по n-битному хэш-коду, а выполнение условия по контрольной сумме обеспечивается конструктивно. Это позволяет учитывать часть контрольной суммы без отдельного перебора по ее возможным значениям.

Значение N учитывается отдельно и увеличивается на 512 после обработки каждого блока. При построении алмазной структуры все переходы одного уровня

выполняются при одинаковом N, а при поиске связывающего блока используется значение N, соответствующее числу уже обработанных блоков префикса. При квантовом построении алмазной структуры значение счетчика для каждого уровня определяется в функции build_diamond_structure_quantum по формуле $N_i = N_0 + 512l$, где N_0 – значение счетчика для первого уровня, а l – номер уровня, начиная с нуля.

3.3 Реализация построения алмазной структуры с использованием квантовых алгоритмов

Квантовое построение алмазной структуры в программном модуле реализовано в соответствии с алгоритмом 2.1 из главы 2. Алмазная структура строится последовательно по уровням, начиная с 2^k различных начальных хэш-кодов и заканчивая единственным корневым хэш-кодом h_T. На каждом уровне моделируется квантовый поиск коллизий между вершинами, после чего число вершин уменьшается в два раза. Управление построением структуры реализовано на языке Python в функции build_diamond_structure_quantum, а вычисления отдельных уровней выполняются общим C++-ядром с реализациями фиксированного поиска и BBHT. Листинг управления построением алмазной структуры на языке Python приведен в приложении Е, а единое C++-ядро поиска коллизий с поддержкой фиксированного Гровера и BBHT – в приложении Ж.

Пусть n соответствует числу сохраняемых бит хэш-кода, а $z = \log_2 \nu$, где ν – число вершин текущего уровня, то есть на первом уровне $z = k$, а при переходе к следующему уровню значение z уменьшается. Для каждого уровня также рассчитываются размеры двух множеств блоков сообщений $L = \left\lceil z^{\frac{2}{3}} 2^{\frac{n-z}{3}} \right\rceil$ и $S = \left\lceil z^{-\frac{2}{3}} 2^{\frac{2(n-z)}{3}} \right\rceil$. Вычисление параметров L и S реализовано на языке Python в функции algorithm2_parameters.

Сначала в C++-ядре генерируется единое множество из L блоков $M = \{m_1, m_2, \dots, m_L\}$, которое используется для всех вершин текущего уровня. Для

каждой вершины x_i вычисляются результаты редуцированной функции сжатия $G_n(x_i, m_j, N), m_j \in M$. Формирование множеств блоков выполняется в C++-функциях build_quantum_layer и quantum_diamond_bbht::build_layer для фиксированного Гровера и BBHT соответственно, а вычисления функции сжатия – в функции evaluate_message_space_into.

Полученные значения вместе с номером исходной вершины и соответствующим блоком сообщения сохраняются в программной модели qRAM, реализованной структурой QuantumQram. В качестве ключа поиска используется n-битное значение хэш-кода, которому сопоставляется список записей, содержащих номер исходной вершины и использованный блок сообщения. При совпадении хэш-кодов записи не удаляются, что позволяет обнаруживать коллизии между различными вершинами.

После заполнения qRAM для каждой вершины x_i выполняется z независимых поисков Гровера. Перед каждым поиском в C++-ядре формируется новое множество $M' = \{m'_1, m'_2, \dots, m'_S\}$, содержащее S блоков сообщений.

Для всех блоков из множества M' предварительно вычисляются значения $G_n(x_i, m'_j, N)$, на основании которых формируется массив помеченных элементов. Блок m'_j считается помеченным, если полученный хэш-код уже содержится в qRAM и соответствует другой вершине x_l, где $l \neq i$. Тем самым выполняется поиск коллизии $G_n(x_i, m'_a, N) = G_n(x_l, m_b, N), i \neq l$.

Моделирование квантового поиска выполняется выбранным вариантом алгоритма Гровера. В фиксированном варианте используется функция simulate_grover_statevector, выполняющая одну попытку поиска с заранее определенным числом итераций для каждого множества кандидатов. В адаптивном варианте используется bbht_adaptive::search, в котором число итераций выбирается случайным образом из диапазона, расширяемого после неудачных попыток. Максимальное число попыток BBHT ограничивается в зависимости от размера пространства поиска.

По найденным коллизиям строится граф, вершинами которого являются

промежуточные хэш-коды текущего уровня, а ребрами – коллизии между различными вершинами. В полученном графе с использованием библиотеки NetworkX строится набор непересекающихся пар вершин, причем каждая вершина текущего уровня должна входить ровно в одну пару. Для каждой такой пары уже имеются два блока сообщений, переводящие разные исходные хэш-коды в один общий: $G_n(x_i, m_i, N) = G_n(x_j, m_j, N) = x'$. Поиск такого набора выполняется на языке Python в функции build_one_layer_quantum.

Полученные общие хэш-коды образуют следующий уровень алмазной структуры. Если найденных коллизий недостаточно для формирования требуемых пар или полученные хэш-коды не являются попарно различными, построение данного уровня считается неудачным и выполняется повторно с новыми наборами блоков.

При использовании модели $X = (h_n, \Sigma_s)$, блоки из множеств M и M' генерируются с дополнительным ограничением на младшие s бит. Поэтому каждый найденный переход одновременно обеспечивает требуемое изменение части контрольной суммы. Установка младших бит блоков выполняется в C++-функции set_low_sigma_bits, которая вызывается при формировании множеств блоков.

Значения контрольной суммы для последовательных уровней заранее определяются на языке Python функцией sigma_schedule. После выбора пар вершин программа независимо повторяет вычисления обоих переходов каждой выбранной коллизии и проверяет совпадение полученного хэш-кода и корректность изменения Σ_s. Проверка реализована в Python-функции build_one_layer_quantum с использованием метода ReducedGostModel.transition и функции update_sigma_reduced. В экспериментальном модуле дополнительно проверяется выполнение условий по контрольной сумме для выбранных переходов, а после завершения полной herding-атаки независимо пересчитывается сформированное сообщение.

Для оценки вычислительных затрат отдельно регистрируются число

модельных квантовых запросов к оракулу и фактическое количество вычислений функции сжатия, выполненных классическим симулятором. В C++-ядре соответствующие показатели сохраняются в счетчиках oracle_queries и simulator_evaluations, после чего учитываются функцией build_one_layer_quantum.

Для построения алмазной структуры пользователь открывает вкладку «Алмазная структура» в верхней части окна программы, как показано на рисунке 3.1. В левой части окна находится панель «Параметры эксперимента», в которой выбирается метод построения и задаются исходные параметры.

В списке «Алгоритм построения» пользователь выбирает классический алгоритм, фиксированный Гровер или BBHT. Общими параметрами служат размер части хэш-кода n, глубина алмазной структуры k и размер части контрольной суммы s. При $s = 0$ часть контрольной суммы не учитывается.

После установки параметров пользователь нажимает кнопку «Построить алмазную структуру». Программа последовательно строит уровни алмазной структуры, начиная с 2^k начальных хэш-кодов. Если на каком-либо уровне не удалось найти необходимую пару, выполняется повторная попытка построения этого уровня.

После успешного завершения в центральной части окна отображается построенная алмазная структура, а справа – результаты вычислений, включая число модельных запросов к квантовому оракулу и число фактических вычислений функции сжатия. В блоке «Результаты» также находится журнал, содержащий сообщения о ходе работы, и форматированный JSON файл данного построения. Построенная структура сохраняется программой в JSON-файл, который впоследствии можно загрузить через кнопку открытия файла в панели параметров. В метаданных сохраняется метод построения, что позволяет впоследствии загрузить эту же структуру во вкладке «Herding-атака» и независимо выбрать метод поиска связывающего блока.

Также после построения структуры можно провести повторную проверку алмазной структуры с помощью кнопки «Проверить структуру». При нажатии на

вершину можно проследить путь от нее и основную информацию.

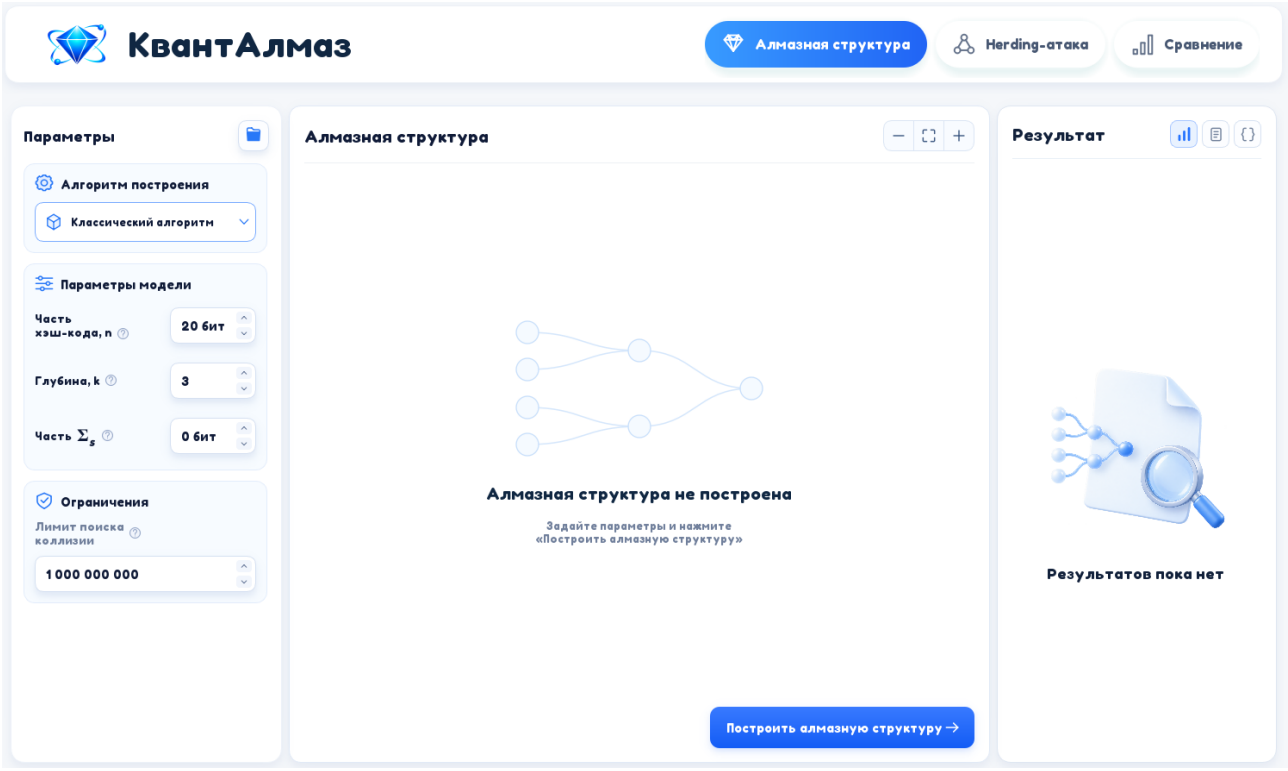


Рисунок 3.1 – Интерфейс программного модуля. Вкладка «Алмазная структура»

Примеры:

1 Классическое построение алмазной структуры при $n = 16$, $k = 3$, $s = 4$. В результате получена структура из 8 начальных хэш-кодов с 3 уровнями объединений: $8 \to 4 \to 2 \to 1$. Корневой хэш-код составил 2c8e. Программная модель зарегистрировала 3622 вычисления функции сжатия. Результат показан на рисунке 3.2.

2 Квантовое построение алмазной структуры, использующее алгоритм фиксированного Гровера при $n = 16$, $k = 3$, $s = 4$. В результате получена структура из 8 начальных хэш-кодов с 3 уровнями объединений: $8 \to 4 \to 2 \to 1$. Корневой хэш-код составил 1c68. Программная модель зарегистрировала 2132 модельных запросов к оракулу и 23326 фактических вычислений функции сжатия. Результат показан на рисунке 3.3.

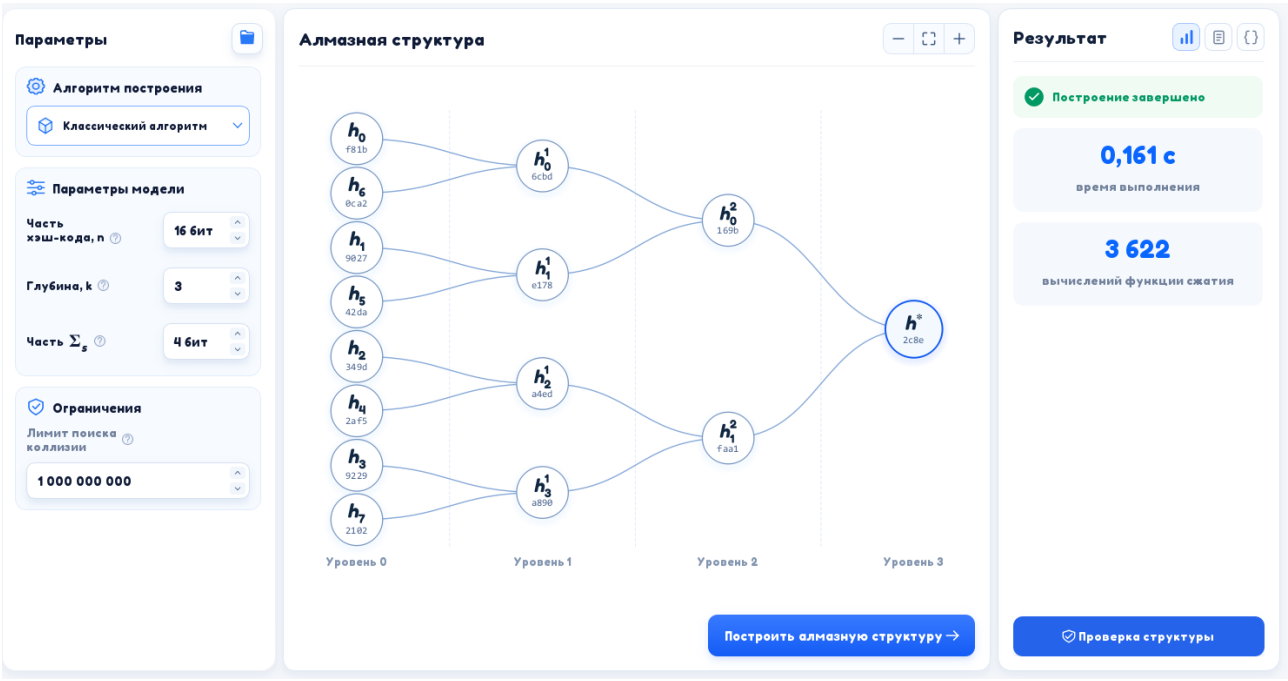


Рисунок 3.2 – Результат классического построения при $n = 16$, $k = 3$, $s = 4$

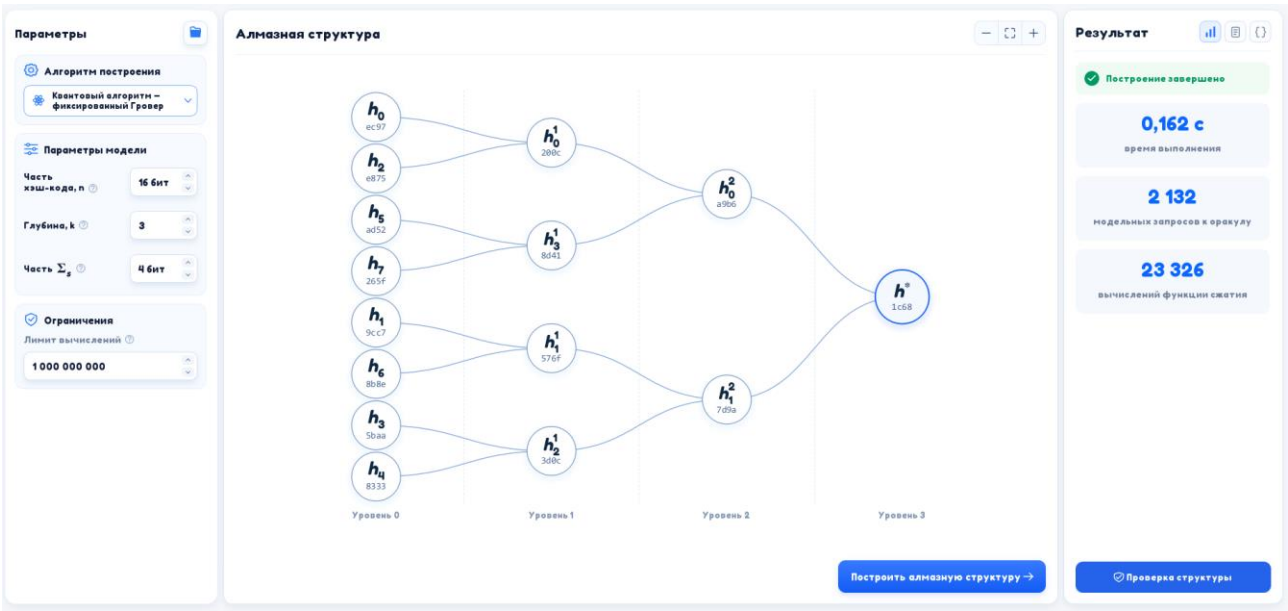


Рисунок 3.3 – Результат фиксированного Гровера при $n = 16$, $k = 3$, $s = 4$

3 Квантовое построение алмазной структуры, использующее адаптивный алгоритм ВВНТ при $n = 16$, $k = 3$, $s = 4$. В результате получена структура из 8 начальных хэш-кодов с 3 уровнями объединений: $8 \to 4 \to 2 \to 1$. Корневой хэш-код составил 6088. Программная модель зарегистрировала 3010 модельных запросов к квантовому оракулу и 14606 фактических вычислений функции сжатия. Результат показан на рисунке 3.4.

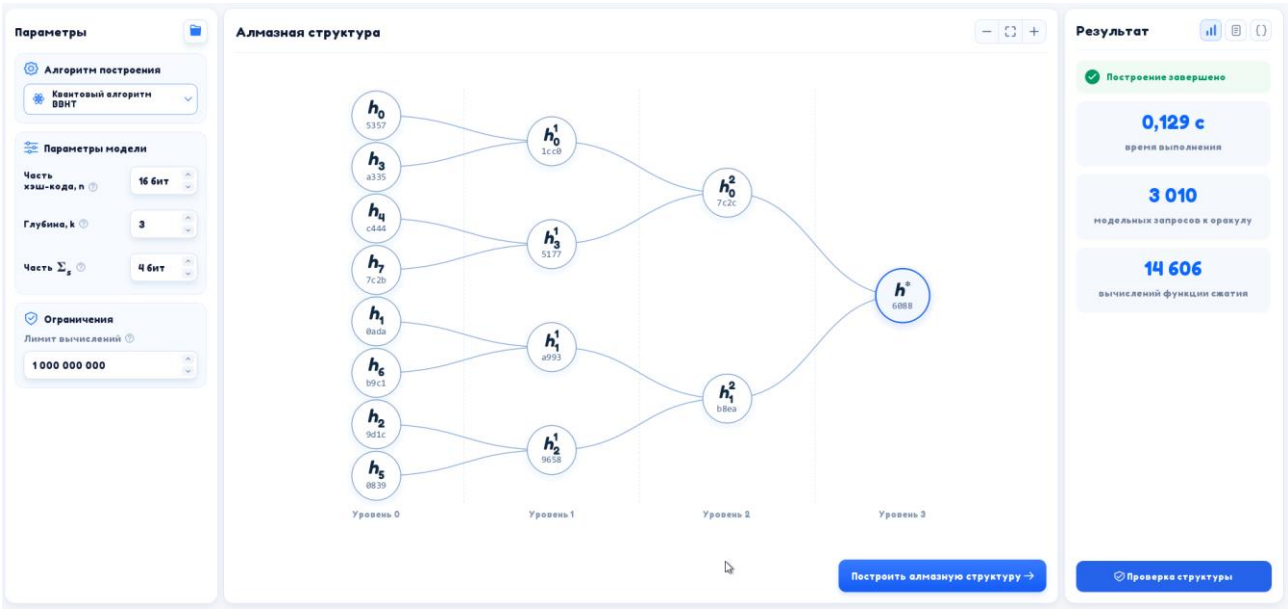


Рисунок 3.4 – Результат адаптивного BBHT при $n = 16$, $k = 3$ и $s = 4$

3.4 Программная демонстрация классической и квантовой herding-атаки для функции хэширования ГОСТ Р 34.11–2012

Для демонстрации herding-атаки разработан единый программный модуль, позволяющий выполнять классический и квантовый варианты атаки для одной и той же предварительно построенной алмазной структуры. Общая последовательность действий соответствует схеме herding-атаки: предварительно фиксируется корневой хэш-код h_T, после получения префикса выполняется поиск связывающего блока M_{link}, а затем к сообщению добавляется заранее известный путь по алмазной структуре до h_T [4, 17].

В программе сначала загружается JSON-файл с ранее построенной алмазной структурой. Файл содержит начальные хэш-коды, переходы между уровнями, блоки сообщений, корневой хэш-код h_T, параметры n, k, N, число раундов функции сжатия и параметры Σ_s. Перед использованием структура проходит проверку корректности.

Реализация полной herding-атаки, включая восстановление и независимую проверку пути, приведена в приложении З; общее C++-ядро трех стратегий поиска связывающего блока – в приложении И. Управление этапами пользовательского сценария вынесено в приложение К, а командный интерфейс

cli.py вызова тех же алгоритмов представлен в приложении Л.

Демонстрация построена по сценарию Нострадамуса. До получения результата некоторого события пользователь задает неизменяемую часть сообщения и фиксирует корневой хэш-код h_T. После появления результата события из этих двух частей формируется префикс $P = P_{fixed} \parallel P_{event}$.

В текущей реализации текст кодируется в Windows-1251, после чего размещается в заданном числе 512-битных блоков. После текстовых данных добавляется байт 0x80, а оставшаяся часть выделенного пространства заполняется нулевыми байтами. Число блоков префикса задается при построении алмазной структуры, поэтому после обработки P значение счетчика N должно совпадать со значением, для которого предусмотрен последующий поиск связывающего блока.

После обработки блоков префикса вычисляются редуцированный хэш-код h_P и, если используется расширенная модель, значение Σ_p. Далее требуется найти такой блок M_{link}, который переводит h_P в один из 2^k начальных хэш-кодов алмазной структуры $G_n(h_P, M_{link}, N_P) = h_j, h_j \in D$.

При использовании модели с усеченной контрольной суммой найденный блок дополнительно должен обеспечить требуемый переход $\Sigma_j = \left(\Sigma_P + (M_{link} \bmod 2^s)\right) \bmod 2^s$.

В классическом варианте программа последовательно генерирует допустимые 512-битные блоки и вычисляет для каждого из них функцию сжатия. Поиск завершается, когда полученный n-битный хэш-код совпадает с одним из начальных хэш-кодов алмазной структуры. Для 2^k целевых хэш-кодов ожидаемая сложность такого поиска имеет порядок $O(2^{n-k})$.

Число фактически выполненных вычислений функции сжатия сохраняется программой и используется при последующем экспериментальном анализе.

В обоих квантовых вариантах тот же поиск выполняется посредством программной модели алгоритма 2.2. Размер пространства поиска выбирается равным $S_{link} = 2^{n-k}$.

Для каждого кандидата проверяется принадлежность полученного хэш-кода множеству листьев алмазной структуры. Подходящие кандидаты рассматриваются как помеченные элементы алгоритма Гровера. В результате теоретическое число запросов к квантовому оракулу имеет порядок $O\left(\sqrt{S_{link}}\right) = O\left(2^{\frac{n-k}{2}}\right)$.

В программном комплексе квантовый поиск выполняется классическим симулятором на C++. После вычисления результатов функции сжатия для кандидатов моделируются итерации Гровера и результат измерения. Для метода grover_fixed используется заранее выбранное число итераций, а для grover_bbht – адаптивная стратегия bbht_adaptive::search с последовательными попытками. Обе стратегии возвращают связывающий блок для одного и того же набора целевых листьев и проверяют одинаковые условия по части хэш-кода и части контрольной суммы. Число модельных запросов к оракулу учитывается отдельно от фактических вычислений сжатия классического симулятора.

После нахождения M_{link} программа определяет, в какой начальный хэш-код h_j был выполнен переход. Затем из сохраненной алмазной структуры восстанавливается последовательность блоков $M_{path} = M_1 \parallel M_2 \parallel \cdots \parallel M_k$, которая переводит выбранный лист в корневой хэш-код h_T. Итоговое сообщение формируется в виде $M = P \parallel M_{link} \parallel M_{path}$.

После формирования сообщения выполняется независимая проверка всего найденного пути. Программа повторно обрабатывает префикс, связывающий блок и каждый блок пути, используя соответствующее значение N на каждом уровне. В модели (h_n, Σ_s) одновременно проверяются как хэш-код, так и усеченная контрольная сумма. Успешной считается проверка, при которой в конце повторного вычисления получена та же пара, которая была сохранена для корня алмазной структуры $\left(h_{verified}, \Sigma_{verified}\right) = (h_T, \Sigma_T)$.

Пользовательский интерфейс отражает основные этапы атаки. Сначала выбирается алмазная структура и фиксируется h_T, затем вводится результат

события и выбирается один из трех методов поиска: классический, фиксированный Гровер или BBHT. После завершения программа отображает хэш-код после префикса, найденный лист, связывающий блок, длину пути и корневой хэш-код. Отдельно визуализируется найденный путь через алмазную структуру.

В программном модуле для классического поиска сохраняются время и число вычислений функции сжатия. Для обоих квантовых вариантов дополнительно регистрируются число модельных запросов к оракулу, количество фактических вычислений функции сжатия, число попыток и диагностические показатели симуляции. Результаты проверки, метод построения структуры и метод поиска сохраняются в базе экспериментов. Те же алгоритмы доступны из командной строки через cli.py.

При загрузке алмазной структуры метод ее построения не ограничивает метод последующего поиска связывающего блока. Например, структура, построенная посредством BBHT, может использоваться для классического linking.

Для проведения herding-атаки пользователь открывает вкладку «Herding-атака» в верхней части окна программы, как показано на рисунке 3.5. В левой части окна находится панель «Параметры», содержащая три раздела: «Алмазная структура», «Коммитмент» и «Поиск связующего блока».

Сначала пользователь загружает ранее построенную алмазную структуру через кнопку «Загрузить структуру» либо использует структуру, полученную в текущем сеансе работы программы. Параметры n, k и s определяются загруженной структурой и не задаются повторно. Метод ее построения не ограничивает выбор алгоритма последующего поиска связывающего блока.

В разделе «Коммитмент» пользователь вводит неизменяемую часть сообщения в поле «Префикс» и нажимает кнопку «Зафиксировать». После этого программа фиксирует корневой хэш-код h_T загруженной алмазной структуры, который отображается на кнопке. Данное значение определяется до получения результата события и не изменяется при последующем выполнении атаки.

После получения результата события пользователь переходит к разделу «Поиск связующего блока». В поле «Результат события» вводится соответствующий текст, который объединяется с ранее заданной частью префикса. В списке «Метод» выбирается классический алгоритм, фиксированный Гровер или BBHT. Дополнительно задается лимит поиска, ограничивающий количество рассматриваемых кандидатов.

Для выполнения атаки пользователь нажимает кнопку «Начать поиск». Программа вычисляет промежуточное состояние после обработки сформированного префикса и выбранным алгоритмом ищет связывающий блок, переводящий его в один из начальных хэш-кодов загруженной алмазной структуры. После нахождения подходящего блока восстанавливается заранее подготовленный путь от соответствующего листа до корня и формируется итоговое сообщение.

После успешного завершения в центральной части окна отображается найденный путь по алмазной структуре, а справа – результаты вычислений. Для классического алгоритма выводятся время выполнения и количество вычислений функции сжатия. Для квантовых вариантов дополнительно отображается число модельных запросов к квантовому оракулу и фактическое количество вычислений функции сжатия, выполненных классическим симулятором. В панели «Результат» также доступны журнал выполнения и форматированные JSON-данные проведенной атаки.

Полученное сообщение автоматически проверяется на соответствие зафиксированному корневому состоянию. Дополнительно пользователь может запустить независимую проверку с помощью кнопки «Проверка сообщения», при которой повторно вычисляются переходы по сформированному сообщению с учетом счетчика длины и контрольной суммы. Результаты проведенной атаки сохраняются в базе экспериментов для последующего анализа и сравнения методов.

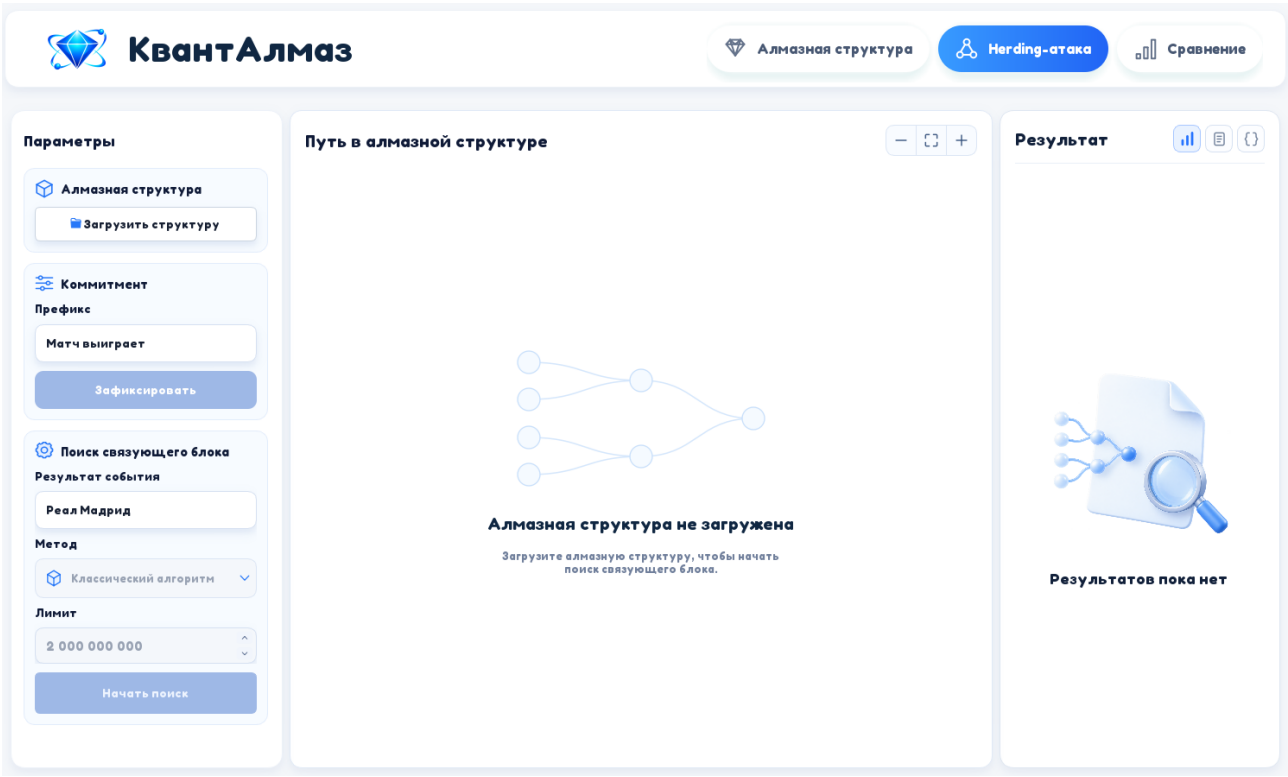


Рисунок 3.5 – Интерфейс программного модуля. Вкладка «Herding-атака»

Примеры:

1 Проведена классическая herding-атака с использованием предварительно построенной алмазной структуры из примера 1 раздела 3.3 при $n = 16$, $k = 3$, $s = 4$. В качестве неизменяемой части сообщения задан префикс «Золотой мяч выиграет», а после фиксации корневого хэш-кода введен результат события «Килиан Мбаппе». После восстановления пути сформировано итоговое сообщение, для которого подтверждено совпадение с предварительно зафиксированным корневым хэш-кодом 2c8e. Программа зарегистрировала 9312 вычислений функции сжатия. Результат представлен на рисунке 3.6.

2 Проведена квантовая herding-атака с использованием алгоритма Гровера с фиксированным числом итераций. Для алмазной структуры из примера 2 раздела 3.3 с параметрами $n = 16$, $k = 3$, $s = 4$ сформирован префикс «Акции компании БромС», а после фиксации корневого хэш-кода введен результат события «вырастут на 30%». В результате моделирования квантового поиска найден связывающий блок, после добавления которого и соответствующего пути по алмазной структуре получено сообщение с корневым хэш-кодом 1c68.

Программа зарегистрировала 288 модельных запросов к квантовому оракулу и 32772 фактических вычислений функции сжатия. Независимая проверка подтвердила корректность сформированного сообщения. Результат показан на рисунке 3.7.

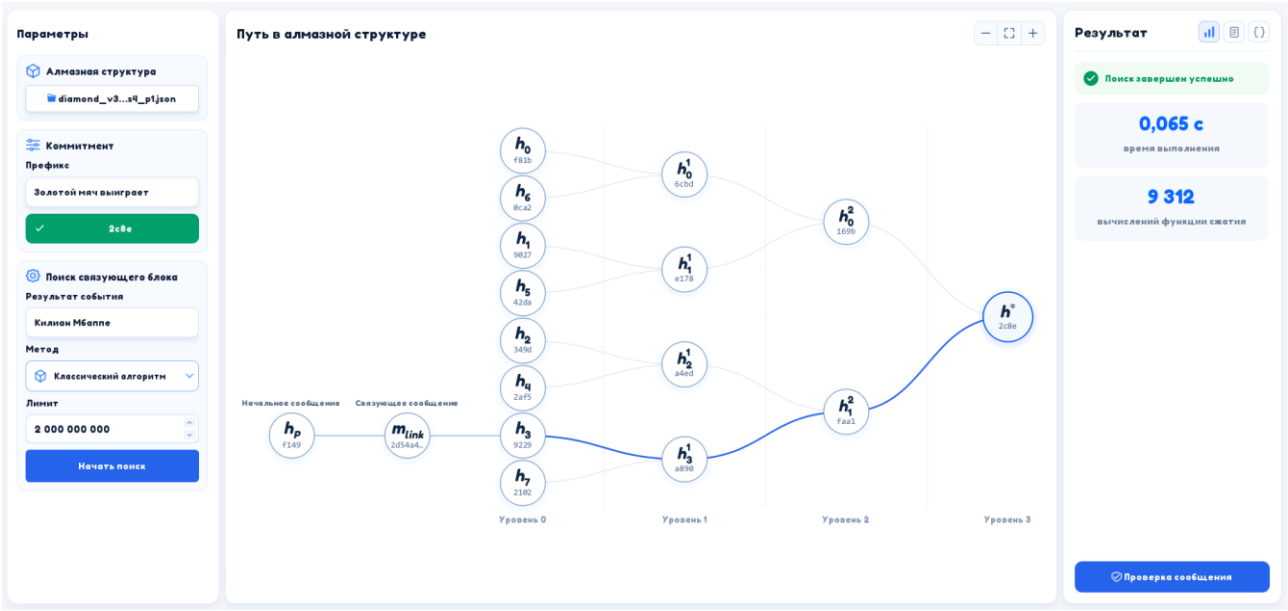


Рисунок 3.6 – Результат классической herding-атаки при $n = 16$, $k = 3$, $s = 4$

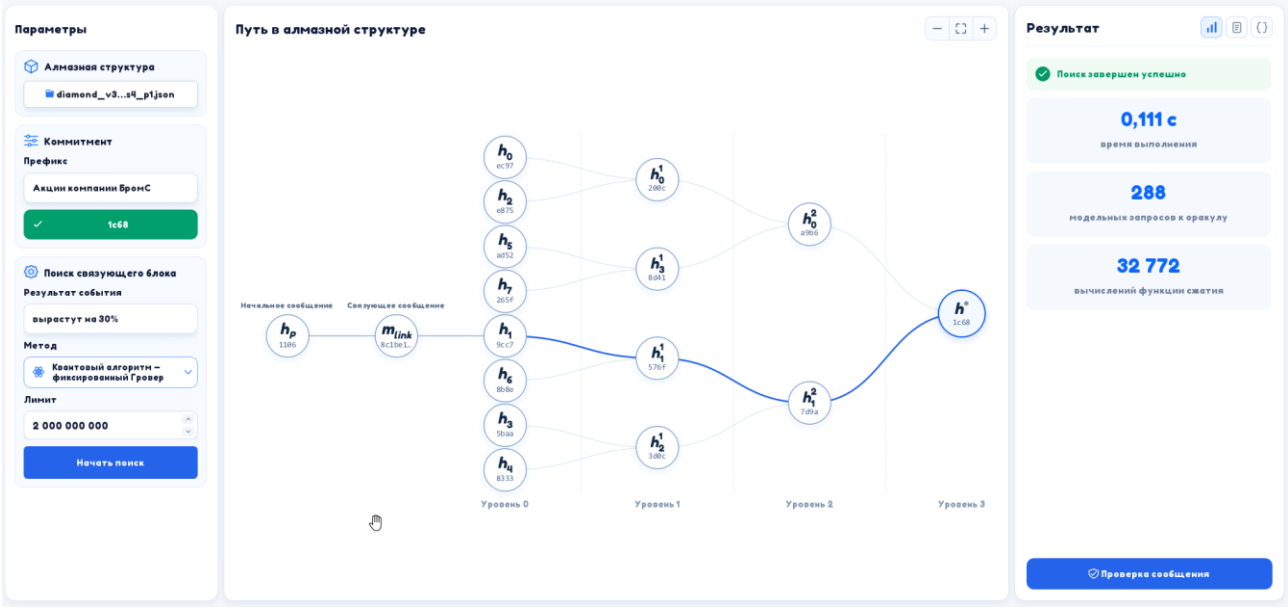


Рисунок 3.7 – Результат herding-атаки с фиксированным алгоритмом Гровера при $n = 16$, $k = 3$, $s = 4$

3 Проведена квантовая herding-атака с использованием адаптивного алгоритма BBHT и алмазной структуры из примера 3 раздела 3.3 при $n = 16$, $k = 3$, $s = 4$, сформирован префикс «Оперативная память подорожает на», а после

фиксации корневого хэш-кода введен результат события «30 процентов». Найденный блок обеспечил переход в один из листьев алмазной структуры, после чего программа восстановила путь до корневого хэш-кода и сформировала итоговое сообщение. При данном запуске зарегистрировано 1965 модельных запросов к квантовому оракулу и 32885 вычислений функции сжатия классическим симулятором. Результат представлен на рисунке 3.8.

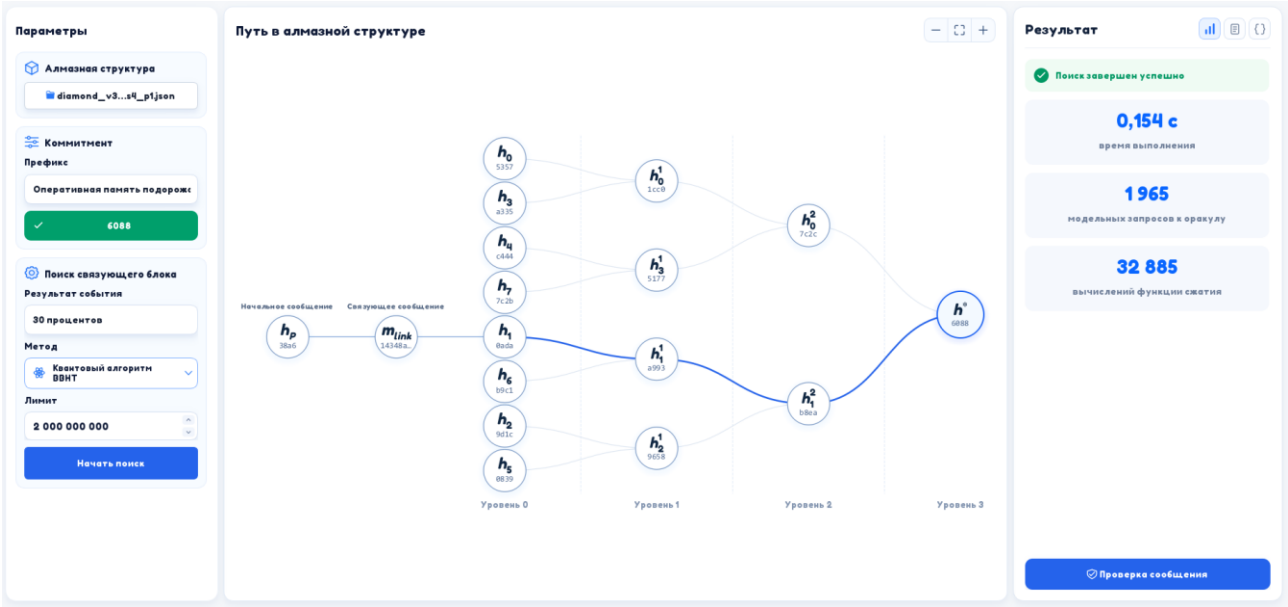


Рисунок 3.8 – Результат herding-атаки с адаптивным алгоритмом BBHT при $n = 16$, $k = 3$, $s = 4$

3.5 Анализ результатов экспериментального исследования

Для анализа экспериментальных результатов в программном модуле предусмотрена вкладка «Сравнение», представленная на рисунке 3.9. Она предназначена для сопоставления вычислительных затрат классического алгоритма, фиксированного Гровера и BBHT, а также для проверки соответствия полученных результатов теоретическим оценкам сложности. Листинг реализации вкладки «Сравнение» на языке Python представлен в приложении М.

В верхней части вкладки находится список выбора графика. С помощью кнопки настроек пользователь может выбрать этап herding-атаки, задать параметры n, k, s и другие условия отбора экспериментальных результатов. Выбранный график автоматически отображается в центральной части окна.

Кнопка экспорта позволяет сохранить его для дальнейшего использования.

В нижней части вкладки расположена таблица «Эксперименты», содержащая сведения о проведенных запусках, включая используемый алгоритм, параметры n, k, s и вычислительные затраты. При выборе записи в панели «Выбранный эксперимент» отображаются ее подробные результаты.

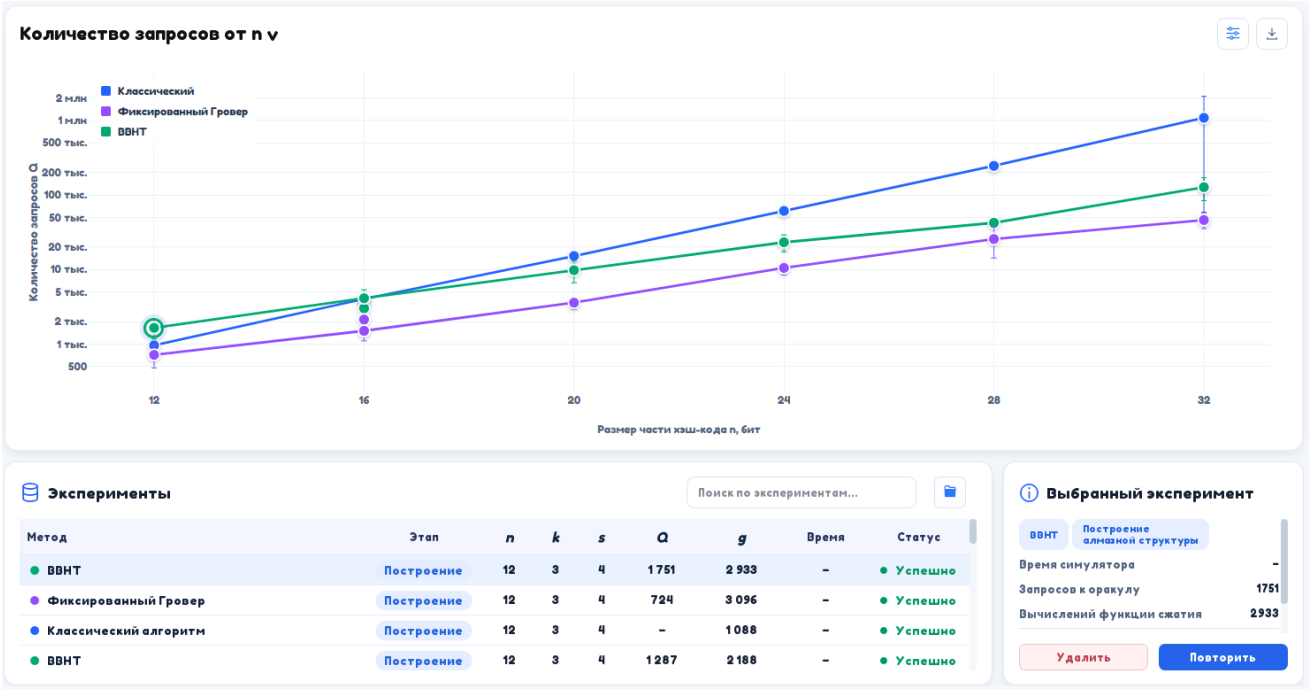


Рисунок 3.9 – Интерфейс программного модуля. Вкладка «Сравнение»

Для каждой сравниваемой конфигурации выполнялась серия независимых запусков. При расчете результатов, представленных на рисунках 3.10–3.12 и в таблицах 3.1–3.3, учитывались только успешно завершенные запуски. Значения на графиках представляют среднее арифметическое количества запросов.

Для оценки эффективности реализованных алгоритмов проведено сравнение классической herding-атаки, фиксированного алгоритма Гровера и адаптивного алгоритма BBHT. Исследование выполнено для функции хэширования ГОСТ Р 34.11–2012 с использованием алмазной структуры, построенной на частях промежуточных хэш-кодов и контрольных сумм.

Основным показателем сравнения является количество запросов Q: для классического метода – число вычислений функции сжатия, для квантовых – число модельных запросов к оракулу. Результаты рассматриваются отдельно для построения алмазной структуры, поиска связывающего блока и полной herding-

атаки.

Согласно теории, сложность классического построения алмазной структуры оценивается как $Q_{\text{кл}} = \tilde{O}(2^{(n+k)/2})$, а квантового построения – $Q_{\text{кв}} = \tilde{O}(2^{(n+2k)/3})$, где символ $\tilde{O}$ обозначает, что полиномиальные множители не учитываются.

При фиксированной глубине k теоретические показатели роста сложности по n составляют $\frac{1}{2}$ для классического метода и $\frac{1}{3} \approx 0.3333$ для квантовых методов. На рисунке 3.10 показана зависимость количества запросов от n при $k = 4$ и $s = 4$.

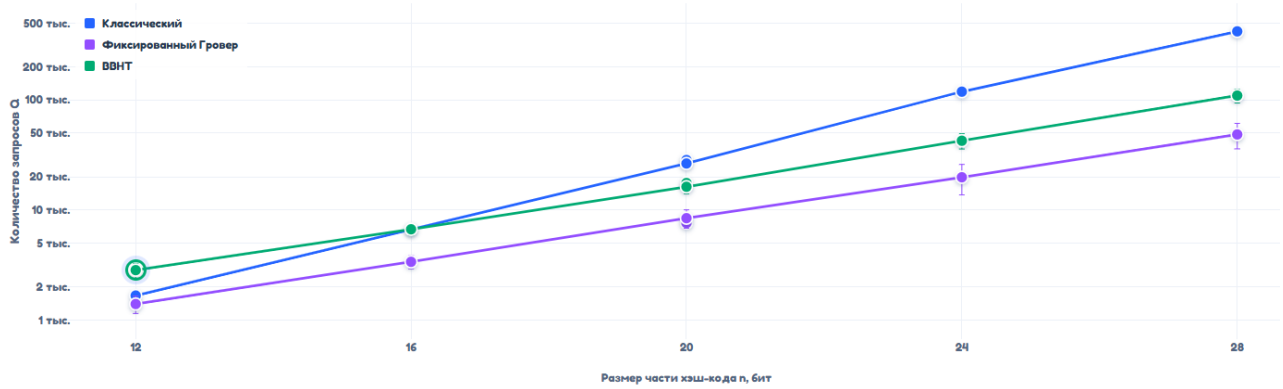


Рисунок 3.10 – Зависимость количества запросов при построении алмазной структуры от n

Полученные результаты показывают, что затраты классического алгоритма растут быстрее. При увеличении n с 16 до 28 бит количество его запросов возросло приблизительно в 63 раза, фиксированного Гровера – в 14 раз, BBHT – в 16 раз. Теоретически для такого изменения n ожидается рост примерно в 64 и 16 раз соответственно.

Для более точного сравнения рассчитан экспериментальный показатель α из зависимости $\log_2 Q = \alpha n + C$, где α характеризует скорость роста количества запросов, а C – постоянная величина. Теоретические и экспериментальные показатели роста сложности представлены в таблице 3.1.

Полученные показатели близки к теоретическим. При этом фиксированный Гровер в исследованных условиях потребовал меньше модельных запросов, чем BBHT. Таким образом, результаты согласуются с

ожидаемой зависимостью сложности построения алмазной структуры порядка $2^{\frac{n}{3}}$ для квантовых методов при фиксированном k.

Таблица 3.1 – Сравнение теоретических и экспериментальных показателей роста сложности построения алмазной структуры

Алгоритм	Теория	Эксперимент
Классический	0,500000	0,502712
Фиксированный Гровер	0,333333	0,319712
BBHT	0,333333	0,330225

После построения алмазной структуры необходимо найти блок, переводящий промежуточный хэш-код сообщения в одно из 2^k начальных состояний структуры.

Теоретическая сложность классического поиска составляет $Q_{\text{кл}} = O(2^{n-k})$, а квантового – $Q_{\text{кв}} = O\left(2^{\frac{n-k}{2}}\right)$. Следовательно, при фиксированном k теоретические показатели роста равны 1 для классического поиска и 0,5 для квантового.

На рисунке 3.11 представлены результаты поиска связывающего блока при $k = 4$ и $s = 4$.

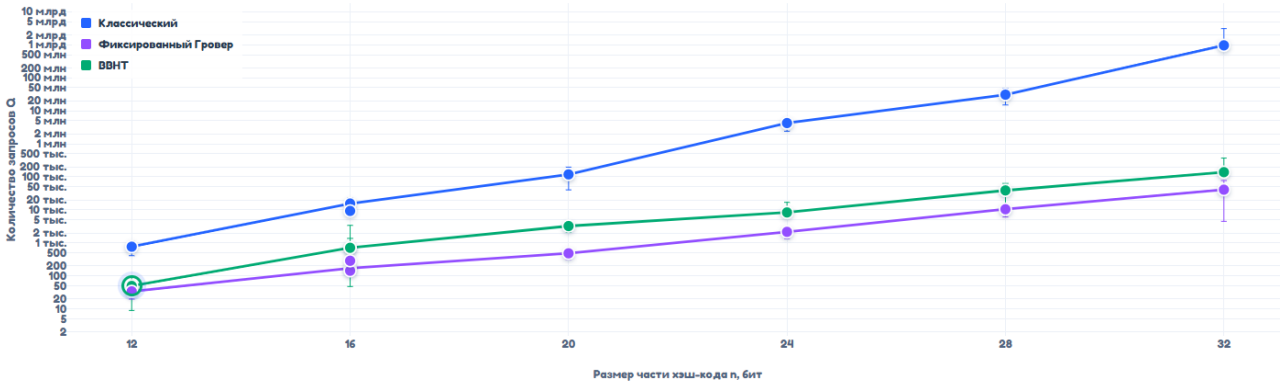


Рисунок 3.11 – Зависимость количества запросов при поиске связывающего блока от n

При $n = 28$ классический алгоритм потребовал в среднем 11217306 запросов, фиксированный Гровер – 7077, а BBHT – 15374. Таким образом, фиксированный Гровер использовал приблизительно в 1585 раз меньше

запросов, чем классический алгоритм, а BBHT – в 730 раз меньше. Экспериментальные показатели роста представлены в таблице 3.2.

Таблица 3.2 – Сравнение теоретических и экспериментальных показателей роста сложности поиска связывающего блока

Алгоритм	Теория	Эксперимент
Классический	1,000000	0,914621
Фиксированный Гровер	0,500000	0,508041
BBHT	0,500000	0,509511

Показатели обоих квантовых методов практически совпадают с теоретическим значением 0,5. У классического алгоритма наблюдается некоторое отклонение, связанное в том числе со случайным количеством проверок до нахождения подходящего блока. Наиболее заметное преимущество квантовых методов по числу запросов наблюдается при поиске связывающего блока.

Полная herding-атака включает построение алмазной структуры и поиск связывающего блока. Ее общие затраты определяются выражением $Q_{\text{полн}} = Q_{\text{алмаз}} + Q_{\text{связь}}$.

На рисунке 3.12 представлена зависимость суммарного количества запросов от n при $k = 4$ и $s = 4$.

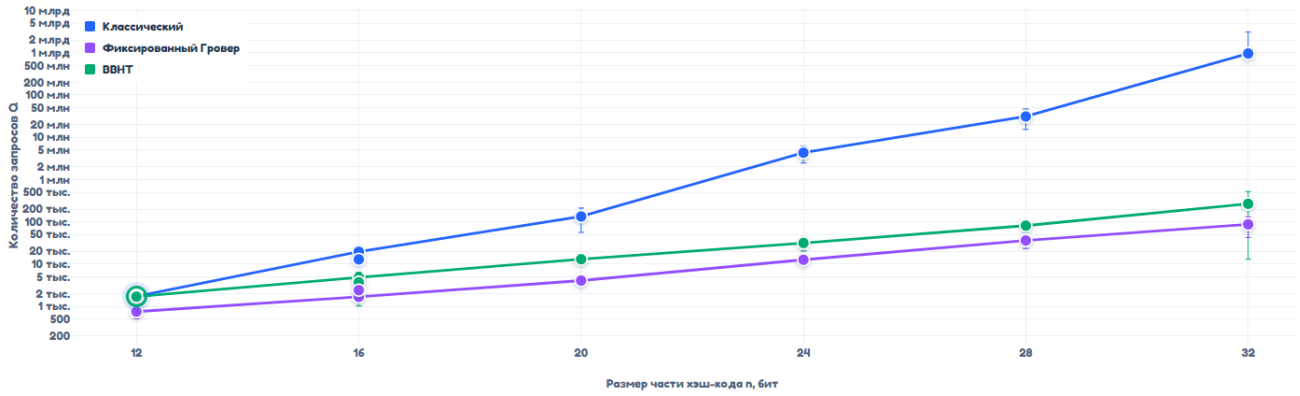


Рисунок 3.12 – Зависимость количества запросов полной herding-атаки от n при $k = 4$ и $s = 4$.

Для примера рассмотрим результаты при $n = 24$, которые представлены в таблице 3.3.

Таблица 3.3 – Распределение запросов между этапами полной herding-атаки при $n = 24$, $k = 4$ и $s = 4$

Алгоритм	Алмаз	Связывающий блок	Всего
Классический	118973	889651	1008624
Фиксированный Гровер	19848	1690	21538
ВВНТ	42610	5814	48424

Для количественной оценки криптостойкости относительно herding-атаки используется трудоемкость ее выполнения, выраженная количеством запросов Q. В логарифмическом представлении она определяется как $B = log_2 Q$. При $n = 24$, $k = 4$ и $s = 4$ получены значения $B \approx 19{,}94$ для классического алгоритма, $B \approx 14{,}39$ для фиксированного Гровера и $B \approx 15{,}56$ для ВВНТ.

Фиксированный Гровер потребовал приблизительно в 46,8 раза меньше запросов, чем классический алгоритм, а ВВНТ – в 20,8 раза меньше.

При этом распределение затрат между этапами существенно различается. У классического алгоритма около 88% запросов приходится на поиск связывающего блока, тогда как у фиксированного Гровера около 92%, а у ВВНТ около 88% запросов затрачивается на построение алмазной структуры.

На суммарные затраты также влияет глубина k. При ее увеличении построение алмазной структуры усложняется, но поиск связывающего блока становится проще.

При $n = 24$ среди исследованных значений минимальные затраты классического алгоритма получены при $k = 6$, а обоих квантовых методов – при $k = 3$. Согласно теории, суммарная сложность классической атаки оценивается как $Q_{\text{кл}} = \tilde{O}\left(2^{\frac{n+k}{2}} + 2^{n-k}\right)$, а квантовой – $Q_{\text{кв}} = \tilde{O}\left(2^{\frac{n+2k}{3}} + 2^{\frac{n-k}{2}}\right)$.

При выборе глубины, уравновешивающей затраты двух этапов, получаются следующие теоретические оценки:

- классическая атака: $k \approx n/3$, сложность $\tilde{O}\left(2^{\frac{2n}{3}}\right)$;

• квантовая атака: $k \approx n/7$, сложность $\tilde{O}\left(2^{\frac{3n}{7}}\right)$.

Теоретические показатели роста полной атаки составляют $2/3 \approx 0{,}6667$ и $3/7 \approx 0{,}4286$ соответственно [25].

ЗАКЛЮЧЕНИЕ

В ходе выполнения дипломной работы были исследованы классическая и квантовая herding-атаки, рассмотрены алгоритмы построения алмазной структуры и поиска связывающего блока, а также получены теоретические оценки их вычислительной сложности.

Для практического исследования разработан программный модуль классической и квантовой herding-атаки с использованием функции сжатия ГОСТ Р 34.11–2012. В программе реализованы классический алгоритм, фиксированный алгоритм Гровера и адаптивный алгоритм BBHT. Программный модуль позволяет строить алмазную структуру, выполнять herding-атаку и проверять полученные результаты.

Экспериментальное исследование показало, что при увеличении размера части хэш-кода количество запросов классического алгоритма растет быстрее, чем у квантовых методов. Полученные показатели роста сложности построения алмазной структуры и поиска связывающего блока согласуются с теоретическими оценками. В ходе эксперимента фиксированный Гровер оказался экономнее BBHT по числу модельных запросов.

Таким образом, разработан программный модуль для исследования классической и квантовой herding-атак с использованием функции сжатия ГОСТ Р 34.11–2012 и проведено экспериментальное сравнение алгоритмов. Поставленные задачи выполнены, цель дипломной работы достигнута.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1 Hamlin, B. Quantum Security of Hash Functions and Property-Preservation of Iterated Hashing [Электронный ресурс] / B. Hamlin, F. Song // Post-Quantum Cryptography : 10th International Conference, PQCrypto 2019. – Lecture Notes in Computer Science. – Cham : Springer, 2019. – Загл. с экрана. – Яз. англ.

2 Матюхин, Д. В. Обзор результатов анализа хэш-функции ГОСТ Р 34.11–2012. 10 лет спустя [Электронный ресурс] / Д. В. Матюхин, Г. Б. Маршалко, Д. А. Столовник, Д. Б. Фомин // XXVII научно-практическая конференция «РусКрипто'2025». – 2025. – URL: https://ruscrypto.ru/resource/archive/rc2025/files/02_matykhin_marshalko_fomin_stolovnik.pdf (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. рус.

3 Денисенко, Д. В. О реализации хэш-функции ГОСТ 34.11–2018 в виде квантовой схемы [Электронный ресурс] / Д. В. Денисенко, В. И. Рудской // XXIV научно-практическая конференция «РусКрипто'2022». – 2022. – URL: https://ruscrypto.ru/resource/archive/rc2022/files/02_denisenko_rudskoy.pdf (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. рус.

4 Kelsey, J. Herding Hash Functions and the Nostradamus Attack [Электронный ресурс] / J. Kelsey, T. Kohno. – National Institute of Standards and Technology, CSE Department, UC San Diego. – 2006. – 19 с. – URL: https://www.iacr.org/archive/eurocrypt2006/40040184/40040184.pdf (дата обращения: 26.04.2025). – Загл. с экрана. – Яз. англ.

5 Dong, X. Improved Meet-in-the-Middle Nostradamus Attacks on AES-like Hashing [Электронный ресурс] / X. Dong, J. Guo, S. Li, P. Pham, T. Zhang // IACR Transactions on Symmetric Cryptology. – 2024. – URL: https://doi.org/10.46586/tosc.v2024.i1.158-187 (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. англ.

6 Салий, В. Н. Криптографические методы и средства защиты информации: учебное пособие [Электронный ресурс] / В. Н. Салий // Саратовский национальный исследовательский государственный университет

имени Н. Г. Чернышевского [Электронный ресурс]. – Саратов, 2017. – 43 с. : ил., табл. – URL: http://elibrary.sgu.ru/uch_lit/622.pdf (дата обращения: 26.04.2025). – Загл. с экрана. – Яз. рус.

7 Ростовцев, А. Г. Теоретическая криптография [Электронный ресурс] / А. Г. Ростовцев, Е. Б. Маховенко. – СПб. : Профессионал, 2004. – 479 с. – Загл. с экрана. – Яз. рус.

8 Ищукова, Е. А. Криптографические основы блокчейн-технологий [Электронный ресурс] / Е. А. Ищукова, С. П. Панасенко, К. С. Романенко, В. Д. Салманов. – М. : ДМК Пресс, 2022. – 302 с. – Загл. с экрана. – Яз. рус.

9 ГОСТ Р 34.11–2012. Информационная технология. Криптографическая защита информации. Функция хэширования [Электронный ресурс] // Технический комитет по стандартизации «Криптографическая защита информации» [Электронный ресурс]. – URL: https://tc26.ru/standarts/natsionalnye-standarty/gost-r-34-11-2012-informatsionnaya-tekhnologiya-kriptograficheskaya-zashchita-informatsii-funktsiya-kheshirovaniya.html (дата обращения: 26.04.2025). – Загл. с экрана. – Яз. рус.

10 Szydlo, M. Merkle Tree Traversal in Log Space and Time [Электронный ресурс] / M. Szydlo. – Advances in Cryptology – EUROCRYPT 2004. – Berlin, Heidelberg: Springer. – 2004. – 13 с. – Загл. с экрана. – Яз. англ.

11 Gusfield, D. Algorithms on Strings, Trees, and Sequences [Электронный ресурс] / D. Gusfield. – Cambridge (UK): Cambridge University Press. – 1997. – 545 с. – Загл. с экрана. – Яз. англ.

12 Rjasko, M. On Chosen Target Forced Prefix Preimage Resistance [Электронный ресурс] / M. Rjasko. – Tatra Mt. Math. Publ., 47. – 2010. – 20 с. – URL: https://www.sav.sk/journals/uploads/0317155007rjasko.pdf (дата обращения: 25.05.2025). – Загл. с экрана. – Яз. англ.

13 ISO/IEC 4879:2024. Information technology – Quantum computing – Vocabulary [Электронный ресурс]. – Geneva : ISO, 2024. – 14 p. – Загл. с экрана. – Яз. англ.

14 ГОСТ Р 57257–2016/ISO/TS 80004-12:2016. Нанотехнологии. Часть 12. Квантовые явления. Термины и определения [Электронный ресурс] // Федеральное агентство по техническому регулированию и метрологии [Электронный ресурс]. – URL: https://protect.gost.ru/gost/details/52e89cef-e59c-4a8a-8d55-1783103e55df (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. рус.

15 ГОСТ Р 34.14–2025. Информационная технология. Криптографическая защита информации. Термины и определения [Электронный ресурс] // Федеральное агентство по техническому регулированию и метрологии [Электронный ресурс]. – https://protect.gost.ru/gost/details/ef581a55-0108-46cc-831f-8715ee8b42a6 (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. рус.

16 Grover, L. K. A Fast Quantum Mechanical Algorithm for Database Search [Электронный ресурс] / L. K. Grover. – Proceedings of the Twenty-Eighth Annual ACM Symposium on Theory of Computing. – New York : ACM, 1996. – Загл. с экрана. – Яз. англ.

17 Bao, Z. Evaluating the Security of Merkle-Damgård Hash Functions and Combiners in Quantum Settings [Электронный ресурс] / Z. Bao, J. Guo, S. Li, P. Pham. – Network and System Security : 16th International Conference, NSS 2022. – Lecture Notes in Computer Science. – Vol. 13787. – Springer, 2022. – URL: https://eprint.iacr.org/2022/1058.pdf (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. англ.

18 Streebog: открытый конкурс научных работ по исследованию хэш-функции ГОСТ Р 34.11–2012 [Электронный ресурс]. – Москва, 2013. – Загл. с экрана. – Яз. рус.

19 Zhang, Z. Classical and Quantum Meet-in-the-Middle Nostradamus Attacks on AES-Like Hashing [Электронный ресурс] / Z. Zhang, S. Sun, C. Wang, L. Hu. – IACR Transactions on Symmetric Cryptology. – 2023. – 28 с. – URL: https://eprint.iacr.org/2023/772 (дата обращения: 26.04.2025). – Загл. с экрана. – Яз. англ.

20 Blackburn, S. R. On the Complexity of the Herding Attack and Some Related Attacks on Hash Functions [Электронный ресурс] / S. R. Blackburn, D. R. Stinson, J. Upadhyay // Designs, Codes and Cryptography. – 2012. – Загл. с экрана. – Яз. англ.

21 Boyer, M. Tight Bounds on Quantum Searching [Электронный ресурс] / M. Boyer, G. Brassard, P. Høyer, A. Tapp // Fortschritte der Physik. – 1998. – Загл. с экрана. – Яз. англ.

22 Nielsen, M. A. Quantum Computation and Quantum Information. 10th Anniversary Edition [Электронный ресурс] / M. A. Nielsen, I. L. Chuang. – Cambridge : Cambridge University Press, 2010. – Загл. с экрана. – Яз. англ.

23 Grover's Algorithm [Электронный ресурс] // IBM Quantum Documentation. – URL: https://quantum.cloud.ibm.com/docs/en/tutorials/grovers-algorithm (дата обращения: 03.10.2026). – Загл. с экрана. – Яз. англ.

24 Brassard, G. Quantum Amplitude Amplification and Estimation [Электронный ресурс] / G. Brassard, P. Høyer, M. Mosca, A. Tapp // Contemporary Mathematics. – 2002. – Vol. 305. – Загл. с экрана. – Яз. англ.

25 Benedikt, B. J. Nostradamus Goes Quantum / B. J. Benedikt, M. Fischlin, M. Huppert // Advances in Cryptology – ASIACRYPT 2022. – Lecture Notes in Computer Science. – Vol. 13793. – Загл. с экрана. – Яз. англ.

ПРИЛОЖЕНИЕ А

Реализация алгоритма Гровера на языке Python

```
from __future__ import annotations

import math
import random
import os
import time

from .statevector_limits import check_resources, check_deadline,
SIMULATION_TIMEOUT
from dataclasses import dataclass
from typing import Callable, Iterable, Optional


@dataclass(frozen=True)
class GroverResult:
    measured_index: Optional[int]
    is_marked: bool
    domain_size: int
    marked_count: int
    iterations: int
    quantum_queries: int
    success_probability: float
    norm_error: float
    statevector_elements_processed: int = 0
    statevector_seconds: float = 0.0


def optimal_grover_iterations(domain_size: int, marked_count: int = 1) -
> int:
    """Вычисляет целое число итераций Гровера для амплитудного усиления
помеченных состояний."""
    if domain_size <= 0:
        raise ValueError("domain_size должен быть положительным.")
    if marked_count < 0 or marked_count > domain_size:
        raise ValueError("marked_count должен лежать в диапазоне
0..domain_size.")
    if marked_count == 0 or marked_count == domain_size:
        return 0
    theta = math.asin(math.sqrt(marked_count / domain_size))
    return max(0, round(math.pi / (4.0 * theta) - 0.5))


def grover_success_probability(
    domain_size: int,
    marked_count: int,
```

```
    iterations: int,
) -> float:
    if domain_size <= 0 or not 0 <= marked_count <= domain_size:
        raise ValueError("Некорректные размеры пространства Grover.")
    if iterations < 0:
        raise ValueError("iterations должен быть неотрицательным.")
    if marked_count == 0:
        return 0.0
    theta = math.asin(math.sqrt(marked_count / domain_size))
    return math.sin((2 * iterations + 1) * theta) ** 2


def simulate_grover_from_marked(
    domain_size: int,
    marked_indices: Iterable[int],
    *,
    iterations: Optional[int] = None,
    rng: Optional[random.Random] = None,
    max_statevector_size: Optional[int] = None,
    final_amplitudes: Optional[list[float]] = None,
) -> GroverResult:
    """Моделирует поиск Гровера фазовым оракулом и диффузией вектора
состояния на классическом компьютере."""
    if domain_size <= 0:
        raise ValueError("domain_size должен быть положительным.")
    if max_statevector_size is not None and domain_size >
max_statevector_size:
        raise ValueError(
            f"State-vector из {domain_size} состояний превышает лимит "
            f"{max_statevector_size}. Уменьшите n/depth."
        )
    check_resources(domain_size, 0, python=True)
    marked = frozenset(marked_indices)
    if any(index < 0 or index >= domain_size for index in marked):
        raise ValueError("Отмеченный индекс находится вне пространства
поиска.")
    marked_count = len(marked)
    if iterations is None:
        iterations = optimal_grover_iterations(domain_size,
marked_count)
    if iterations < 0:
        raise ValueError("iterations должен быть неотрицательным.")

    check_resources(domain_size, iterations, python=True)
    started = time.monotonic()
    timeout = min(SIMULATION_TIMEOUT,
float(os.environ.get('GROVER_TIMEOUT_SECONDS', SIMULATION_TIMEOUT)))
    if not math.isfinite(timeout) or timeout <= 0:
        raise ValueError('Лимит времени моделирования должен быть
положительным.')
```

```
    amplitude = 1.0 / math.sqrt(domain_size)
    state = [amplitude] * domain_size
    for _ in range(iterations):
        check_deadline()
        if time.monotonic() - started > timeout:
            raise TimeoutError('Превышен лимит времени моделирования
вектора амплитуд.')
        for index in range(domain_size):
            if index in marked:
                state[index] = -state[index]
        mean = math.fsum(state) / domain_size
        for index, value in enumerate(state):
            state[index] = 2.0 * mean - value

    probabilities = [value * value for value in state]
    probability_sum = math.fsum(probabilities)
    norm_error = abs(probability_sum - 1.0)
    success_probability = math.fsum(probabilities[index] for index in
marked)

    generator = rng or random.Random()
    threshold = generator.random() * probability_sum
    cumulative = 0.0
    measured_index = domain_size - 1
    for index, probability in enumerate(probabilities):
        cumulative += probability
        if threshold < cumulative:
            measured_index = index
            break

    if final_amplitudes is not None:
        final_amplitudes[:] = state
    check_deadline()
    if time.monotonic() - started > timeout:
        raise TimeoutError('Превышен лимит времени моделирования вектора
амплитуд.')
    return GroverResult(
        measured_index=measured_index,
        is_marked=measured_index in marked,
        domain_size=domain_size,
        marked_count=marked_count,
        iterations=iterations,
        quantum_queries=iterations,
        success_probability=success_probability,
        norm_error=norm_error,
        statevector_elements_processed=domain_size * (2 + 3 *
iterations) + measured_index + 1,
        statevector_seconds=time.monotonic() - started,
    )
```

```
def simulate_grover_search(
    domain_size: int,
    oracle: Callable[[int], bool],
    **kwargs,
) -> GroverResult:
    """Формирует помеченные индексы оракула для моделирования поиска
Гровера."""
    marked = [index for index in range(domain_size) if oracle(index)]
    return simulate_grover_from_marked(domain_size, marked, **kwargs)
```

ПРИЛОЖЕНИЕ Б

Моделирование алгоритма Гровера с фиксированным числом итераций на языке C++

```
#pragma once

#include <algorithm>
#include <chrono>
#include <cmath>
#include <cstdint>
#include <cstdlib>
#include <limits>
#include <random>
#include <stdexcept>
#include <string>
#include <vector>

// Выполняет фазовый оракул, диффузию и измерение полного вещественного
вектора амплитуд.
namespace grover_statevector {

struct Limits {
    uint64_t max_states = 1048576;
    uint64_t max_work = 200000000;
    uint64_t max_memory_bytes = 536870912;
    double timeout_seconds = 30.0;
};

inline Limits current_limits() {
    Limits limits;
    auto lower = [](const char* name, uint64_t maximum) {
        const char* raw = std::getenv(name);
        if (!raw) return maximum;
        const std::string text(raw);
        if (text.empty() || text.front() == '-') throw
std::runtime_error("Лимит ресурсов должен быть положительным.");
        size_t consumed = 0;
        const uint64_t value = std::stoull(text, &consumed);
        if (!value || consumed != text.size()) throw
std::runtime_error("Некорректный лимит ресурсов.");
        return std::min(value, maximum);
    };
    limits.max_states = lower("GROVER_MAX_STATES", limits.max_states);
    limits.max_work = lower("GROVER_MAX_WORK", limits.max_work);
    limits.max_memory_bytes = lower("GROVER_MAX_MEMORY_BYTES",
limits.max_memory_bytes);
    if (const char* raw = std::getenv("GROVER_TIMEOUT_SECONDS")) {
```

```
        const double seconds = std::stod(raw);
        if (!std::isfinite(seconds) || seconds <= 0) throw
std::runtime_error("Время симуляции должно быть положительным.");
        limits.timeout_seconds = std::min(seconds,
limits.timeout_seconds);
    }
    return limits;
}

inline void check_resources(uint64_t states, uint64_t iterations,
uint64_t other_bytes = 0) {
    const auto limits = current_limits();
    if (!states) throw std::runtime_error("Пространство вектора амплитуд
должно быть непустым.");
    if (states > limits.max_states || states >
std::numeric_limits<size_t>::max())
        throw std::runtime_error("Размер вектора амплитуд S=" +
std::to_string(states) + " превышает лимит " +
std::to_string(limits.max_states) + ".");
    if (iterations > limits.max_work / states)
        throw std::runtime_error("Объем симуляции S × iterations
превышает лимит " + std::to_string(limits.max_work) + ".");
    if (other_bytes > limits.max_memory_bytes || states >
(limits.max_memory_bytes - other_bytes) / 9)
        throw std::runtime_error("Оценка памяти симулятора превышает
лимит " + std::to_string(limits.max_memory_bytes) + " байт.");
}

class Deadline {
public:
    Deadline(): start_(std::chrono::steady_clock::now()),
seconds_(current_limits().timeout_seconds) {}
    void check() const {
        if (elapsed() > seconds_) throw std::runtime_error("Превышен
лимит времени моделирования вектора амплитуд.");
    }
    double elapsed() const { return
std::chrono::duration<double>(std::chrono::steady_clock::now() -
start_).count(); }
private:
    std::chrono::steady_clock::time_point start_;
    double seconds_;
};

// Суммирует значения с компенсацией ошибок округления без изменения
амплитуд.
struct CompensatedSum {
    double value = 0.0, correction = 0.0;
    void add(double item) {
        const double adjusted = item - correction;
```

```
        const double next = value + adjusted;
        correction = (next - value) - adjusted;
        value = next;
    }
};

inline void apply_phase_oracle(std::vector<double>& amplitudes, const
std::vector<uint8_t>& marked) {
    if (amplitudes.size() != marked.size()) throw
std::runtime_error("Размеры оракула и вектора амплитуд различаются.");
    for (size_t i = 0; i < amplitudes.size(); ++i) if (marked[i])
amplitudes[i] = -amplitudes[i];
}

inline void apply_diffusion(std::vector<double>& amplitudes) {
    if (amplitudes.empty()) throw std::runtime_error("Диффузия требует
непустого вектора амплитуд.");
    CompensatedSum sum;
    for (double value : amplitudes) sum.add(value);
    const double twice_mean = 2.0 * sum.value /
static_cast<double>(amplitudes.size());
    for (double& value : amplitudes) value = twice_mean - value;
}

struct Result {
    size_t measured_index = 0;
    bool is_marked = false;
    uint64_t marked_count = 0;
    uint64_t iterations = 0;
    uint64_t oracle_queries = 0;
    double success_probability = 0.0;
    double norm_error = 0.0;
    uint64_t statevector_elements_processed = 0;
    double statevector_seconds = 0.0;
};

inline Result simulate_grover_statevector(const std::vector<uint8_t>&
marked, uint64_t iterations,
                                          std::mt19937_64& rng,
std::vector<double>* final_amplitudes = nullptr) {
    const uint64_t states = static_cast<uint64_t>(marked.size());
    check_resources(states, iterations, final_amplitudes ? states *
sizeof(double) : 0);
    Deadline deadline;
    Result result;
    result.iterations = iterations;
    result.oracle_queries = iterations + 1;
    for (uint8_t flag : marked) {
        if (flag > 1) throw std::runtime_error("Признак помеченного
элемента должен быть 0 или 1.");
```

```
            result.marked_count += flag;
        }
        std::vector<double> amplitudes(static_cast<size_t>(states), 1.0 /
std::sqrt(static_cast<double>(states)));
        for (uint64_t iteration = 0; iteration < iterations; ++iteration) {
            deadline.check();
            apply_phase_oracle(amplitudes, marked);
            apply_diffusion(amplitudes);
        }
        CompensatedSum probability_sum, marked_probability;
        for (size_t i = 0; i < amplitudes.size(); ++i) {
            const double probability = amplitudes[i] * amplitudes[i];
            probability_sum.add(probability);
            if (marked[i]) marked_probability.add(probability);
        }
        result.success_probability = marked_probability.value;
        result.norm_error = std::abs(probability_sum.value - 1.0);
        if (!std::isfinite(probability_sum.value) || probability_sum.value
<= 0 || result.norm_error > 1e-9)
            throw std::runtime_error("Нарушена нормировка вектора
амплитуд.");
        std::uniform_real_distribution<double> unit(0.0, 1.0);
        const double threshold = unit(rng) * probability_sum.value;
        CompensatedSum cumulative;
        bool measured = false;
        for (size_t i = 0; i < amplitudes.size(); ++i) {
            cumulative.add(amplitudes[i] * amplitudes[i]);
            if (threshold < cumulative.value) {
                result.measured_index = i;
                result.is_marked = marked[i] != 0;
                measured = true;
                break;
            }
        }
        if (!measured) throw std::runtime_error("Не удалось измерить
нормированный вектор амплитуд.");
        result.statevector_elements_processed = states * (2 + 3 *
iterations) + result.measured_index + 1;
        if (final_amplitudes) *final_amplitudes = amplitudes;
        deadline.check();
        result.statevector_seconds = deadline.elapsed();
        return result;
}

}
```

ПРИЛОЖЕНИЕ В

Листинг адаптивного алгоритма Гровера

```
#pragma once

#include <algorithm>
#include <cmath>
#include <cstdint>
#include <limits>
#include <random>
#include <stdexcept>
#include <vector>
#include <ostream>
#include <iomanip>
#include "grover_statevector.hpp"
#include "bbht_dense_statevector.hpp"
#include "grover_symmetric.hpp"
#include "grover_analytic.hpp"

// Реализует ограниченный адаптивный поиск BBHT без предварительного
знания числа решений.
namespace bbht_adaptive {

inline constexpr const char* search_protocol = "fixed-single-bbht-auto-
v1";

// Ограничивает эвристический резерв после роста окна до порядка
квадратного корня из S.
inline uint64_t effective_max_attempts(uint64_t states, double growth,
uint64_t cap = 128) {
    if (!states || !cap || !std::isfinite(growth) || !(growth > 1.0 &&
growth < 4.0/3.0))
        throw std::runtime_error("BBHT: неверное пространство,
коэффициент роста или верхний предел попыток.");
    if (cap <= 8) return cap;
    const long double steps = 0.5L * std::log(static_cast<long
double>(states)) /
        std::log1p(static_cast<long double>(growth)-1.0L);
    if (steps >= static_cast<long double>(cap-8)) return cap;
    return std::min(cap, static_cast<uint64_t>(std::ceil(steps)) +
uint64_t{8});
}

struct Options {
    double growth = 1.2;
    uint64_t max_attempts = 128;
    uint64_t max_oracle_queries = 100000;
    uint64_t dense_max_states = 8192;
```

```
    double max_stage_seconds = 0.0;  // 0 = unlimited
};

struct Result {
    bool found = false;
    size_t measured_index = 0;
    uint64_t attempts = 0;
    uint64_t failures = 0;
    uint64_t oracle_queries = 0;
    uint64_t phase_oracle_queries = 0;
    uint64_t verification_queries = 0;
    uint64_t statevector_iterations = 0;
    uint64_t statevector_elements_processed = 0;
    uint64_t marked_count_diagnostic = 0;
    double success_probability_sum = 0.0;
    double norm_error_max = 0.0;
    double statevector_seconds = 0.0;
    bool exhausted = false;
    uint64_t search_messages = 0;
    double growth = 1.2;
    uint64_t effective_max_attempts = 0;
    const char* attempt_policy = "bbht_auto";
    const char* stop_reason = "attempt_limit";
};

// Записывает фактическую политику и причину остановки каждого поиска
без изменения счетчиков.
inline void write_diagnostics(std::ostream& stream, const
std::vector<Result>& searches) {
    const auto precision = stream.precision();
    stream << std::setprecision(17);
    stream << " SEARCH " << search_protocol << ' ' << searches.size();
    for (const auto& search : searches)
        stream << ' ' << search.attempt_policy << ' ' <<
search.search_messages << ' '
               << search.growth << ' ' << search.effective_max_attempts
<< ' '
               << search.attempts << ' ' << search.failures << ' ' <<
search.stop_reason;
    stream.precision(precision);
}

// Выполняет единственное измерение фиксированного Гровера независимо от
ограничений BBHT.
template<class Oracle, class Verifier>
inline Result fixed_single(const Oracle& marked, std::mt19937_64& rng,
Verifier&& verify) {
    if (marked.empty()) throw std::runtime_error("Фиксированный Гровер:
пространство поиска пусто.");
    const auto iterations =
```

```
grover_analytic::optimal_grover_iterations(marked.size(), 1);
    const auto simulation = marked.size() <=
grover_oracle::dense_max_states
        ? grover_statevector::simulate_grover_statevector(marked,
iterations, rng)
        : grover_symmetric::simulate(marked,
grover_oracle::count(marked), iterations, rng, true);
    const bool accepted =
static_cast<bool>(verify(simulation.measured_index));
    if (accepted != simulation.is_marked)
        throw std::runtime_error("Фиксированный Гровер: результат ГОСТ
не совпал с фазовым оракулом.");
    Result result;
    result.found = accepted;
    result.measured_index = simulation.measured_index;
    result.attempts = 1;
    result.failures = accepted ? 0 : 1;
    result.oracle_queries = simulation.oracle_queries;
    result.phase_oracle_queries = simulation.iterations;
    result.verification_queries = 1;
    result.statevector_iterations = simulation.iterations;
    result.statevector_elements_processed =
simulation.statevector_elements_processed;
    result.marked_count_diagnostic = simulation.marked_count;
    result.success_probability_sum = simulation.success_probability;
    result.norm_error_max = simulation.norm_error;
    result.statevector_seconds = simulation.statevector_seconds;
    result.exhausted = !accepted;
    result.search_messages = marked.size();
    result.effective_max_attempts = 1;
    result.attempt_policy = "fixed_single";
    result.stop_reason = accepted ? "found" : "unmarked_measurement";
    return result;
}

// Проверка выполняется внешним предикатом на реальном результате
функции сжатия.
template<class Oracle, class Verifier>
inline Result search(const Oracle& marked, std::mt19937_64& rng,
                     Verifier&& verify, const Options& options = {}) {
    if (marked.empty()) throw std::runtime_error("ВВНТ: пространство
поиска пусто.");
    if (!(options.growth > 1.0 && options.growth < 4.0 / 3.0) ||
!std::isfinite(options.growth) ||
        !options.max_attempts || !options.max_oracle_queries)
        throw std::runtime_error("ВВНТ: неверные ограничения или
коэффициент роста.");
    const bool dense = marked.size() <= options.dense_max_states;
    if(dense) grover_statevector::check_resources(marked.size(), 0);
    const auto M=grover_oracle::count(marked);
```

```
    Result result;
    result.search_messages = marked.size();
    result.growth = options.growth;
    result.effective_max_attempts =
effective_max_attempts(marked.size(), options.growth,
options.max_attempts);
    double width = 1.0;
    const double maximum = std::max(1.0,
std::sqrt(static_cast<double>(marked.size())));
    for (uint64_t attempt = 0; attempt < result.effective_max_attempts;
++attempt) {
        const uint64_t window = std::max<uint64_t>(1,
static_cast<uint64_t>(std::ceil(width)));
        std::uniform_int_distribution<uint64_t> choose(0, window - 1);
        const uint64_t iterations = choose(rng);
        if (iterations + 1 > options.max_oracle_queries -
result.oracle_queries) {
            result.exhausted = true;
            result.stop_reason = "query_limit";
            return result;
        }
        const auto sim = dense ?
bbht_dense_statevector::simulate(marked, iterations, rng)
                               : grover_symmetric::simulate(marked, M,
iterations, rng);
        const bool accepted =
static_cast<bool>(verify(sim.measured_index));
        if (accepted != sim.is_marked)
            throw std::runtime_error("BBHT: проверка через ГОСТ не
совпала с фазовым оракулом.");
        ++result.attempts;
        result.oracle_queries += sim.oracle_queries;
        result.phase_oracle_queries += sim.iterations;
        ++result.verification_queries;
        result.statevector_iterations += sim.iterations;
        result.statevector_elements_processed +=
sim.statevector_elements_processed;
        result.marked_count_diagnostic = sim.marked_count;
        result.success_probability_sum += sim.success_probability;
        result.norm_error_max = std::max(result.norm_error_max,
sim.norm_error);
        result.statevector_seconds += sim.statevector_seconds;
        if (accepted) {
            result.found = true;
            result.stop_reason = "found";
            result.measured_index = sim.measured_index;
            return result;
        }
        ++result.failures;
        width = std::min(maximum, width * options.growth);
```

```
    }
    result.exhausted = true;
    return result;
}

// Повторяет фиксированный поиск Гровера на той же таблице решений для
контрольного сравнения.
template<class Oracle, class Verifier>
inline Result fixed_repeated(const Oracle& marked, uint64_t iterations,
                             std::mt19937_64& rng, Verifier&& verify,
                             const Options& options = {}) {
    if (marked.empty() || !options.max_attempts ||
!options.max_oracle_queries)
        throw std::runtime_error("Fixed Grover: неверные параметры.");
    const bool dense = marked.size() <= options.dense_max_states;
    if(dense) grover_statevector::check_resources(marked.size(), 0);
    const auto M=grover_oracle::count(marked);
    Result result;
    for (uint64_t attempt = 0; attempt < options.max_attempts;
++attempt) {
        if (iterations >= options.max_oracle_queries -
result.oracle_queries) {
            result.exhausted = true;
            return result;
        }
        const auto sim = dense ?
bbht_dense_statevector::simulate(marked, iterations, rng)
                               : grover_symmetric::simulate(marked, M,
iterations, rng);
        const bool accepted =
static_cast<bool>(verify(sim.measured_index));
        if (accepted != sim.is_marked)
            throw std::runtime_error("Fixed Grover: проверка через ГОСТ
не совпала с оракулом.");
        ++result.attempts;
        result.oracle_queries += sim.oracle_queries;
        result.phase_oracle_queries += sim.iterations;
        ++result.verification_queries;
        result.statevector_iterations += sim.iterations;
        result.statevector_elements_processed +=
sim.statevector_elements_processed;
        result.marked_count_diagnostic = sim.marked_count;
        result.success_probability_sum += sim.success_probability;
        result.norm_error_max = std::max(result.norm_error_max,
sim.norm_error);
        result.statevector_seconds += sim.statevector_seconds;
        if (accepted) {
            result.found = true;
            result.measured_index = sim.measured_index;
            return result;
```

```
        }
        ++result.failures;
    }
    result.exhausted = true;
    return result;
}

} // namespace bbht_adaptive
```

ПРИЛОЖЕНИЕ Г

Листинг редуцированной модели ГОСТ на Python

```
from __future__ import annotations

import math
import random
import secrets
from dataclasses import dataclass
from typing import Sequence, Tuple

from HashCalculator import gost_compress


LEGACY_MODEL_NAME = "reduced-prefix-state-v2"
MODEL_NAME = "reduced-h-sigma-state-v3"
FORMAT_VERSION = 3
LEGACY_FORMAT_VERSION = 2
MAX_SIGMA_BITS = 64  # Оптимизированное ядро C++ поддерживает s ≤ 64.


def validate_truncate_bits(truncate_bits: int) -> None:
    if truncate_bits <= 0 or truncate_bits > 512 or truncate_bits % 4 !=
0:
        raise ValueError("truncate_bits должен быть кратен 4 и лежать в
диапазоне 4..512.")


def validate_sigma_bits(sigma_bits: int) -> None:
    if sigma_bits < 0 or sigma_bits > MAX_SIGMA_BITS:
        raise ValueError(f"sigma_bits должен лежать в диапазоне
0..{MAX_SIGMA_BITS}.")


def sigma_mask(sigma_bits: int) -> int:
    validate_sigma_bits(sigma_bits)
    return (1 << sigma_bits) - 1 if sigma_bits else 0


def _validate_block_hex(block: str) -> str:
    block = block.lower()
    if len(block) != 128:
        raise ValueError("Блок должен быть 512-битной hex-строкой длины
128 символов.")
    int(block, 16)
    return block
```

```
def block_low_bits(block: str, sigma_bits: int) -> int:
    """Возвращает младшие s бит 512-битного блока сообщения."""
    validate_sigma_bits(sigma_bits)
    if sigma_bits == 0:
        return 0
    block = _validate_block_hex(block)
    return int(block[-16:], 16) & sigma_mask(sigma_bits)


def update_sigma_reduced(sigma_value: int, block: str, sigma_bits: int)
-> int:
    """Вычисляет часть контрольной суммы (σ + m) mod 2^s."""
    validate_sigma_bits(sigma_bits)
    if sigma_bits == 0:
        return 0
    mask = sigma_mask(sigma_bits)
    return (int(sigma_value) + block_low_bits(block, sigma_bits)) & mask


def required_block_low_bits(current_sigma: int, target_sigma: int,
sigma_bits: int) -> int:
    validate_sigma_bits(sigma_bits)
    if sigma_bits == 0:
        return 0
    mask = sigma_mask(sigma_bits)
    return (int(target_sigma) - int(current_sigma)) & mask


def set_low_sigma_bits(block: str, required_low: int, sigma_bits: int) -
> str:
    """Устанавливает младшие s бит блока в правом конце
шестнадцатеричной записи."""
    validate_sigma_bits(sigma_bits)
    block = _validate_block_hex(block)
    if sigma_bits == 0:
        return block
    mask = sigma_mask(sigma_bits)
    value = int(block, 16)
    value = (value & ~mask) | (int(required_low) & mask)
    return f"{value:0128x}"


def constrained_random_block(
    current_sigma: int,
    target_sigma: int,
    sigma_bits: int,
    rng: random.Random | None = None,
) -> str:
    """Создает случайный 512-битный блок с заданным переходом части
контрольной суммы."""
```

```
    validate_sigma_bits(sigma_bits)
    if rng is None:
        block = secrets.token_hex(64)
    else:
        block = f"{rng.getrandbits(512):0128x}"
    required = required_block_low_bits(current_sigma, target_sigma,
sigma_bits)
    return set_low_sigma_bits(block, required, sigma_bits)


def process_sigma_blocks(blocks: Sequence[str], sigma_bits: int,
initial_sigma: int = 0) -> int:
    validate_sigma_bits(sigma_bits)
    sigma_value = int(initial_sigma) & sigma_mask(sigma_bits)
    for block in blocks:
        sigma_value = update_sigma_reduced(sigma_value, block,
sigma_bits)
    return sigma_value


def truncate_state(full_state: str, truncate_bits: int) -> str:
    validate_truncate_bits(truncate_bits)
    return full_state[:truncate_bits // 4].lower()


def embed_reduced_state(state: str, truncate_bits: int) -> str:
    validate_truncate_bits(truncate_bits)
    expected = truncate_bits // 4
    state = state.lower()
    if len(state) != expected:
        raise ValueError(f"Редуцированное состояние должно иметь
{expected} hex-символов.")
    int(state, 16)
    return state + "0" * (128 - expected)


def random_reduced_state(truncate_bits: int, rng: random.Random | None =
None) -> str:
    validate_truncate_bits(truncate_bits)
    nbytes = math.ceil(truncate_bits / 8)
    value = rng.getrandbits(truncate_bits) if rng is not None else
int.from_bytes(secrets.token_bytes(nbytes), "big") & ((1 <<
truncate_bits) - 1)
    return f"{value:0{truncate_bits // 4}x}"


@dataclass(frozen=True)
class DiamondState:
    """Хранит состояние из части хэш-кода h_n и части контрольной суммы
Σ mod 2^s."""
```

```
    h: str
    sigma: int = 0


@dataclass(frozen=True)
class ReducedGostModel:
    """Вычисляет первые n бит g_N дополненного состояния, обновляя часть
контрольной суммы отдельно."""

    truncate_bits: int
    rounds: int = 12

    def __post_init__(self) -> None:
        validate_truncate_bits(self.truncate_bits)
        if not (1 <= self.rounds <= 12):
            raise ValueError("rounds должен быть в диапазоне 1..12.")

    @property
    def zero_state(self) -> str:
        return "0" * (self.truncate_bits // 4)

    def transition(self, block: str, state: str, N: int) -> str:
        full_h = embed_reduced_state(state, self.truncate_bits)
        full_out = gost_compress(block, full_h, N, self.rounds)
        return truncate_state(full_out, self.truncate_bits)

    def transition_pair(
        self,
        block: str,
        state: DiamondState,
        N: int,
        sigma_bits: int,
    ) -> DiamondState:
        return DiamondState(
            self.transition(block, state.h, N),
            update_sigma_reduced(state.sigma, block, sigma_bits),
        )

    def process_blocks(self, blocks: Sequence[str], start_N: int = 0) ->
Tuple[str, int]:
        state = self.zero_state
        N = start_N
        for block in blocks:
            state = self.transition(block, state, N)
            N += 512
        return state, N

    def process_blocks_pair(
        self,
```

```
        blocks: Sequence[str],
        sigma_bits: int,
        start_N: int = 0,
        initial_sigma: int = 0,
    ) -> tuple[DiamondState, int]:
        state = DiamondState(self.zero_state, initial_sigma &
sigma_mask(sigma_bits))
        N = start_N
        for block in blocks:
            state = self.transition_pair(block, state, N, sigma_bits)
            N += 512
        return state, N
```

ПРИЛОЖЕНИЕ Д

Листинг редуцированной модели ГОСТ на C++

```
#pragma once

#include <array>
#include <cstddef>
#include <cstdint>
#include <stdexcept>
#include <string>

namespace gost_reduced_model {

using std::size_t;
using std::uint8_t;
using std::uint64_t;
using State = std::array<uint64_t, 8>;
inline constexpr uint8_t SBOX[256] = {
252,238,221,17,207,110,49,22,251,196,250,218,35,197,4,77,233,119,240,219
,147,46,153,186,23,54,241,187,20,205,95,193,249,24,101,90,226,92,239,33,
129,28,60,66,139,1,142,79,5,132,2,174,227,106,143,160,6,11,237,152,127,2
12,211,31,235,52,44,81,234,200,72,171,242,42,104,162,253,58,206,204,181,
112,14,86,8,12,118,18,191,114,19,71,156,183,93,135,21,161,150,41,16,123,
154,199,243,145,120,111,157,158,178,177,50,117,25,61,255,53,138,126,109,
84,198,128,195,189,13,87,223,245,36,169,62,168,67,201,215,121,214,246,12
4,34,185,3,224,15,236,222,122,148,176,188,220,232,40,80,78,51,10,74,167,
151,96,115,30,0,98,68,26,184,56,130,100,159,38,65,173,69,70,146,39,94,85
,47,140,163,165,125,105,213,149,59,7,88,179,64,134,172,29,247,48,55,107,
228,136,217,231,137,225,27,131,73,76,63,248,254,141,83,170,144,202,216,1
33,97,32,113,103,164,45,43,9,91,203,155,37,208,190,229,108,82,89,166,116
,210,230,244,180,192,209,102,175,194,57,75,99,182};

inline constexpr uint64_t LBOX[64] = {
0x8e20faa72ba0b470ULL,0x47107ddd9b505a38ULL,0xad08b0e0c3282d1cULL,0xd804
5870ef14980eULL,
0x6c022c38f90a4c07ULL,0x3601161cf205268dULL,0x1b8e0b0e798c13c8ULL,0x8347
8b07b2468764ULL,
0xa011d380818e8f40ULL,0x5086e740ce47c920ULL,0x2843fd2067adea10ULL,0x14af
f010bdd87508ULL,
0x0ad97808d06cb404ULL,0x05e23c0468365a02ULL,0x8c711e02341b2d01ULL,0x46b6
0f011a83988eULL,
0x90dab52a387ae76fULL,0x486dd4151c3dfdb9ULL,0x24b86a840e90f0d2ULL,0x125c
354207487869ULL,
0x092e94218d243cbaULL,0x8a174a9ec8121e5dULL,0x4585254f64090fa0ULL,0xaccc
9ca9328a8950ULL,
0x9d4df05d5f661451ULL,0xc0a878a0a1330aa6ULL,0x60543c50de970553ULL,0x302a
1e286fc58ca7ULL,
0x18150f14b9ec46ddULL,0x0c84890ad27623e0ULL,0x0642ca05693b9f70ULL,0x0321
```

```
658cba93c138ULL,
0x86275df09ce8aaa8ULL,0x439da0784e745554ULL,0xafc0503c273aa42aULL,0xd960
281e9d1d5215ULL,
0xe230140fc0802984ULL,0x71180a8960409a42ULL,0xb60c05ca30204d21ULL,0x5b06
8c651810a89eULL,
0x456c34887a3805b9ULL,0xac361a443d1c8cd2ULL,0x561b0d22900e4669ULL,0x2b83
8811480723baULL,
0x9bcf4486248d9f5dULL,0xc3e9224312c8c1a0ULL,0xeffa11af0964ee50ULL,0xf97d
86d98a327728ULL,
0xe4fa2054a80b329cULL,0x727d102a548b194eULL,0x39b008152acb8227ULL,0x9258
048415eb419dULL,
0x492c024284fbaec0ULL,0xaa16012142f35760ULL,0x550b8e9e21f7a530ULL,0xa48b
474f9ef5dc18ULL,
0x70a6a56e2440598eULL,0x3853dc371220a247ULL,0x1ca76e95091051adULL,0x0edd
37c48a08a6d8ULL,
0x07e095624504536cULL,0x8d70c431ac02a736ULL,0xc83862965601dd1bULL,0x641c
314b2b8ee083ULL};

inline constexpr std::array<State, 12> ROUND_C = {{
{{0xb1085bda1ecadae9ULL,0xebcb2f81c0657c1fULL,0x2f6a76432e45d016ULL,0x71
4eb88d7585c4fcULL,0x4b7ce09192676901ULL,0xa2422a08a460d315ULL,0x05767436
cc744d23ULL,0xdd806559f2a64507ULL}},
{{0x6fa3b58aa99d2f1aULL,0x4fe39d460f70b5d7ULL,0xf3feea720a232b98ULL,0x61
d55e0f16b50131ULL,0x9ab5176b12d69958ULL,0x5cb561c2db0aa7caULL,0x55dda21b
d7cbcd56ULL,0xe679047021b19bb7ULL}},
{{0xf574dcac2bce2fc7ULL,0x0a39fc286a3d8435ULL,0x06f15e5f529c1f8bULL,0xf2
ea7514b1297b7bULL,0xd3e20fe490359eb1ULL,0xc1c93a376062db09ULL,0xc2b6f443
867adb31ULL,0x991e96f50aba0ab2ULL}},
{{0xef1fdfb3e81566d2ULL,0xf948e1a05d71e4ddULL,0x488e857e335c3c7dULL,0x9d
721cad685e353fULL,0xa9d72c82ed03d675ULL,0xd8b71333935203beULL,0x3453eaa1
93e837f1ULL,0x220cbebc84e3d12eULL}},
{{0x4bea6bacad474799ULL,0x9a3f410c6ca92363ULL,0x7f151c1f1686104aULL,0x35
9e35d7800fffbdULL,0xbfcd1747253af5a3ULL,0xdfff00b723271a16ULL,0x7a56a27e
a9ea63f5ULL,0x601758fd7c6cfe57ULL}},
{{0xae4faeae1d3ad3d9ULL,0x6fa4c33b7a3039c0ULL,0x2d66c4f95142a46cULL,0x18
7f9ab49af08ec6ULL,0xcffaa6b71c9ab7b4ULL,0x0af21f66c2bec6b6ULL,0xbf71c572
36904f35ULL,0xfa68407a46647d6eULL}},
{{0xf4c70e16eeaac5ecULL,0x51ac86febf240954ULL,0x399ec6c7e6bf87c9ULL,0xd3
473e33197a93c9ULL,0x0992abc52d822c37ULL,0x06476983284a0504ULL,0x3517454c
a23c4af3ULL,0x8886564d3a14d493ULL}},
{{0x9b1f5b424d93c9a7ULL,0x03e7aa020c6e4141ULL,0x4eb7f8719c36de1eULL,0x89
b4443b4ddbc49aULL,0xf4892bcb929b0690ULL,0x69d18d2bd1a5c42fULL,0x36acc235
5951a8d9ULL,0xa47f0dd4bf02e71eULL}},
{{0x378f5a541631229bULL,0x944c9ad8ec165fdeULL,0x3a7d3a1b25894224ULL,0x3c
d955b7e00d0984ULL,0x800a440bdbb2ceb1ULL,0x7b2b8a9aa6079c54ULL,0x0e38dc92
cb1f2a60ULL,0x7261445183235adbULL}},
{{0xabbedea680056f52ULL,0x382ae548b2e4f3f3ULL,0x8941e71cff8a78dbULL,0x1f
ffe18a1b336103ULL,0x9fe76702af69334bULL,0x7a1e6c303b7652f4ULL,0x3698fad1
153bb6c3ULL,0x74b4c7fb98459cedULL}},
{{0x7bcd9ed0efc889fbULL,0x3002c6cd635afe94ULL,0xd8fa6bbbebab0761ULL,0x20
```

```
01802114846679ULL,0x8a1d71efea48b9caULL,0xefbacd1d7d476e98ULL,0xdea2594a
c06fd85dULL,0x6bcaa4cd81f32d1bULL}},
{{0x378ee767f11631baULL,0xd21380b00449b17aULL,0xcda43c32bcdf1d77ULL,0xf8
2012d430219f9bULL,0x5d80ef9d1891cc86ULL,0xe71da4aa88e12852ULL,0xfaf417d5
d9b21b99ULL,0x48bc924af11bd720ULL}}
}};

// Таблица T включает подстановку S для каждой позиции входного байта.
// Вычисляет таблицу при компиляции без проверки инициализации в цикле
поиска.
using LpsTable = std::array<std::array<uint64_t, 256>, 8>;
inline constexpr LpsTable make_lps_table() {
    LpsTable n{};
    for (int pos = 0; pos < 8; ++pos) {
        for (int x = 0; x < 256; ++x) {
            const uint8_t sb = SBOX[x];
            uint64_t acc = 0;
            for (int bit = 0; bit < 8; ++bit) {
                if (sb & (uint8_t(1u) << (7 - bit))) {
                    acc ^= LBOX[pos * 8 + bit];
                }
            }
            n[pos][x] = acc;
        }
    }
    return n;
}
inline constexpr LpsTable LPS_TABLE = make_lps_table();

inline uint8_t byte_be(uint64_t x, int byte_index) noexcept {
    return static_cast<uint8_t>(x >> (56 - 8 * byte_index));
}

// Объединяет S, P и L, сохраняя состояние в восьми 64-битных словах.
inline State LPS(const State& a) noexcept {
    const auto& T = LPS_TABLE;
    State r{};
    for (int out = 0; out < 8; ++out) {
        r[out] = T[0][byte_be(a[0], out)] ^
                 T[1][byte_be(a[1], out)] ^
                 T[2][byte_be(a[2], out)] ^
                 T[3][byte_be(a[3], out)] ^
                 T[4][byte_be(a[4], out)] ^
                 T[5][byte_be(a[5], out)] ^
                 T[6][byte_be(a[6], out)] ^
                 T[7][byte_be(a[7], out)];
    }
    return r;
}
```

```
// Объединяет побитовое исключающее ИЛИ с преобразованием раунда без
промежуточных массивов.

inline State LPS_XOR(const State& a, const State& b) noexcept {
    const auto& T = LPS_TABLE;
    const uint64_t x0 = a[0] ^ b[0];
    const uint64_t x1 = a[1] ^ b[1];
    const uint64_t x2 = a[2] ^ b[2];
    const uint64_t x3 = a[3] ^ b[3];
    const uint64_t x4 = a[4] ^ b[4];
    const uint64_t x5 = a[5] ^ b[5];
    const uint64_t x6 = a[6] ^ b[6];
    const uint64_t x7 = a[7] ^ b[7];
    State r{};
    for (int out = 0; out < 8; ++out) {
        r[out] = T[0][byte_be(x0, out)] ^
                 T[1][byte_be(x1, out)] ^
                 T[2][byte_be(x2, out)] ^
                 T[3][byte_be(x3, out)] ^
                 T[4][byte_be(x4, out)] ^
                 T[5][byte_be(x5, out)] ^
                 T[6][byte_be(x6, out)] ^
                 T[7][byte_be(x7, out)];
    }
    return r;
}

struct CompressionContext {
    State h{};
    std::array<State, 12> round_keys{};
    State final_key{};
    int rounds = 12;
};

// Расписание ключей зависит от h, N и числа раундов и переиспользуется
для всех блоков кандидатов.

inline CompressionContext make_context(const State& h, const State& n,
int rounds) {
    if (rounds < 1 || rounds > 12) throw std::runtime_error("rounds must
be 1..12");
    CompressionContext ctx{};
    ctx.h = h;
    ctx.rounds = rounds;

    State K = LPS_XOR(h, n);
    for (int i = 0; i < rounds; ++i) {
        ctx.round_keys[static_cast<size_t>(i)] = K;
        K = LPS_XOR(K, ROUND_C[static_cast<size_t>(i)]);
    }
```

```
    ctx.final_key = K;
    return ctx;
}

inline CompressionContext make_context(const State& h, uint64_t N, int
rounds) {
    State n{};
    n[7] = N;
    return make_context(h, n, rounds);
}

// Выполняет скалярное сжатие с предварительно вычисленными ключами
раундов.

inline State compress_with_context(const State& m, const
CompressionContext& ctx) noexcept {
    State state = m;
    for (int i = 0; i < ctx.rounds; ++i) {
        state = LPS_XOR(ctx.round_keys[static_cast<size_t>(i)], state);
    }

    State out{};
    for (size_t i = 0; i < 8; ++i) {
        out[i] = ctx.final_key[i] ^ state[i] ^ ctx.h[i] ^ m[i];
    }
    return out;
}

inline State compress_state(const State& m, const State& h, uint64_t N,
int rounds) {
    return compress_with_context(m, make_context(h, N, rounds));
}

inline int hex_value(char c) {
    if (c >= '0' && c <= '9') return c - '0';
    if (c >= 'a' && c <= 'f') return c - 'a' + 10;
    if (c >= 'A' && c <= 'F') return c - 'A' + 10;
    throw std::runtime_error("invalid hex character");
}

// Восстанавливает состояние из первых n бит части хэш-кода, заполняя
остальные биты нулями.
inline State reduced_state_from_hex(const std::string& s, int n) {
    if (n <= 0 || n > 512 || (n % 4) != 0) throw std::runtime_error("n
must be multiple of 4 in 4..512");
    if (s.size() != static_cast<size_t>(n / 4)) throw
std::runtime_error("bad reduced state length");
    State out{};
    for (size_t nib = 0; nib < s.size(); ++nib) {
        const uint64_t v = static_cast<uint64_t>(hex_value(s[nib]));
```

```
        const size_t bit = nib * 4;
        const size_t word = bit / 64;
        const int shift = 60 - static_cast<int>(bit % 64);
        out[word] |= v << shift;
    }
    return out;
}

inline std::string reduced_state_to_hex(const State& s, int n) {
    const size_t nhex = static_cast<size_t>(n / 4);
    static constexpr char HEX[] = "0123456789abcdef";
    std::string out(nhex, '0');
    for (size_t nib = 0; nib < nhex; ++nib) {
        const size_t bit = nib * 4;
        const size_t word = bit / 64;
        const int shift = 60 - static_cast<int>(bit % 64);
        out[nib] = HEX[(s[word] >> shift) & 0xfu];
    }
    return out;
}

// Часть контрольной суммы содержит младшие s бит State[7] при s ≤ 64.

inline uint64_t sigma_mask64(int s) {
    if (s < 0 || s > 64) throw std::runtime_error("sigma bits must be in
0..64");
    if (s == 0) return 0;
    if (s == 64) return ~uint64_t{0};
    return (uint64_t{1} << s) - 1;
}

inline uint64_t required_sigma_low64(uint64_t current_sigma, uint64_t
target_sigma, int s) {
    const uint64_t mask = sigma_mask64(s);
    return (target_sigma - current_sigma) & mask;
}

inline void set_low_sigma_bits(State& block, uint64_t required_low, int
s) {
    if (s == 0) return;
    const uint64_t mask = sigma_mask64(s);
    block[7] = (block[7] & ~mask) | (required_low & mask);
}

template<class Generator> inline State random_state(Generator& rng)
noexcept {
    State s{};
    for (uint64_t& x : s) x = rng();
    return s;
}
```

```
template<class Generator> inline State
random_state_constrained(Generator& rng, uint64_t required_low, int s)
noexcept {
    State block = random_state(rng);
    if (s != 0) {
        const uint64_t mask = s == 64 ? ~uint64_t{0} : ((uint64_t{1} <<
s) - 1);
        block[7] = (block[7] & ~mask) | (required_low & mask);
    }
    return block;
}

}
```

ПРИЛОЖЕНИЕ Е

Листинг построения алмазной структуры на Python

```
"""Управляет квантовым построением алмазной структуры для приложения
E."""

from __future__ import annotations

import math
import random
from dataclasses import dataclass
from typing import Any, Dict, Optional, Sequence

import networkx as nx

from CppCore import cpp_core_available, cpp_quantum_layer
from .search_methods import BBHTOptions, SearchMethod, search_method,
SEARCH_PROTOCOL
from .diamond import _normalize_initial_states, sigma_schedule
from .collision_probability import plan_search_messages
from .statevector_limits import check_layer_resources, check_deadline,
DENSE_MAX_STATES
from .grover import GroverResult, simulate_grover_from_marked,
optimal_grover_iterations
from .model import ReducedGostModel, constrained_random_block,
update_sigma_reduced, validate_sigma_bits, validate_truncate_bits


class QuantumLayerFailure(RuntimeError):
    """Сообщает об отсутствии совершенного паросочетания в вероятностном
алгоритме 2."""


@dataclass(frozen=True)
class QuantumLayerParameters:
    state_bits: int
    layer_bits: int
    vertex_count: int
    table_messages: int
    search_messages: int


def _ceil_rational_cube_root(numerator: int, denominator: int = 1) ->
int:
    """Вычисляет точный целый потолок кубического корня отношения
числителя и знаменателя."""
    low, high = 0, 1 << ((numerator.bit_length() + 2) // 3)
    while low + 1 < high:
```

```
        middle = (low + high) // 2
        if middle ** 3 * denominator >= numerator:
            high = middle
        else:
            low = middle
    return high


def algorithm2_parameters(state_bits: int, layer_bits: int) ->
QuantumLayerParameters:
    """Вычисляет L = ⌈∛(r² · 2^(n-r))⌉ и S = ⌈∛(2^(2(n-r))/r²)⌉ для r =
layer_bits."""
    if state_bits <= 0:
        raise ValueError("state_bits должен быть положительным.")
    if layer_bits < 1 or layer_bits > state_bits:
        raise ValueError("layer_bits должен лежать в диапазоне
1..state_bits.")
    vertex_count = 1 << layer_bits
    # Целочисленные кубы исключают ошибки округления при вычислении L и
S.
    gap = state_bits - layer_bits
    table_messages = _ceil_rational_cube_root(layer_bits ** 2 * (1 <<
gap))
    search_messages = _ceil_rational_cube_root(1 << (2 * gap),
layer_bits ** 2)
    return QuantumLayerParameters(
        state_bits=state_bits,
        layer_bits=layer_bits,
        vertex_count=vertex_count,
        table_messages=table_messages,
        search_messages=search_messages,
    )


def select_layer_parameters(
    state_bits: int,
    layer_bits: int,
    *,
    parameter_strategy: str = "theoretical",
    target_probability: float = 0.95,
) -> tuple[QuantumLayerParameters, dict[str, float | int]]:
    """Выбирает размер поиска для одного шага без изменения исходной
таблицы алгоритма 2."""
    if parameter_strategy not in ("theoretical", "target_probability"):
        raise ValueError("Неизвестный режим выбора параметров.")
    if not isinstance(target_probability, (int, float)) or not
math.isfinite(target_probability) or not 0 < target_probability < 1:
        raise ValueError("Целевая вероятность должна быть строго между 0
и 1.")
    baseline = algorithm2_parameters(state_bits, layer_bits)
```

```
    estimate = plan_search_messages(
        state_bits, baseline.vertex_count, baseline.table_messages,
        baseline.search_messages, target_probability,
    )
    if parameter_strategy == "theoretical":
        selected = baseline
        predicted = estimate.baseline_probability
    else:
        selected = QuantumLayerParameters(
            baseline.state_bits, baseline.layer_bits,
baseline.vertex_count,
            baseline.table_messages, estimate.search_messages,
        )
        predicted = estimate.predicted_probability
    return selected, {
        "estimated_distinct_targets": estimate.distinct_targets,
        "predicted_candidate_probability": predicted,
        "baseline_candidate_probability": estimate.baseline_probability,
        "baseline_S": baseline.search_messages,
    }


def _new_message_space(
    count: int, rng: random.Random, used: set[str], *, sigma_bits: int =
0,
    current_sigma: int = 0, target_sigma: int = 0,
) -> list[str]:
    messages = []
    while len(messages) < count:
        check_deadline()
        block = constrained_random_block(current_sigma, target_sigma,
sigma_bits, rng)
        if block in used:
            continue
        used.add(block)
        messages.append(block)
    return messages


def build_one_layer_quantum(
    states: Sequence[str],
    *,
    level_N: int,
    truncate_bits: int,
    rounds: int = 12,
    rng: Optional[random.Random] = None,
    max_statevector_size: Optional[int] = None,
    stats: Optional[Dict[str, int]] = None,
    use_cpp: Optional[bool] = None,
    sigma_bits: int = 0,
```

```
    current_sigma: int = 0,
    target_sigma: int = 0,
    threads: int = 0,
    parameter_strategy: str = "theoretical",
    target_probability: float = 0.95,
    method: str = "grover_fixed",
    bbht_options: BBHTOptions | None = None,
) -> list[tuple[str, str, str, str, str]]:
    """Строит уровень алгоритмом 2 через C++ или резервный Python с
проверкой паросочетания и переходов."""
    validate_truncate_bits(truncate_bits)
    validate_sigma_bits(sigma_bits)
    states = list(states)
    vertex_count = len(states)
    if vertex_count < 2 or vertex_count & (vertex_count - 1):
        raise ValueError("Число состояний слоя должно быть степенью
двойки >= 2.")
    if len(set(states)) != len(states):
        raise ValueError("Состояния входного слоя должны быть
различными.")
    layer_bits = int(math.log2(vertex_count))
    params, probability_stats = select_layer_parameters(
        truncate_bits, layer_bits,
parameter_strategy=parameter_strategy,
        target_probability=target_probability,
    )
    if (
        max_statevector_size is not None
        and params.search_messages > max_statevector_size
    ):
        raise ValueError(
            f"Algorithm 2 требует state-vector
S={params.search_messages}, "
            f"лимит симулятора={max_statevector_size}."
        )

    selected = search_method(method)
    if selected is SearchMethod.CLASSICAL:
        raise ValueError("Квантовое построение требует фиксированного
Гровера или BBHT.")
    cpp_enabled = cpp_core_available() if use_cpp is None else use_cpp
    if selected is SearchMethod.GROVER_BBHT and not cpp_enabled:
        raise RuntimeError("BBHT требует собранное C++-ядро.")
    dense_limit = (bbht_options or BBHTOptions()).dense_max_states if
selected is SearchMethod.GROVER_BBHT else DENSE_MAX_STATES
    check_layer_resources(params.table_messages, params.search_messages,
vertex_count, layer_bits,
                          python=not cpp_enabled,
dense_max_states=dense_limit)
    generator = rng or random.Random()
```

```
    model = ReducedGostModel(truncate_bits, rounds)
    graph = nx.Graph()
    graph.add_nodes_from(range(vertex_count))
    quantum_queries = 0
    simulator_evaluations = 0
    grover_successes = 0
    vector_stats = dict(elements=0, iterations=0, attempts=0, marked=0,
probability_sum=0.0, norm_max=0.0, seconds=0.0, verification=0)
    diagnostics = []
    if cpp_enabled:
        cpp_result = cpp_quantum_layer(
            states,
            params.table_messages,
            params.search_messages,
            layer_bits,
            level_N,
            truncate_bits,
            rounds,
            generator.getrandbits(64),
            sigma_bits=sigma_bits,
            current_sigma=current_sigma,
            target_sigma=target_sigma,
            threads=threads,
            **({"method": selected, "bbht_options": bbht_options} if
selected is SearchMethod.GROVER_BBHT else {}),
        )
        quantum_queries = cpp_result.oracle_queries
        simulator_evaluations = cpp_result.simulator_evaluations
        grover_successes = cpp_result.grover_successes
        vector_stats =
dict(elements=cpp_result.statevector_elements_processed,
iterations=cpp_result.statevector_iterations,
                             attempts=cpp_result.grover_attempts,
marked=cpp_result.marked_elements,

probability_sum=cpp_result.success_probability_sum,
norm_max=cpp_result.statevector_norm_error_max,
                             seconds=cpp_result.statevector_seconds,
verification=cpp_result.verification_evaluations)
        for left, right, left_block, right_block, common in
cpp_result.edges:
            if left == right or not (0 <= left < vertex_count and 0 <=
right < vertex_count):
                raise AssertionError("C++ quantum-layer вернул
некорректное ребро.")
            if not graph.has_edge(left, right):
                graph.add_edge(
                    left,
                    right,
                    common_state=common,
```

```
                    blocks={left: left_block, right: right_block},
                )
        qram_entries = cpp_result.qram_entries
    else:
        used_messages: set[str] = set()

        # Общее множество M используется для всех вершин с сохранением
каждой записи.
        table_space = _new_message_space(
            params.table_messages, generator, used_messages,
sigma_bits=sigma_bits,
            current_sigma=current_sigma, target_sigma=target_sigma,
        )
        qram: dict[str, list[tuple[int, str]]] = {}
        for vertex_index, state in enumerate(states):
            for block_index, block in enumerate(table_space):
                if block_index % 256 == 0:
                    check_deadline()
                output = model.transition(block, state, level_N)
                qram.setdefault(output, []).append((vertex_index,
block))
        qram_entries = vertex_count * params.table_messages
        simulator_evaluations = qram_entries

        # Для каждой вершины выполняется layer_bits поисков Гровера.
        for vertex_index, state in enumerate(states):
            for _ in range(layer_bits):
                check_deadline()
                search_space = _new_message_space(
                    params.search_messages, generator, used_messages,
sigma_bits=sigma_bits,
                    current_sigma=current_sigma,
target_sigma=target_sigma,
                )
                outputs = []
                for block_index, block in enumerate(search_space):
                    if block_index % 256 == 0:
                        check_deadline()
                    outputs.append(model.transition(block, state,
level_N))
                simulator_evaluations += params.search_messages
                marked = [
                    index
                    for index, output in enumerate(outputs)
                    if any(other != vertex_index for other, _ in
qram.get(output, ()))
                ]
                grover: GroverResult = simulate_grover_from_marked(
                    params.search_messages,
                    marked,
```

```
iterations=optimal_grover_iterations(params.search_messages, 1),
                    rng=generator,
                    max_statevector_size=max_statevector_size,
                )
                vector_stats['elements'] +=
grover.statevector_elements_processed
                vector_stats['iterations'] += grover.iterations
                vector_stats['attempts'] += 1
                diagnostics.append(dict(attempt_policy='fixed_single',
search_messages=params.search_messages,
                    growth=1.2, effective_max_attempts=1, attempts=1,
failures=int(not grover.is_marked),
                    stop_reason='found' if grover.is_marked else
'unmarked_measurement'))
                vector_stats['marked'] += grover.marked_count
                vector_stats['probability_sum'] +=
grover.success_probability
                vector_stats['norm_max'] = max(vector_stats['norm_max'],
grover.norm_error)
                vector_stats['seconds'] += grover.statevector_seconds
                measured = grover.measured_index
                if model.transition(search_space[measured], state,
level_N) != outputs[measured]:
                    raise AssertionError('Переход измеренного кандидата
не воспроизводится.')
                simulator_evaluations += 1
                vector_stats['verification'] += 1
                quantum_queries += grover.quantum_queries + 1  #
Учитывает проверку измеренного кандидата.
                if not grover.is_marked or grover.measured_index is
None:
                    continue

                candidate_index = grover.measured_index
                output = outputs[candidate_index]
                candidate_block = search_space[candidate_index]
                other_entries = [
                    entry for entry in qram[output] if entry[0] !=
vertex_index
                ]
                if not other_entries:
                    raise AssertionError(
                        "Grover отметил значение без межвершинной
коллизии."
                    )
                other_vertex, other_block = other_entries[0]
                grover_successes += 1
                if not graph.has_edge(vertex_index, other_vertex):
                    graph.add_edge(
```

```
                        vertex_index,
                        other_vertex,
                        common_state=output,
                        blocks={vertex_index: candidate_block,
other_vertex: other_block},
                    )

    if not cpp_enabled:
        quantum_queries += qram_entries  # Учитывает запросы функции
сжатия при подготовке qRAM.
    grover_iterations = vector_stats['iterations']
    if cpp_enabled and stats is not None:
        stats['amplitude_backend'] = cpp_result.amplitude_backend
    if stats is not None:
        stats['search_protocol'] = cpp_result.search_protocol if
cpp_enabled else SEARCH_PROTOCOL
        stats['quantum_search_diagnostics'] = [dict(item,
layer_bits=layer_bits,
            vertex=index//layer_bits, repetition=index%layer_bits) for
index, item in
            enumerate(cpp_result.search_diagnostics if cpp_enabled else
diagnostics)]
        stats['quantum_phase_oracle_queries'] =
vector_stats['iterations']
        stats['quantum_verification_queries'] =
vector_stats['verification']
        stats['quantum_failed_attempts'] = vector_stats['attempts'] -
grover_successes
    if selected is SearchMethod.GROVER_BBHT and stats is not None:
        stats['quantum_phase_oracle_queries'] =
cpp_result.phase_oracle_queries
        stats['quantum_verification_queries'] =
cpp_result.verification_queries
        stats['quantum_failed_attempts'] = cpp_result.failed_attempts
    # Ищет совершенное паросочетание в графе коллизий.
    raw_matching = nx.algorithms.matching.max_weight_matching(
        graph, maxcardinality=True
    )
    matching = {
        (min(left, right), max(left, right)) for left, right in
raw_matching
    }
    if stats is not None:
        for suffix, key in (('elements_processed', 'elements'),
('iterations', 'iterations'), ('seconds', 'seconds')):
            field = 'quantum_statevector_' + suffix
            stats[field] = stats.get(field, 0) + vector_stats[key]
        stats['quantum_statevector_norm_error_max'] =
max(stats.get('quantum_statevector_norm_error_max', 0.0),
vector_stats['norm_max'])
```

```
        for field, key in (('quantum_grover_attempts', 'attempts'),
('quantum_marked_elements', 'marked'),
('quantum_success_probability_sum', 'probability_sum'),
('quantum_verification_hash_evaluations', 'verification')):
            stats[field] = stats.get(field, 0) + vector_stats[key]
        stats['quantum_matching_attempts'] =
stats.get('quantum_matching_attempts', 0) + 1
        stats["quantum_oracle_queries"] =
stats.get("quantum_oracle_queries", 0) + quantum_queries
        stats["quantum_simulator_hash_evaluations"] = (
            stats.get("quantum_simulator_hash_evaluations", 0) +
simulator_evaluations
        )
        stats["quantum_qram_entries"] =
stats.get("quantum_qram_entries", 0) + qram_entries
        stats["quantum_qram_total_entries"] =
stats.get("quantum_qram_total_entries", 0) + qram_entries
        stats["quantum_qram_peak_entries"] =
max(stats.get("quantum_qram_peak_entries", 0), qram_entries)
        stats["quantum_grover_iterations"] =
stats.get("quantum_grover_iterations", 0) + grover_iterations
        stats["quantum_table_queries"] =
stats.get("quantum_table_queries", 0) + qram_entries
        stats["quantum_search_verification_queries"] =
stats.get("quantum_search_verification_queries", 0) +
vector_stats["verification"]
        stats["quantum_grover_successes"] =
stats.get("quantum_grover_successes", 0) + grover_successes
        stats["collision_queries"] = stats.get("collision_queries", 0) +
simulator_evaluations
        for field, value in probability_stats.items():
            stats[f"quantum_layer_{layer_bits}_{field}"] = value
        stats[f"quantum_layer_{layer_bits}_L"] = params.table_messages
        stats[f"quantum_layer_{layer_bits}_S"] = params.search_messages
        stats[f"quantum_layer_{layer_bits}_edges"] =
graph.number_of_edges()
        backend_key = "quantum_cpp_layer_calls" if cpp_enabled else
"quantum_python_layer_calls"
        stats[backend_key] = stats.get(backend_key, 0) + 1
    if len(matching) != vertex_count // 2:
        raise QuantumLayerFailure(
            "Algorithm 2 не получил perfect matching: "
            f"matching={len(matching)}/{vertex_count // 2}, "
            f"|E|={graph.number_of_edges()}, L={params.table_messages},
"
            f"S={params.search_messages}. Повторите вероятностный слой."
        )
    matched_outputs = [
        graph.get_edge_data(left, right)["common_state"] for left, right
in matching
```

```
    ]
    if len(set(matched_outputs)) != len(matched_outputs):
        raise QuantumLayerFailure(
            "Perfect matching содержит одинаковые выходные состояния; "
            "для следующего слоя нужны различные вершины. Повторите
слой."
        )

    level = []
    for left, right in sorted(matching):
        edge = graph.get_edge_data(left, right)
        common = edge["common_state"]
        blocks = edge["blocks"]
        left_block = blocks[left]
        right_block = blocks[right]
        if model.transition(left_block, states[left], level_N) !=
common:
            raise AssertionError("Левое ребро Algorithm 2 не
воспроизводится.")
        if model.transition(right_block, states[right], level_N) !=
common:
            raise AssertionError("Правое ребро Algorithm 2 не
воспроизводится.")
        if update_sigma_reduced(current_sigma, left_block, sigma_bits)
!= target_sigma:
            raise AssertionError("Левое sigma-ребро Algorithm 2 не
воспроизводится.")
        if update_sigma_reduced(current_sigma, right_block, sigma_bits)
!= target_sigma:
            raise AssertionError("Правое sigma-ребро Algorithm 2 не
воспроизводится.")
        level.append((states[left], states[right], left_block,
right_block, common))

    if stats is not None:
        stats['quantum_matching_successes'] =
stats.get('quantum_matching_successes', 0) + 1
        stats['quantum_independent_verification_hash_evaluations'] =
stats.get('quantum_independent_verification_hash_evaluations', 0) +
vertex_count
    return level


def _merge_layer_stats(stats, values, attempt):
    """Объединяет счетчики успешных и неудачных попыток по одинаковым
правилам."""
    if stats is None:
        return
    for key, value in values.items():
        if key == 'quantum_search_diagnostics':
```

```
            stats.setdefault(key, []).extend(dict(item,
level_attempt=attempt) for item in value)
        elif key in ('amplitude_backend', 'search_protocol'):
            stats[key] = value if stats.get(key, value) == value else
'mixed'
        elif key in ('quantum_qram_peak_entries',
'quantum_statevector_norm_error_max'):
            stats[key] = max(stats.get(key, 0), value)
        elif key.startswith('quantum_layer_') and key.endswith(('_L',
'_S', '_edges', '_targets', '_probability', '_baseline_S')):
            stats[key] = value
        else:
            stats[key] = stats.get(key, 0) + value


def build_diamond_structure_quantum(
    search,
    depth: int = 3,
    truncate_bits: int = 12,
    trials_limit: int = 1_000_000,
    rounds: int = 12,
    logger: Optional[Any] = None,
    first_level_N: int = 1024,
    stats: Optional[Dict[str, int]] = None,
    initial_states: Optional[Sequence[str]] = None,
    max_layer_attempts: int = 12,
    seed: Optional[int] = None,
    use_cpp: Optional[bool] = None,
    sigma_bits: int = 0,
    sigma_seed: Optional[int] = None,
    threads: int = 0,
    parameter_strategy: str = "theoretical",
    target_probability: float = 0.95,
    method: str = "grover_fixed",
    bbht_options: BBHTOptions | None = None,
):
    """Строит алмазную структуру по уровням с заданными N, частью
контрольной суммы и повторными попытками."""
    del search
    validate_truncate_bits(truncate_bits)
    validate_sigma_bits(sigma_bits)
    if depth < 1:
        raise ValueError("depth должен быть не меньше 1.")
    if first_level_N < 512 or first_level_N % 512 != 0:
        raise ValueError("first_level_N должен быть положительным
кратным 512.")
    if trials_limit < 1 or max_layer_attempts < 1:
        raise ValueError("Лимиты должны быть положительными.")

    selected = search_method(method)
```

```
    if selected is SearchMethod.CLASSICAL:
        raise ValueError("Квантовое построение требует фиксированного
Гровера или BBHT.")
    cpp_enabled = cpp_core_available() if use_cpp is None else use_cpp
    if selected is SearchMethod.GROVER_BBHT and not cpp_enabled:
        raise RuntimeError("BBHT требует собранное C++-ядро.")
    for layer_bits in range(depth, 0, -1):
        params, _ = select_layer_parameters(
            truncate_bits, layer_bits,
parameter_strategy=parameter_strategy,
            target_probability=target_probability,
        )
        dense_limit = (bbht_options or BBHTOptions()).dense_max_states
if selected is SearchMethod.GROVER_BBHT else DENSE_MAX_STATES
        check_layer_resources(params.table_messages,
params.search_messages, params.vertex_count, layer_bits,
                              python=not cpp_enabled,
dense_max_states=dense_limit)
    generator = random.Random(seed)
    schedule = sigma_schedule(depth, sigma_bits, sigma_seed if
sigma_seed is not None else (seed or 0))
    leaf_count = 1 << depth
    leaves_states = _normalize_initial_states(initial_states,
leaf_count, truncate_bits, generator)
    leaves = [("", state) for state in leaves_states]
    current = leaves_states[:]
    structure = []
    total_simulator_evaluations = 0
    if stats is not None:
        stats["sigma_bits"] = sigma_bits
        stats["leaf_sigma"] = schedule[0]
        for i, value in enumerate(schedule[1:], start=1):
            stats[f"sigma_level_{i}"] = value

    if logger:
        logger._log(
            f"[{selected.label}] Листьев: {leaf_count}, "
            f"state-vector, без реального квантового ускорения."
        )

    if logger and hasattr(logger, "level_completed"):
        logger.level_completed(leaves, structure)

    for level_index in range(depth):
        level_N = first_level_N + 512 * level_index
        layer = None
        for attempt in range(1, max_layer_attempts + 1):
            check_deadline()
            if stats is not None:
                stats['quantum_layer_attempts'] =
```

```
stats.get('quantum_layer_attempts', 0) + 1
            attempt_stats: Dict[str, int] = {}
            try:
                candidate = build_one_layer_quantum(
                    current,
                    level_N=level_N,
                    truncate_bits=truncate_bits,
                    rounds=rounds,
                    rng=generator,
                    stats=attempt_stats,
                    use_cpp=use_cpp,
                    sigma_bits=sigma_bits,
                    current_sigma=schedule[level_index],
                    target_sigma=schedule[level_index + 1],
                    threads=threads,
                    parameter_strategy=parameter_strategy,
                    target_probability=target_probability,
                    method=selected, bbht_options=bbht_options,
                )
            except QuantumLayerFailure as exc:
                total_simulator_evaluations += attempt_stats.get(
                    "quantum_simulator_hash_evaluations", 0
                )
                if logger:
                    logger._log(
                        f"[Quantum] Уровень {level_index + 1}, попытка
{attempt}: {exc}"
                    )
                _merge_layer_stats(stats, attempt_stats, attempt)
                if total_simulator_evaluations > trials_limit:
                    raise RuntimeError(
                        "Quantum Algorithm 2 превысил лимит классических
"
                        f"вычислений симулятора:
{total_simulator_evaluations} > "
                        f"{trials_limit}."
                    ) from exc
                continue
            layer = candidate
            total_simulator_evaluations += attempt_stats.get(
                "quantum_simulator_hash_evaluations", 0
            )
            _merge_layer_stats(stats, attempt_stats, attempt)
            if total_simulator_evaluations > trials_limit:
                raise RuntimeError(
                    "Quantum Algorithm 2 превысил лимит классических "
                    f"вычислений симулятора:
{total_simulator_evaluations} > "
                    f"{trials_limit}."
                )
```

```
            break

        if layer is None:
            raise RuntimeError(
                f"Quantum Algorithm 2: уровень {level_index + 1} не
построен "
                f"за {max_layer_attempts} попыток."
            )
        structure.append(layer)
        if logger and hasattr(logger, "level_completed"):
            logger.level_completed(leaves, structure)
        current = [node[4] for node in layer]
        if logger:
            logger._log(
                f"[Quantum] Уровень {level_index + 1}: "
                f"{len(layer)} пар, N={level_N}."
            )

    if stats is not None:
        # Разделяет вычисления симулятора и модельные запросы без
множителя 2^s.
        stats["compression_calls"] =
int(stats.get("quantum_simulator_hash_evaluations", 0))
        stats["generated_candidates"] =
int(stats.get("quantum_simulator_hash_evaluations", 0))
        stats["oracle_queries"] =
int(stats.get("quantum_oracle_queries", 0))
        stats["grover_iterations"] =
int(stats.get("quantum_grover_iterations", 0))

    return leaves, structure
```

ПРИЛОЖЕНИЕ Ж

Квантовый поиск коллизий при построении алмазной структуры на языке C++

```
#pragma once

#include <cstdlib>
#include <iostream>
#include <random>
#include <unordered_map>
#include "search_runtime.hpp"
#include "gost_reduced_model.hpp"
#include "grover_analytic.hpp"
#include "grover_statevector.hpp"
#include "grover_symmetric.hpp"
#include "bbht_adaptive.hpp"
#include <algorithm>
#include <cstdint>
#include <chrono>
#include <stdexcept>
#include <string>
#include <vector>

// Реализует квантовый поиск коллизий одного уровня для приложения Ж.


namespace quantum_diamond {

using gost_reduced_model::make_context;
using gost_reduced_model::reduced_state_from_hex;
using gost_reduced_model::required_sigma_low64;
using gost_reduced_model::set_low_sigma_bits;
using grover_analytic::optimal_grover_iterations;

struct QuantumTableEntry {
    size_t vertex = 0;
    State block{};
};

// Хранит все записи qRAM в порядке добавления с общей классической
таблицей открытой адресации.

class QuantumQram {
public:
    QuantumQram(uint64_t expected,int bits): bits_(bits) {
        if(bits<=64 && requested_qram!="generic") {

small_=std::make_unique<FlatTable64<std::vector<QuantumTableEntry>>>(exp
ected);
```

```
            small_->begin_epoch();
        } else generic_.reserve(static_cast<size_t>(expected*2+1));
    }
    void insert(const State& output,const QuantumTableEntry& entry) {
        if(small_)small_-
>get_or_insert(reduced_key64(output,bits_)).push_back(entry);
        else generic_[reduced_key_full(output,bits_)].push_back(entry);
    }
    const std::vector<QuantumTableEntry>* find(const State& output)
const {
        if(small_)return small_->find(reduced_key64(output,bits_));
        const auto it=generic_.find(reduced_key_full(output,bits_));
        return it==generic_.end()?nullptr:&it->second;
    }
    bool uses_u64() const { return bool(small_); }
private:
    int bits_;
    std::unique_ptr<FlatTable64<std::vector<QuantumTableEntry>>> small_;

std::unordered_map<ReducedKey,std::vector<QuantumTableEntry>,ReducedKeyH
ash> generic_;
};

struct QuantumEdge {
    size_t left = 0;
    size_t right = 0;
    State left_block{};
    State right_block{};
    State common{};
};

struct QuantumLayerResult {
    std::vector<QuantumEdge> edges;
    uint64_t oracle_queries = 0;
    uint64_t simulator_evaluations = 0;
    uint64_t qram_entries = 0;
    uint64_t grover_successes = 0;
    uint64_t statevector_elements_processed = 0;
    uint64_t statevector_iterations = 0;
    uint64_t grover_attempts = 0;
    uint64_t marked_elements = 0;
    uint64_t verification_evaluations = 0;
    double success_probability_sum = 0.0;
    double statevector_norm_error_max = 0.0;
    double statevector_seconds = 0.0;
    std::vector<bbht_adaptive::Result> search_diagnostics;
};

struct QuantumLayerWorkspace {
    std::vector<State> messages,outputs;
```

```
    uint64_t reallocations=0;
    bool debug=std::getenv("NOSTRADAMUS_WORKSPACE_DEBUG")!=nullptr;
    void resize_messages(size_t n) {
        if(debug && messages.capacity()<n)++reallocations;
        messages.resize(n);
    }
    void prepare_outputs(size_t n) {
        if(debug && outputs.capacity()<n)++reallocations;
        outputs.resize(n);
    }
};

// Вычисляет переходы общего множества сообщений пакетами по
непересекающимся диапазонам потоков.

inline void evaluate_message_space_into(
    const CompressionContext& ctx,
    const std::vector<State>& messages,
    std::vector<State>& outputs,
    int requested_threads,
    WorkerPool* pool=nullptr
) {
    outputs.resize(messages.size());
    if (messages.empty()) return;
    const uint64_t total = static_cast<uint64_t>(messages.size());
    auto worker = [&](WorkRange range) {
        SearchBatch batch_messages{};
        SearchBatch batch_outputs{};
        uint64_t index = range.begin;
        while (index < range.end) {
            const size_t count = static_cast<size_t>(
                std::min<uint64_t>(SEARCH_BATCH, range.end - index));
            for (size_t lane = 0; lane < count; ++lane) {
                batch_messages[lane] =
messages[static_cast<size_t>(index + lane)];
            }
            compress_batch(batch_messages, batch_outputs, count, ctx);
            for (size_t lane = 0; lane < count; ++lane) {
                outputs[static_cast<size_t>(index + lane)] =
batch_outputs[lane];
            }
            index += count;
        }
    };

    if(pool)pool->run(total,128,worker);
    else parallel_ranges(total,requested_threads,128,worker);
}

// Воспроизводит прежний блок кандидата по его абсолютному индексу без
```

```
расходования случайных чисел.
inline State indexed_message(uint64_t index, uint64_t seed, uint64_t
low, int sigma_bits) {
    State block{};
    block[0] = mix64(index + seed);
    for (size_t word = 1; word < block.size(); ++word)
        block[word] = mix64(index ^ seed ^ (0x9e3779b97f4a7c15ULL *
word));
    set_low_sigma_bits(block, low, sigma_bits);
    return block;
}

inline uint64_t layer_working_bytes(uint64_t L, uint64_t S, size_t
vertices, uint64_t dense_limit) {
    const auto budget =
grover_statevector::current_limits().max_memory_bytes;
    if (vertices > budget / 256 || L > budget / 256 / vertices)
        throw std::runtime_error("Оценка памяти qRAM превышает лимит
симулятора.");
    const auto buffers = std::max(L, S <= dense_limit ? S :
grover_oracle::batch_states);
    if (buffers > budget / 128) throw std::runtime_error("Рабочие буферы
превышают лимит памяти.");
    const auto bytes = 128 * buffers + 256 * L * vertices + 256 *
vertices;
    grover_oracle::check_resources(S, bytes);
    return bytes;
}

// Вычисляет весь реальный оракул пакетами и сохраняет только индексы
помеченных кандидатов.
template<class Check>
inline grover_oracle::MarkedSpace streamed_oracle(QuantumLayerWorkspace&
workspace,
    const CompressionContext& context, const QuantumQram& qram, size_t
vertex,
    uint64_t base, uint64_t count, uint64_t seed, uint64_t low, int
sigma_bits,
    int threads, WorkerPool& pool, Check&& check, uint64_t other_bytes)
{
    grover_oracle::MarkedSpace marked(count, 0, other_bytes);
    for (uint64_t offset = 0; offset < count;) {
        check();
        const auto batch = std::min(grover_oracle::batch_states, count -
offset);
        workspace.resize_messages(static_cast<size_t>(batch));
        for (size_t j = 0; j < batch; ++j)
            workspace.messages[j] = indexed_message(base + offset + j,
seed, low, sigma_bits);
        evaluate_message_space_into(context, workspace.messages,
```

```
workspace.outputs, threads, &pool);
        for (size_t j = 0; j < batch; ++j) {
            const auto* found = qram.find(workspace.outputs[j]);
            if (found) for (const auto& item : *found) if (item.vertex
!= vertex) {
                marked.add(offset + j);
                break;
            }
        }
        offset += batch;
    }
    return marked;
}


}

// Строит один уровень алмазной структуры посредством адаптивного режима
BBHT.
namespace quantum_diamond_bbht {

// Сохраняет диагностические показатели каждого поиска на одном уровне
алмаза.
struct SearchObservation {
    uint64_t vertex = 0;
    uint64_t repetition = 0;
    uint64_t marked_count = 0;
    bool found = false;
    bool exhausted = false;
    uint64_t oracle_queries = 0;
    uint64_t phase_oracle_queries = 0;
    uint64_t verification_queries = 0;
    uint64_t attempts = 0;
    uint64_t failed_attempts = 0;
    uint64_t search_messages = 0;
    double growth = 1.2;
    uint64_t effective_max_attempts = 0;
    const char* attempt_policy = "bbht_auto";
    const char* stop_reason = "attempt_limit";
};

struct Result {
    quantum_diamond::QuantumLayerResult layer;
    std::vector<SearchObservation> searches;
    uint64_t failures = 0;
    uint64_t exhausted_searches = 0;
    uint64_t successful_searches = 0;
    uint64_t total_searches = 0;
    uint64_t phase_oracle_queries = 0;
    uint64_t verification_queries = 0;
```

```
};

inline Result build_layer(
    const std::vector<std::string>& state_hex,
    uint64_t table_messages, uint64_t search_messages, int layer_bits,
    uint64_t N, int n, int rounds, uint64_t seed, int requested_threads,
    int sigma_bits = 0, uint64_t current_sigma = 0, uint64_t
target_sigma = 0,
    const bbht_adaptive::Options& options = {}, bool fixed_control =
false
) {
    using namespace gost_reduced_model;
    using namespace quantum_diamond;
    if (state_hex.size() < 2 || (state_hex.size() & (state_hex.size() -
1)) ||
        layer_bits < 1 || layer_bits >= 63 ||
        (uint64_t{1} << layer_bits) != state_hex.size() ||
        !table_messages || !search_messages)
        throw std::runtime_error("BBHT diamond: неверные параметры
уровня.");
    const uint64_t low = required_sigma_low64(current_sigma,
target_sigma, sigma_bits);
    if (!fixed_control) {
        bbht_adaptive::effective_max_attempts(search_messages,
options.growth, options.max_attempts);
        if (!options.max_oracle_queries) throw std::runtime_error("BBHT:
предел запросов должен быть положительным.");
    }
    const auto dense_limit = fixed_control ?
grover_oracle::dense_max_states : options.dense_max_states;
    const auto other_bytes = layer_working_bytes(table_messages,
search_messages, state_hex.size(), dense_limit);
    if (search_messages <= dense_limit)
        grover_statevector::check_resources(search_messages, 0,
other_bytes);
    const auto started = std::chrono::steady_clock::now();
    grover_oracle::Deadline fixed_deadline(search_messages >
dense_limit);
    auto check_deadline = [&]() {
        if (fixed_control) fixed_deadline.check();
        if(!fixed_control && options.max_stage_seconds > 0 &&

std::chrono::duration<double>(std::chrono::steady_clock::now()-
started).count()
           > options.max_stage_seconds) throw std::runtime_error("BBHT
diamond: stage timeout");
    };
    std::vector<CompressionContext> contexts;
    contexts.reserve(state_hex.size());
    for (const auto& hex : state_hex)
```

```
        contexts.push_back(make_context(reduced_state_from_hex(hex, n),
N, rounds));
    std::mt19937_64 rng(seed);
    uint64_t candidate_index = 0;
    QuantumLayerWorkspace workspace;
    WorkerPool pool(requested_threads, std::max(table_messages,
search_messages), 128);
    auto create_space = [&](uint64_t count) {
        workspace.resize_messages(static_cast<size_t>(count));
        for (auto& message : workspace.messages) {
            const uint64_t index = candidate_index++;
            State block{};
            block[0] = mix64(index + seed);
            for (size_t word = 1; word < block.size(); ++word)
                block[word] = mix64(index ^ seed ^
(0x9e3779b97f4a7c15ULL * word));
            set_low_sigma_bits(block, low, sigma_bits);
            message = block;
        }
    };
    Result result;
    result.searches.reserve(state_hex.size() *
static_cast<size_t>(layer_bits));
    auto& stats = result.layer;
    create_space(table_messages);
    QuantumQram qram(table_messages * state_hex.size(), n);
    for (size_t vertex = 0; vertex < state_hex.size(); ++vertex) {
        check_deadline();
        workspace.prepare_outputs(table_messages);
        evaluate_message_space_into(contexts[vertex],
workspace.messages,
                                    workspace.outputs,
requested_threads, &pool);
        for (size_t j = 0; j < workspace.outputs.size(); ++j)
            qram.insert(workspace.outputs[j], {vertex,
workspace.messages[j]});
    }
    stats.qram_entries = table_messages * state_hex.size();
    stats.simulator_evaluations = stats.qram_entries;
    stats.oracle_queries = stats.qram_entries;
    for (size_t vertex = 0; vertex < state_hex.size(); ++vertex) {
        for (int repetition = 0; repetition < layer_bits; ++repetition)
{
            check_deadline();
            const auto base = candidate_index;
            grover_oracle::MarkedSpace marked(search_messages,
dense_limit, other_bytes);
            if (marked.dense()) {
                create_space(search_messages);
                workspace.prepare_outputs(search_messages);
```

```
                evaluate_message_space_into(contexts[vertex],
workspace.messages,
                                            workspace.outputs,
requested_threads, &pool);
                for (size_t j = 0; j < search_messages; ++j) {
                    const auto* found = qram.find(workspace.outputs[j]);
                    if (found) for (const auto& item : *found) if
(item.vertex != vertex) {
                        marked.add(j);
                        break;
                    }
                }
            } else {
                if (search_messages > UINT64_MAX - candidate_index)
                    throw std::runtime_error("Исчерпан диапазон индексов
кандидатов.");
                candidate_index += search_messages;
                marked = streamed_oracle(workspace, contexts[vertex],
qram, vertex, base,
                    search_messages, seed, low, sigma_bits,
requested_threads, pool, check_deadline, other_bytes);
            }
            stats.simulator_evaluations += search_messages;
            State measured_block{}, measured_output{};
            auto verify = [&](size_t index) {
                measured_block = marked.dense() ?
workspace.messages[index]
                    : indexed_message(base + index, seed, low,
sigma_bits);
                measured_output = compress_with_context(measured_block,
contexts[vertex]);
                if (marked.dense() && measured_output !=
workspace.outputs[index])
                    throw std::runtime_error("BBHT diamond: ошибка
повторного вычисления ГОСТ.");
                const auto* found = qram.find(measured_output);
                if (!found) return false;
                for (const auto& item : *found)
                    if (item.vertex != vertex) return true;
                return false;
            };
            const auto search = fixed_control
                ? bbht_adaptive::fixed_single(marked, rng, verify)
                : bbht_adaptive::search(marked, rng, verify, options);
            stats.search_diagnostics.push_back(search);
            // Диагностика не влияет на выбор кандидатов и состояние
генератора случайных чисел.
            result.searches.push_back(SearchObservation{
                static_cast<uint64_t>(vertex),
static_cast<uint64_t>(repetition),
```

```
                marked.count(),
                search.found, search.exhausted, search.oracle_queries,
                search.phase_oracle_queries,
search.verification_queries,
                search.attempts, search.failures,
search.search_messages, search.growth,
                search.effective_max_attempts, search.attempt_policy,
search.stop_reason
            });
            stats.oracle_queries += search.oracle_queries;
            stats.simulator_evaluations += search.verification_queries;
            stats.verification_evaluations +=
search.verification_queries;
            stats.grover_attempts += search.attempts;
            stats.marked_elements += search.marked_count_diagnostic;
            stats.success_probability_sum +=
search.success_probability_sum;
            stats.statevector_norm_error_max =
std::max(stats.statevector_norm_error_max, search.norm_error_max);
            stats.statevector_iterations +=
search.statevector_iterations;
            stats.statevector_elements_processed +=
search.statevector_elements_processed;
            stats.statevector_seconds += search.statevector_seconds;
            result.phase_oracle_queries += search.phase_oracle_queries;
            result.verification_queries += search.verification_queries;
            result.failures += search.failures;
            result.exhausted_searches += search.exhausted ? 1 : 0;
            ++result.total_searches;
            if (!search.found) continue;
            ++result.successful_searches;
            const auto* matches = qram.find(measured_output);
            if (!matches) throw std::runtime_error("BBHT diamond: не
найдено ребро qRAM.");
            const QuantumTableEntry* other = nullptr;
            for (const auto& item : *matches)
                if (item.vertex != vertex) { other = &item; break; }
            if (!other) throw std::runtime_error("BBHT diamond: петля
вместо коллизии.");
            QuantumEdge edge{};
            edge.left = vertex;
            edge.right = other->vertex;
            edge.left_block = measured_block;
            edge.right_block = other->block;
            edge.common = measured_output;
            stats.edges.push_back(edge);
            ++stats.grover_successes;
        }
    }
    if(workspace.debug) std::cerr << "workspace_reallocations=" <<
```

```
workspace.reallocations
        << " messages_capacity=" << workspace.messages.capacity() << "
outputs_capacity=" << workspace.outputs.capacity() << '\n';
    check_deadline();
    return result;
}

} // namespace quantum_diamond_bbht

namespace quantum_diamond {
inline QuantumLayerResult build_quantum_layer(
    const std::vector<std::string>& states, uint64_t L, uint64_t S, int
layer_bits,
    uint64_t N, int n, int rounds, uint64_t seed, int threads,
    int sigma_bits = 0, uint64_t current_sigma = 0, uint64_t
target_sigma = 0) {
    return
quantum_diamond_bbht::build_layer(states,L,S,layer_bits,N,n,rounds,seed,
threads,
        sigma_bits,current_sigma,target_sigma,{},true).layer;
}
}
```

ПРИЛОЖЕНИЕ З

Полная herding-атака и проверка сообщения на языке Python

```
"""Управляет классической и квантовой herding-атакой с независимой
проверкой пути по состояниям (h_n, Sigma_s)."""

from __future__ import annotations

import math
import random
from dataclasses import asdict, dataclass, replace
from typing import Any, Dict, List, Optional, Sequence

from CppCore import cpp_core_available, cpp_quantum_link
from .search_methods import BBHTOptions, SearchMethod, search_method,
SEARCH_PROTOCOL
from .diamond import get_diamond_root, get_diamond_root_state,
validate_diamond_data
from .statevector_limits import check_resources, check_native_resources,
check_deadline, DENSE_MAX_STATES
from .grover import GroverResult, optimal_grover_iterations,
simulate_grover_from_marked
from .interfaces import SearchAlgorithms
from .model import (DiamondState, ReducedGostModel,
constrained_random_block,
                    update_sigma_reduced, validate_sigma_bits,
validate_truncate_bits)


@dataclass
class AttackResult:
    description: str
    event_result: str
    prefix_blocks: List[str]
    prefix_state: str
    linking_block: str
    leaf_state: str
    suffix_blocks: List[str]
    root: str
    verified_root: str
    forged_hex: str
    link_queries: int
    link_backend: str = "classical"
    link_simulator_hash_evaluations: int = 0
    link_attempts: int = 1
    link_marked_count: int = 0
    link_success_probability: float = 0.0
    sigma_bits: int = 0
```

```
    prefix_sigma: int = 0
    leaf_sigma: int = 0
    root_sigma: int = 0
    verified_sigma: int = 0
    link_grover_iterations: int = 0
    link_statevector_elements_processed: int = 0
    link_statevector_norm_error_max: float = 0.0
    link_statevector_seconds: float = 0.0
    link_success_probability_sum: float = 0.0
    link_search_messages: int = 0
    link_verification_hash_evaluations: int = 0
    link_independent_verification_hash_evaluations: int = 0
    link_seed: Optional[int] = None
    build_method: str = ""
    link_method: str = ""
    link_phase_oracle_queries: int | None = None
    link_verification_queries: int | None = None
    link_failed_attempts: int = 0
    amplitude_backend: str = "dense"
    bbht_options: dict | None = None
    search_protocol: str = "legacy-v0"
    attempt_policy: str = ""
    search_diagnostics: list | None = None


def encode_text_prefix_blocks(text: str, expected_blocks: int) ->
List[str]:
    if expected_blocks <= 0:
        raise ValueError("expected_blocks должен быть положительным.")
    try:
        raw = text.encode("windows-1251")
    except UnicodeEncodeError as exc:
        raise ValueError("Текст содержит символы вне windows-1251.")
from exc
    capacity = expected_blocks * 64
    if len(raw) + 1 > capacity:
        raise ValueError(
            f"Префикс слишком длинный: зарезервировано {capacity} байт,
а требуется {len(raw) + 1}."
        )
    aligned = raw + b"\x80" + b"\x00" * (capacity - len(raw) - 1)
    return [aligned[i:i + 64].hex() for i in range(0, len(aligned), 64)]


def process_prefix_blocks(blocks: Sequence[str], truncate_bits: int,
rounds: int):
    """Обрабатывает префикс без части контрольной суммы для
совместимости."""
    return ReducedGostModel(truncate_bits,
rounds).process_blocks(list(blocks), 0)
```

```
def process_prefix_state(
    blocks: Sequence[str], truncate_bits: int, rounds: int, sigma_bits:
int
) -> tuple[DiamondState, int]:
    return ReducedGostModel(truncate_bits, rounds).process_blocks_pair(
        list(blocks), sigma_bits, 0, 0
    )


def reconstruct_suffix(
    leaves: List[str], structure: List[List[dict]], target_leaf: str
) -> List[str]:
    if target_leaf not in leaves:
        raise ValueError("Целевой лист отсутствует в diamond.")
    suffix = []
    current_state = target_leaf
    for level_index, level in enumerate(structure, start=1):
        selected = None
        for node in level:
            if node["h_left"] == current_state:
                selected = (node["m_link_left"], node["h_common"])
                break
            if node["h_right"] == current_state:
                selected = (node["m_link_right"], node["h_common"])
                break
        if selected is None:
            raise ValueError(f"Для состояния {current_state} нет ребра
на уровне {level_index}.")
        block, current_state = selected
        suffix.append(block)
    return suffix


def find_linking_message(
    search: SearchAlgorithms,
    prefix_state: str,
    target_states: List[str],
    N: int,
    truncate_bits: int,
    trials_limit: int = 1_000_000,
    rounds: int = 12,
    stats: Optional[Dict[str, int]] = None,
    *,
    sigma_bits: int = 0,
    prefix_sigma: int = 0,
    leaf_sigma: int = 0,
):
    result = search.find_link(
```

```
        prefix_state, target_states, N, truncate_bits, rounds,
trials_limit,
        sigma_bits=sigma_bits, current_sigma=prefix_sigma,
target_sigma=leaf_sigma,
    )
    if stats is not None:
        stats["link_queries"] = stats.get("link_queries", 0) +
result.queries
        stats["linking_attempts"] = stats.get("linking_attempts", 0) +
result.queries
    return result.block, result.target_state


def _leaf_sigma(data: dict, leaf_h: str) -> int:
    for item in data["leaves"]:
        if item["state"] == leaf_h:
            return int(item.get("sigma", data.get("leaf_sigma", 0)))
    raise AssertionError("Выбранный leaf отсутствует в JSON.")


def verify_forged_path(
    prefix_blocks: List[str],
    linking_message: str,
    target_leaf: str,
    suffix_blocks: List[str],
    data: dict,
) -> str:
    """Проверяет путь до корня по состоянию (h_n, Σ_s, N)."""
    truncate_bits = int(data["trunc_bits"])
    sigma_bits = int(data.get("sigma_bits", 0))
    rounds = int(data.get("rounds", 12))
    link_N = int(data["link_N"])
    first_level_N = int(data["first_level_N"])
    root_state = get_diamond_root_state(data)
    model = ReducedGostModel(truncate_bits, rounds)

    state, N = process_prefix_state(prefix_blocks, truncate_bits,
rounds, sigma_bits)
    if N != link_N:
        raise AssertionError(f"N после префикса равен {N}, а JSON
ожидает {link_N}.")

    state = model.transition_pair(linking_message, state, N, sigma_bits)
    expected_leaf = DiamondState(target_leaf, _leaf_sigma(data,
target_leaf))
    if state != expected_leaf:
        raise AssertionError(
            f"Linking-блок дал {(state.h, state.sigma)}, ожидается
{(expected_leaf.h, expected_leaf.sigma)}."
        )
```

```
    current = state
    if len(suffix_blocks) != len(data["structure"]):
        raise AssertionError("Длина suffix не совпадает с глубиной
diamond.")
    for level_index, block in enumerate(suffix_blocks):
        level = data["structure"][level_index]
        selected = None
        for node in level:
            left_sigma = int(node.get("sigma_left", 0))
            right_sigma = int(node.get("sigma_right", 0))
            if node["h_left"] == current.h and left_sigma ==
current.sigma and node["m_link_left"] == block:
                selected = node
                break
            if node["h_right"] == current.h and right_sigma ==
current.sigma and node["m_link_right"] == block:
                selected = node
                break
        if selected is None:
            raise AssertionError(f"Suffix block не соответствует
фактическому ребру уровня {level_index + 1}.")
        level_N = first_level_N + 512 * level_index
        current = model.transition_pair(block, current, level_N,
sigma_bits)
        expected_parent = DiamondState(
            selected["h_common"], int(selected.get("sigma_common", 0))
        )
        if current != expected_parent:
            raise AssertionError(
                f"Replay уровня {level_index + 1}: получено {(current.h,
current.sigma)}, "
                f"ожидается {(expected_parent.h,
expected_parent.sigma)}."
            )

    if current != root_state:
        raise AssertionError(
            f"Путь закончился в {(current.h, current.sigma)}, корень
{(root_state.h, root_state.sigma)}."
        )
    return current.h


def _finish_attack(data, description, event_result, prefix_blocks,
prefix_pair,
                   linking_block, leaf_state, root_state, *, leaves,
independent_replay=False,
                   **metrics):
    """Восстанавливает и проверяет общий путь перед созданием результата
```

```
любого метода."""
    suffix = reconstruct_suffix(leaves, data['structure'], leaf_state)
    verified = verify_forged_path(prefix_blocks, linking_block,
leaf_state, suffix, data)
    if independent_replay:
        metrics['link_independent_verification_hash_evaluations'] = (
            2 * sum(len(level) for level in data['structure'])
            + 2 * len(prefix_blocks) + 2 + len(suffix))
    return AttackResult(
        description=description, event_result=event_result,
prefix_blocks=prefix_blocks,
        prefix_state=prefix_pair.h, linking_block=linking_block,
leaf_state=leaf_state,
        suffix_blocks=suffix, root=root_state.h, verified_root=verified,
        forged_hex=''.join(prefix_blocks) + linking_block +
''.join(suffix),
        sigma_bits=int(data.get('sigma_bits', 0)),
prefix_sigma=prefix_pair.sigma,
        leaf_sigma=int(data.get('leaf_sigma', 0)),
root_sigma=root_state.sigma,
        verified_sigma=root_state.sigma,
        build_method=search_method(data.get('builder',
'blackburn')).value, **metrics)


def execute_attack(
    data: dict,
    description: str,
    event_result: str,
    search: SearchAlgorithms,
    trials_limit: int = 2_000_000,
) -> AttackResult:
    validate_diamond_data(data)
    truncate_bits = int(data["trunc_bits"])
    sigma_bits = int(data.get("sigma_bits", 0))
    rounds = int(data.get("rounds", 12))
    prefix_blocks_count = int(data["prefix_blocks"])
    leaves = [x["state"] for x in data["leaves"]]
    root_state = get_diamond_root_state(data)
    leaf_sigma = int(data.get("leaf_sigma", 0))

    prefix_blocks = encode_text_prefix_blocks(description +
event_result, prefix_blocks_count)
    prefix_pair, actual_N = process_prefix_state(prefix_blocks,
truncate_bits, rounds, sigma_bits)
    if actual_N != int(data["link_N"]):
        raise RuntimeError("Длина префикса не соответствует N, с которым
построен diamond.")

    stats: Dict[str, int] = {}
```

```
    linking_message, leaf_state = find_linking_message(
        search, prefix_pair.h, leaves, actual_N, truncate_bits,
        trials_limit=trials_limit, rounds=rounds, stats=stats,
        sigma_bits=sigma_bits, prefix_sigma=prefix_pair.sigma,
leaf_sigma=leaf_sigma,
    )
    return _finish_attack(data, description, event_result,
prefix_blocks, prefix_pair,
                          linking_message, leaf_state, root_state,
leaves=leaves,
                          link_queries=stats.get('link_queries', 0),
                          link_method=SearchMethod.CLASSICAL.value)


class QuantumLinkFailure(RuntimeError):
    """Сообщает о неудаче вероятностных попыток поиска связующего блока
алгоритмом 5."""


@dataclass(frozen=True)
class Algorithm5LinkParameters:
    state_bits: int
    diamond_bits: int
    target_count: int
    search_messages: int


def algorithm5_link_parameters(
    state_bits: int,
    diamond_bits: int,
) -> Algorithm5LinkParameters:
    """Вычисляет S_link = 2^(n-k) для поиска связующего блока к 2^k
листьям."""
    validate_truncate_bits(state_bits)
    if diamond_bits < 1 or diamond_bits > state_bits:
        raise ValueError("diamond_bits должен лежать в диапазоне
1..state_bits.")
    return Algorithm5LinkParameters(
        state_bits=state_bits,
        diamond_bits=diamond_bits,
        target_count=1 << diamond_bits,
        search_messages=1 << (state_bits - diamond_bits),
    )


def _random_message_space(
    count: int,
    rng: random.Random,
    *, sigma_bits: int = 0, current_sigma: int = 0, target_sigma: int =
0,
```

```
) -> list[str]:
    messages: list[str] = []
    used: set[str] = set()
    while len(messages) < count:
        block = constrained_random_block(current_sigma, target_sigma,
sigma_bits, rng)
        if block not in used:
            used.add(block)
            messages.append(block)
    return messages


def find_linking_message_quantum(
    prefix_state: str,
    target_states: Sequence[str],
    N: int,
    truncate_bits: int,
    *,
    rounds: int = 12,
    trials_limit: int = 2_000_000,
    max_attempts: int = 12,
    max_statevector_size: Optional[int] = None,
    rng: Optional[random.Random] = None,
    stats: Optional[Dict[str, Any]] = None,
    use_cpp: Optional[bool] = None,
    sigma_bits: int = 0,
    prefix_sigma: int = 0,
    leaf_sigma: int = 0,
    threads: int = 0,
    method: str = "grover_fixed",
    bbht_options: BBHTOptions | None = None,
) -> tuple[str, str]:
    """Ищет связующий блок алгоритмом 5 через C++ или резервное
моделирование Гровера на Python."""
    selected = search_method(method)
    validate_truncate_bits(truncate_bits)
    validate_sigma_bits(sigma_bits)
    targets = list(dict.fromkeys(target_states))
    target_count = len(targets)
    if target_count < 2 or target_count & (target_count - 1):
        raise ValueError("Число различных целевых листьев должно быть
степенью двойки >= 2.")
    diamond_bits = int(math.log2(target_count))
    params = algorithm5_link_parameters(truncate_bits, diamond_bits)
    if (
        max_statevector_size is not None
        and params.search_messages > max_statevector_size
    ):
        raise ValueError(
            f"Algorithm 5 требует state-vector
```

```
S_link={params.search_messages}, "
            f"лимит симулятора={max_statevector_size}."
        )
    if trials_limit < params.search_messages:
        raise ValueError(
            f"Лимит симулятора должен быть не меньше
S_link={params.search_messages}."
        )
    if max_attempts < 1:
        raise ValueError("max_attempts должен быть положительным.")

    generator = rng or random.Random()
    model = ReducedGostModel(truncate_bits, rounds)
    target_set = set(targets)
    cpp_enabled = cpp_core_available() if use_cpp is None else use_cpp
    if selected is SearchMethod.GROVER_BBHT and not cpp_enabled:
        raise RuntimeError("BBHT требует собранное C++-ядро.")
    phase_queries = 0
    failed_attempts = 0
    search_attempts = 0
    iterations_total = 0
    total_evaluations = 0
    total_queries = 0
    total_marked = 0
    vector_elements = 0
    vector_seconds = 0.0
    norm_max = 0.0
    probability_sum = 0.0
    verification_evaluations = 0
    iterations = optimal_grover_iterations(params.search_messages, 1)
    other_bytes = 262144 + len(targets) * 256
    if cpp_enabled:
        check_native_resources(params.search_messages, iterations,
other_bytes,
            dense_max_states=(bbht_options or
BBHTOptions()).dense_max_states if selected is SearchMethod.GROVER_BBHT
else DENSE_MAX_STATES)
    else:
        check_resources(params.search_messages, iterations, other_bytes
+ params.search_messages * 256, python=True)

    for attempt in range(1, max_attempts + 1):
        check_deadline()
        if total_evaluations + params.search_messages + 1 >
trials_limit:
            raise QuantumLinkFailure(
                "Algorithm 5 исчерпал лимит классических вычислений
симулятора: "
                f"{total_evaluations}/{trials_limit}."
            )
```

```
        if cpp_enabled:
            effective_options = bbht_options or BBHTOptions()
            if selected is SearchMethod.GROVER_BBHT:
                effective_options = replace(effective_options,
max_attempts=min(effective_options.max_attempts,
                    trials_limit - total_evaluations -
params.search_messages))
            result = cpp_quantum_link(
                prefix_state,
                targets,
                params.search_messages,
                N,
                truncate_bits,
                rounds,
                generator.getrandbits(64),
                sigma_bits=sigma_bits,
                current_sigma=prefix_sigma,
                target_sigma=leaf_sigma,
                threads=threads,
                **({"method": selected, "bbht_options":
effective_options} if selected is SearchMethod.GROVER_BBHT else {}),
            )
            if selected is SearchMethod.GROVER_BBHT:
                phase_queries += result.phase_oracle_queries
                failed_attempts += result.failed_attempts
                search_attempts += result.search_attempts
                iterations_total += result.statevector_iterations
            block = result.block
            target = result.target_state
            queries = result.oracle_queries
            evaluations = result.simulator_evaluations
            marked_count = result.marked_count
            success_probability = result.success_probability
            norm_error = result.norm_error
            elements = result.statevector_elements_processed
            seconds = result.statevector_seconds
            verified_evaluations = result.verification_evaluations
        else:
            messages = _random_message_space(
                params.search_messages, generator,
sigma_bits=sigma_bits,
                current_sigma=prefix_sigma, target_sigma=leaf_sigma,
            )
            outputs = []
            for index, block in enumerate(messages):
                if index % 256 == 0:
                    check_deadline()
                outputs.append(model.transition(block, prefix_state, N))
            marked = [index for index, output in enumerate(outputs) if
```

```
output in target_set]
            grover: GroverResult = simulate_grover_from_marked(
                params.search_messages,
                marked,

iterations=optimal_grover_iterations(params.search_messages, 1),
                rng=generator,
                max_statevector_size=max_statevector_size,
            )
            block = messages[grover.measured_index] if grover.is_marked
else None
            target = outputs[grover.measured_index] if grover.is_marked
else None
            queries = grover.quantum_queries + 1
            if model.transition(messages[grover.measured_index],
prefix_state, N) != outputs[grover.measured_index]:
                raise AssertionError('Измеренный переход не
воспроизводится.')
            evaluations = params.search_messages + 1
            elements = grover.statevector_elements_processed
            seconds = grover.statevector_seconds
            verified_evaluations = 1
            marked_count = grover.marked_count
            success_probability = grover.success_probability
            norm_error = grover.norm_error

        total_evaluations += evaluations
        total_queries += queries
        total_marked += marked_count
        vector_elements += elements
        vector_seconds += seconds
        norm_max = max(norm_max, norm_error)
        probability_sum += success_probability * (result.search_attempts
if cpp_enabled and selected is SearchMethod.GROVER_BBHT else 1)
        verification_evaluations += verified_evaluations
        if norm_error > 1e-9:
            raise AssertionError(f"Нарушена нормировка Grover state-
vector: {norm_error}.")

        if stats is not None:
            if cpp_enabled:
                stats['quantum_link_amplitude_backend'] =
result.amplitude_backend
            stats['search_protocol'] = result.search_protocol if
cpp_enabled else SEARCH_PROTOCOL
            diagnostics = result.search_diagnostics if cpp_enabled else
(dict(attempt_policy='fixed_single',
                search_messages=params.search_messages, growth=1.2,
effective_max_attempts=1,
                attempts=1, failures=int(block is None),
```

```
stop_reason='found' if block is not None else 'unmarked_measurement'),)
            stats.setdefault('quantum_link_search_diagnostics',
[]).extend(dict(item, batch_attempt=attempt) for item in diagnostics)
            stats['quantum_link_phase_oracle_queries'] =
iterations_total if selected is SearchMethod.GROVER_BBHT else attempt *
iterations
            stats['quantum_link_verification_queries'] =
verification_evaluations
            stats['quantum_link_failed_attempts'] = failed_attempts if
selected is SearchMethod.GROVER_BBHT else sum(item['failures'] for item
in stats['quantum_link_search_diagnostics'])

stats.update(quantum_link_statevector_elements_processed=vector_elements
,

quantum_link_statevector_seconds=vector_seconds,
quantum_link_statevector_norm_error_max=norm_max,

quantum_link_success_probability_sum=probability_sum,

quantum_link_verification_hash_evaluations=verification_evaluations)
            stats["quantum_link_oracle_queries"] = total_queries
            stats["quantum_link_grover_iterations"] = iterations_total
if selected is SearchMethod.GROVER_BBHT else attempt *
optimal_grover_iterations(params.search_messages, 1)
            if selected is SearchMethod.GROVER_BBHT:

stats.update(quantum_link_phase_oracle_queries=phase_queries,

quantum_link_verification_queries=verification_evaluations,

quantum_link_failed_attempts=failed_attempts,

quantum_link_amplitude_backend=result.amplitude_backend,
                                quantum_link_bbht_attempts=search_attempts)
            stats["quantum_link_simulator_hash_evaluations"] =
total_evaluations
            stats["quantum_link_marked_states"] = total_marked
            stats["quantum_link_attempts"] = attempt
            stats["quantum_link_success_probability"] =
success_probability
            stats["quantum_link_search_messages"] =
params.search_messages
            backend_key = (
                "quantum_link_cpp_calls" if cpp_enabled else
"quantum_link_python_calls"
            )
            stats[backend_key] = stats.get(backend_key, 0) + 1

        if block is None or target is None:
```

```
            continue
        if target not in target_set:
            raise AssertionError("Grover вернул состояние вне множества
листьев.")
        if model.transition(block, prefix_state, N) != target:
            raise AssertionError("Linking-блок Algorithm 5 не
воспроизводится.")
        if update_sigma_reduced(prefix_sigma, block, sigma_bits) !=
leaf_sigma:
            raise AssertionError("Quantum linking-блок нарушает sigma
constraint.")
        return block, target

    raise QuantumLinkFailure(
        f"Algorithm 5 не нашел linking-блок за {max_attempts} попыток "
        f"по S_link={params.search_messages} сообщений."
    )


def execute_algorithm5(
    data: dict,
    description: str,
    event_result: str,
    *,
    trials_limit: int = 2_000_000,
    seed: Optional[int] = None,
    use_cpp: Optional[bool] = None,
    threads: int = 0,
    method: str = "grover_fixed",
    bbht_options: BBHTOptions | None = None,
) -> AttackResult:
    """Выполняет квантовую herding-атаку алгоритмом 5 для модели части
хэш-кода."""
    effective_seed = random.SystemRandom().getrandbits(64) if seed is
None else seed
    validate_diamond_data(data)
    truncate_bits = int(data["trunc_bits"])
    rounds = int(data.get("rounds", 12))
    prefix_blocks_count = int(data["prefix_blocks"])
    leaves = [item["state"] for item in data["leaves"]]
    sigma_bits = int(data.get("sigma_bits", 0))
    leaf_sigma = int(data.get("leaf_sigma", 0))
    root = get_diamond_root(data)
    root_sigma = int(data.get("sigma_schedule", [0])[-1])

    prefix_blocks = encode_text_prefix_blocks(
        description + event_result, prefix_blocks_count
    )
    prefix_pair, actual_N = process_prefix_state(
        prefix_blocks, truncate_bits, rounds, sigma_bits
```

```
    )
    if actual_N != int(data["link_N"]):
        raise RuntimeError("Длина префикса не соответствует N quantum
diamond.")

    stats: Dict[str, Any] = {}
    try:
        linking_block, leaf_state = find_linking_message_quantum(
            prefix_pair.h,
            leaves,
            actual_N,
            truncate_bits,
            rounds=rounds,
            trials_limit=trials_limit,
            rng=random.Random(effective_seed),
            stats=stats,
            use_cpp=use_cpp,
            sigma_bits=sigma_bits,
            prefix_sigma=prefix_pair.sigma,
            leaf_sigma=leaf_sigma,
            threads=threads if threads else (1 if seed is not None else
0),
            method=method, bbht_options=bbht_options,
        )
    except Exception as exc:
        exc.experiment_stats = stats.copy()
        raise
    metrics = {}
    for convert, fields in (
        (int, ('simulator_hash_evaluations', 'grover_iterations',
               'statevector_elements_processed', 'search_messages',
               'verification_hash_evaluations')),
        (float, ('success_probability', 'statevector_norm_error_max',
                 'statevector_seconds', 'success_probability_sum')),
    ):
        metrics.update(('link_' + key, convert(stats.get('quantum_link_'
+ key, 0)))
                       for key in fields)
    return _finish_attack(
        data, description, event_result, prefix_blocks, prefix_pair,
        linking_block, leaf_state, DiamondState(root, root_sigma),
leaves=leaves,
        independent_replay=True,
        link_queries=int(stats.get("quantum_link_oracle_queries", 0)),
        link_backend=(
            "quantum-bbht-cpp" if search_method(method) is
SearchMethod.GROVER_BBHT else "quantum-cpp" if
stats.get("quantum_link_cpp_calls", 0)
            else "quantum-python"
        ),
```

```
        link_attempts=int(stats.get("quantum_link_bbht_attempts",
stats.get("quantum_link_attempts", 0))),
        link_marked_count=int(stats.get("quantum_link_marked_states",
0)),
        link_seed=effective_seed,
        link_method=search_method(method).value,

link_phase_oracle_queries=stats.get('quantum_link_phase_oracle_queries')
,

link_verification_queries=stats.get('quantum_link_verification_queries')
,
        link_failed_attempts=stats.get('quantum_link_failed_attempts',
0),
        amplitude_backend=stats.get('quantum_link_amplitude_backend',
'dense'),
        search_protocol=stats.get('search_protocol', 'legacy-v0'),
        attempt_policy='bbht_auto' if search_method(method) is
SearchMethod.GROVER_BBHT else 'fixed_single',
        search_diagnostics=stats.get('quantum_link_search_diagnostics',
[]),
        bbht_options=asdict(bbht_options or BBHTOptions()) if
search_method(method) is SearchMethod.GROVER_BBHT else None,
        **metrics,
    )
```

ПРИЛОЖЕНИЕ И

Поиск связывающего блока тремя методами на языке C++

```
#pragma once

#include <atomic>
#include <chrono>
#include <cstdlib>
#include <iostream>
#include <random>
#include <unordered_set>
#include "gost_reduced_model.hpp"
#include "grover_analytic.hpp"
#include "grover_statevector.hpp"
#include "grover_symmetric.hpp"
#include "bbht_adaptive.hpp"
#include "search_runtime.hpp"
#include <algorithm>
#include <cstdint>
#include <stdexcept>
#include <string>
#include <vector>


// Реализует оба способа поиска связывающего блока с сохранением
исходного порядка кандидатов и измерений.
namespace herding_link {

using gost_reduced_model::make_context;
using gost_reduced_model::reduced_state_from_hex;
using gost_reduced_model::required_sigma_low64;
using gost_reduced_model::random_state_constrained;
using gost_reduced_model::set_low_sigma_bits;
using grover_analytic::optimal_grover_iterations;

struct LinkResult {
    State block{};
    State state{};
    uint64_t queries = 0;
    bool ok = false;
};

template<class TargetSet,class Key>
inline LinkResult seeded_link(const State& prefix,uint64_t N,int n,int
rounds,uint64_t limit,
    int threads,const TargetSet& targets,int s,uint64_t current,uint64_t
target,Key key) {
    const auto context=make_context(prefix,N,rounds);
```

```
    std::mt19937_64 rng(mix64(requested_seed));
    auto fast_rng=optional_candidate_rng();
    std::vector<State> messages,outputs;
    const uint64_t low=required_sigma_low64(current,target,s);
    WorkerPool pool(threads,std::min<uint64_t>(4096,limit),1024);
    // Увеличивает пакеты от восьми кандидатов для сокращения лишних
вычислений и расходов диспетчеризации потоков.

    uint64_t goal=SEARCH_BATCH;
    uint64_t physical=0;
    for(uint64_t offset=0;offset<limit;) {
        const uint64_t batch_size=std::min(goal,limit-offset);
        messages.resize(static_cast<size_t>(batch_size));
        fill_candidates(messages,rng,fast_rng.get(),low,s);
        hash_seeded_batch(context,messages,outputs,threads,&pool);
        physical+=batch_size;
        for(size_t
j=0;j<outputs.size();++j)if(targets.find(key(outputs[j],n))!=targets.end
()) {
            const uint64_t
logical=std::min<uint64_t>(batch_size,((j/SEARCH_BATCH)+1)*SEARCH_BATCH)
;

if(std::getenv("NOSTRADAMUS_SEARCH_DEBUG"))std::cerr<<"physical_hash_eva
luations="<<physical<<" logical_queries="<<offset+logical<<'\n';
            return {messages[j],outputs[j],offset+logical,true};
        }
        offset+=batch_size;goal=std::min<uint64_t>(4096,goal*2);
    }
    throw std::runtime_error("link not found within trials_limit");
}

inline LinkResult find_link64(const State& prefix, uint64_t N, int n,
int rounds, uint64_t limit,
                              int requested_threads, const
std::vector<std::string>& targets, int sigma_bits = 0,
                              uint64_t current_sigma = 0, uint64_t
target_sigma = 0) {
    const uint64_t required_low = required_sigma_low64(current_sigma,
target_sigma, sigma_bits);
    const int nt = workers_for_limit(requested_threads, limit, 1);
    const CompressionContext ctx = make_context(prefix, N, rounds);
    std::unordered_set<uint64_t> targetset;
    targetset.reserve(targets.size() * 2 + 1);
    for (const auto& target : targets) {
        targetset.insert(reduced_key64(reduced_state_from_hex(target,
n), n));
    }
    if(has_requested_seed && (nthreads_from(requested_threads)!=1 ||
requested_rng!="mt19937"))return
```

```
seeded_link(prefix,N,n,rounds,limit,requested_threads,targetset,sigma_bi
ts,current_sigma,target_sigma,reduced_key64);

    std::atomic<bool> found(false);
    std::vector<uint64_t> completed(static_cast<size_t>(nt), 0);
    LinkResult result;

    auto worker = [&](int tid) {
        const WorkRange range = work_range_for(limit, nt, tid);
        const uint64_t quota = range.end - range.begin;
        uint64_t local_queries = 0;
        std::seed_seq seq{uint32_t(std::random_device{}()),

uint32_t(std::chrono::high_resolution_clock::now().time_since_epoch().co
unt()),
                          uint32_t(tid), uint32_t(range.begin),
uint32_t(range.begin >> 32)};
        std::mt19937_64 rng(seq);
        if (has_requested_seed) rng.seed(mix64(requested_seed ^
uint64_t(tid) ^ range.begin));
        SearchBatch messages{};
        SearchBatch outputs{};

        while (local_queries < quota &&
!found.load(std::memory_order_relaxed)) {
            const size_t count = static_cast<size_t>(
                std::min<uint64_t>(SEARCH_BATCH, quota -
local_queries));
            for (size_t lane = 0; lane < count; ++lane) messages[lane] =
random_state_constrained(rng, required_low, sigma_bits);
            compress_batch(messages, outputs, count, ctx);
            local_queries += count;
            for (size_t lane = 0; lane < count; ++lane) {
                if (targetset.find(reduced_key64(outputs[lane], n)) !=
targetset.end()) {
                    bool expected = false;
                    if (found.compare_exchange_strong(expected, true,

std::memory_order_relaxed)) {
                        result.block = messages[lane];
                        result.state = outputs[lane];
                        result.ok = true;
                    }
                    break;
                }
            }
        }
        completed[static_cast<size_t>(tid)] = local_queries;
    };
```

```
    std::vector<std::thread> threads;
    threads.reserve(static_cast<size_t>(nt));
    for (int i = 0; i < nt; ++i) threads.emplace_back(worker, i);
    for (auto& x : threads) x.join();
    if (result.ok) { result.queries = sum_completed(completed); return
result; }
    throw std::runtime_error("link not found within trials_limit");
}

inline LinkResult find_link_full(const State& prefix, uint64_t N, int n,
int rounds, uint64_t limit,
                                 int requested_threads, const
std::vector<std::string>& targets, int sigma_bits = 0,
                                 uint64_t current_sigma = 0, uint64_t
target_sigma = 0) {
    const uint64_t required_low = required_sigma_low64(current_sigma,
target_sigma, sigma_bits);
    const int nt = workers_for_limit(requested_threads, limit, 1);
    const CompressionContext ctx = make_context(prefix, N, rounds);
    std::unordered_set<ReducedKey, ReducedKeyHash> targetset;
    targetset.reserve(targets.size() * 2 + 1);
    for (const auto& target : targets)
targetset.insert(reduced_key_full(reduced_state_from_hex(target, n),
n));
    if(has_requested_seed && (nthreads_from(requested_threads)!=1 ||
requested_rng!="mt19937"))return
seeded_link(prefix,N,n,rounds,limit,requested_threads,targetset,sigma_bi
ts,current_sigma,target_sigma,reduced_key_full);

    std::atomic<bool> found(false);
    std::vector<uint64_t> completed(static_cast<size_t>(nt), 0);
    LinkResult result;

    auto worker = [&](int tid) {
        const WorkRange range = work_range_for(limit, nt, tid);
        const uint64_t quota = range.end - range.begin;
        uint64_t local_queries = 0;
        std::seed_seq seq{uint32_t(std::random_device{}()),

uint32_t(std::chrono::high_resolution_clock::now().time_since_epoch().co
unt()),
                          uint32_t(tid), uint32_t(range.begin),
uint32_t(range.begin >> 32)};
        std::mt19937_64 rng(seq);
        if (has_requested_seed) rng.seed(mix64(requested_seed ^
uint64_t(tid) ^ range.begin));
        SearchBatch messages{};
        SearchBatch outputs{};

        while (local_queries < quota &&
```

```
!found.load(std::memory_order_relaxed)) {
            const size_t count = static_cast<size_t>(
                std::min<uint64_t>(SEARCH_BATCH, quota -
local_queries));
            for (size_t lane = 0; lane < count; ++lane) messages[lane] =
random_state_constrained(rng, required_low, sigma_bits);
            compress_batch(messages, outputs, count, ctx);
            local_queries += count;
            for (size_t lane = 0; lane < count; ++lane) {
                if (targetset.find(reduced_key_full(outputs[lane], n))
!= targetset.end()) {
                    bool expected = false;
                    if (found.compare_exchange_strong(expected, true,

std::memory_order_relaxed)) {
                        result.block = messages[lane];
                        result.state = outputs[lane];
                        result.ok = true;
                    }
                    break;
                }
            }
        }
        completed[static_cast<size_t>(tid)] = local_queries;
    };

    std::vector<std::thread> threads;
    threads.reserve(static_cast<size_t>(nt));
    for (int i = 0; i < nt; ++i) threads.emplace_back(worker, i);
    for (auto& x : threads) x.join();
    if (result.ok) { result.queries = sum_completed(completed); return
result; }
    throw std::runtime_error("link not found within trials_limit");
}

inline LinkResult find_link(const std::string& prefix_hex, uint64_t N,
int n, int rounds, uint64_t limit,
                            int requested_threads, const
std::vector<std::string>& targets, int sigma_bits = 0,
                            uint64_t current_sigma = 0, uint64_t
target_sigma = 0) {
    const State prefix = reduced_state_from_hex(prefix_hex, n);
    return n <= 64 ? find_link64(prefix, N, n, rounds, limit,
requested_threads, targets, sigma_bits, current_sigma, target_sigma)
                   : find_link_full(prefix, N, n, rounds, limit,
requested_threads, targets, sigma_bits, current_sigma, target_sigma);
}

class QuantumTargets {
public:
```

```
    QuantumTargets(const std::vector<std::string>& values,int bits):
bits_(bits) {
        if(bits<=64 && requested_qram!="generic") {

small_=std::make_unique<FlatTable64<uint8_t>>(values.size());small_-
>begin_epoch();
            for(const auto& value:values)small_-
>insert(reduced_key64(reduced_state_from_hex(value,bits),bits),1);
        }else{
            generic_.reserve(values.size()*2+1);
            for(const auto&
value:values)generic_.insert(reduced_key_full(reduced_state_from_hex(val
ue,bits),bits));
        }
    }
    bool contains(const State& output)const {
        return small_?small_-
>find(reduced_key64(output,bits_))!=nullptr:generic_.find(reduced_key_fu
ll(output,bits_))!=generic_.end();
    }
private:
    int bits_;
    std::unique_ptr<FlatTable64<uint8_t>> small_;
    std::unordered_set<ReducedKey,ReducedKeyHash> generic_;
};

struct QuantumLinkResult {
    State block{};
    State target{};
    uint64_t oracle_queries = 0;
    uint64_t simulator_evaluations = 0;
    uint64_t marked_count = 0;
    double success_probability = 0.0;
    double norm_error = 0.0;
    bool found = false;
    uint64_t statevector_elements_processed = 0;
    uint64_t statevector_iterations = 0;
    uint64_t verification_evaluations = 0;
    double statevector_seconds = 0.0;
    std::vector<bbht_adaptive::Result> search_diagnostics;
};

} // Завершает пространство имен поиска связывающего блока.

// Реализует BBHT над общим пространством допустимых кандидатов ГОСТ.
namespace herding_link_bbht {

struct Result {
    herding_link::QuantumLinkResult link;
    bbht_adaptive::Result search;
```

```
    uint64_t candidate_evaluations = 0;
};

inline Result find(
    const std::string& prefix_hex,
    const std::vector<std::string>& target_hex,
    uint64_t search_messages, uint64_t N, int n, int rounds,
    uint64_t seed, int requested_threads,
    int sigma_bits = 0, uint64_t current_sigma = 0, uint64_t
target_sigma = 0,
    const bbht_adaptive::Options& options = {}, bool fixed_control =
false
) {
    using namespace gost_reduced_model;
    if (target_hex.empty() || !search_messages)
        throw std::runtime_error("BBHT linking: пустые цели или
пространство поиска.");
    if (!fixed_control) {
        bbht_adaptive::effective_max_attempts(search_messages,
options.growth, options.max_attempts);
        if (!options.max_oracle_queries) throw std::runtime_error("BBHT:
предел запросов должен быть положительным.");
    }
    const auto dense_limit = fixed_control ?
grover_oracle::dense_max_states : options.dense_max_states;
    if (search_messages <= dense_limit)
        grover_statevector::check_resources(search_messages, 0, 262144 +
target_hex.size() * 256);
    else grover_oracle::check_resources(search_messages, 262144 +
target_hex.size() * 256);
    if (search_messages > std::numeric_limits<size_t>::max())
        throw std::runtime_error("BBHT linking: candidate mask cannot
fit address space");
    const uint64_t low = required_sigma_low64(current_sigma,
target_sigma, sigma_bits);
    const auto context = make_context(reduced_state_from_hex(prefix_hex,
n), N, rounds);
    const herding_link::QuantumTargets targets(target_hex, n);
    auto message_for_index = [&](uint64_t index) {
        State block{};
        block[0] = mix64(index + seed);
        for (size_t word = 1; word < block.size(); ++word)
            block[word] = mix64(index ^ seed ^ (0x9e3779b97f4a7c15ULL *
word));
        set_low_sigma_bits(block, low, sigma_bits);
        return block;
    };
    grover_oracle::MarkedSpace marked(search_messages, dense_limit,
262144 + target_hex.size() * 256);
    std::vector<State> outputs;
```

```
    WorkerPool pool(requested_threads, std::min<uint64_t>(4096,
search_messages), 1024);
    const auto started = std::chrono::steady_clock::now();
    grover_oracle::Deadline fixed_deadline(search_messages >
dense_limit);
    auto check_deadline = [&]() {
        if(fixed_control) fixed_deadline.check();
        if(!fixed_control && options.max_stage_seconds > 0 &&

std::chrono::duration<double>(std::chrono::steady_clock::now()-
started).count()
           > options.max_stage_seconds) throw std::runtime_error("BBHT
linking: stage timeout");
    };
    for (uint64_t index = 0; index < search_messages;) {
        const size_t count =
static_cast<size_t>(std::min<uint64_t>(4096, search_messages - index));
        outputs.resize(count);
        pool.run(count, 1024, [&](WorkRange range) {
            SearchBatch messages{}, hashes{};
            for (uint64_t i = range.begin; i < range.end;) {
                const size_t batch =
static_cast<size_t>(std::min<uint64_t>(SEARCH_BATCH, range.end - i));
                for (size_t lane = 0; lane < batch; ++lane)
                    messages[lane] = message_for_index(index + i +
lane);
                compress_batch(messages, hashes, batch, context);
                for (size_t lane = 0; lane < batch; ++lane)
                    outputs[static_cast<size_t>(i + lane)] =
hashes[lane];
                i += batch;
            }
        });
        for (size_t lane = 0; lane < count; ++lane)
            if (targets.contains(outputs[lane])) marked.add(index +
lane);
        check_deadline();
        index += count;
    }
    Result result;
    result.candidate_evaluations = search_messages;
    std::mt19937_64 rng(mix64(seed ^ 0xa5a5a5a5a5a5a5a5ULL));
    State measured_output{};
    auto verify = [&](size_t index) {
        measured_output =
compress_with_context(message_for_index(index), context);
        return targets.contains(measured_output);
    };
    result.search = fixed_control
        ? bbht_adaptive::fixed_single(marked, rng, verify)
```

```
        : bbht_adaptive::search(marked, rng, verify, options);
    result.link.search_diagnostics.push_back(result.search);
    result.link.oracle_queries = result.search.oracle_queries;
    result.link.simulator_evaluations = search_messages +
result.search.verification_queries;
    result.link.verification_evaluations =
result.search.verification_queries;
    result.link.marked_count = result.search.marked_count_diagnostic;
    result.link.success_probability = result.search.attempts
        ? result.search.success_probability_sum / result.search.attempts
: 0.0;
    result.link.norm_error = result.search.norm_error_max;
    result.link.statevector_elements_processed =
result.search.statevector_elements_processed;
    result.link.statevector_iterations =
result.search.statevector_iterations;
    result.link.statevector_seconds = result.search.statevector_seconds;
    if (result.search.found) {
        result.link.block =
message_for_index(result.search.measured_index);
        result.link.target = measured_output;  // Возвращает уже
проверенное состояние без скрытого дополнительного вычисления.
        result.link.found = true;
    }
    check_deadline();
    return result;
}

} // namespace herding_link_bbht

namespace herding_link {
inline QuantumLinkResult find_quantum_link(
    const std::string& prefix, const std::vector<std::string>& targets,
    uint64_t S, uint64_t N, int n, int rounds, uint64_t seed, int
threads,
    int sigma_bits = 0, uint64_t current_sigma = 0, uint64_t
target_sigma = 0) {
    return
herding_link_bbht::find(prefix,targets,S,N,n,rounds,seed,threads,
        sigma_bits,current_sigma,target_sigma,{},true).link;
}
}
```

ПРИЛОЖЕНИЕ К

Управление пользовательским сценарием herding-атаки на языке Python

```
"""Управляет стадиями сценария Нострадамуса без оформления виджетов."""

from enum import Enum
from algorithms.diamond import get_diamond_root_state
from algorithms.herding_attack import encode_text_prefix_blocks
from algorithms.search_methods import search_method


class HerdingStage(str, Enum):
    EMPTY = "empty"
    LOADED = "loaded"
    COMMITTED = "committed"
    RUNNING = "running"
    COMPLETED = "completed"
    ERROR = "error"


class HerdingWorkflow:
    def __init__(self, controller):
        self.controller = controller
        self.data = None
        self.description = None
        self.stage = HerdingStage.EMPTY
        self.result = None

    def load(self, data):
        if self.stage is HerdingStage.RUNNING:
            raise RuntimeError("Дождитесь завершения поиска или
остановите его.")
        self.data = data
        self.description = None
        self.result = None
        self.stage = HerdingStage.LOADED

    def commit(self, description):
        if self.data is None:
            raise ValueError("Сначала загрузите алмазную структуру.")
        if not description:
            raise ValueError("Введите публикуемую часть префикса.")
        if self.stage is HerdingStage.RUNNING:
            raise RuntimeError("Сначала остановите поиск связующего
блока.")
        encode_text_prefix_blocks(description,
int(self.data["prefix_blocks"]))
        self.description = description
```

```
        self.stage = HerdingStage.COMMITTED
        return get_diamond_root_state(self.data).h

    def start(self, description, event_result, trials_limit, method, *,
bbht_options=None):
        if self.description is None:
            raise ValueError("Сначала зафиксируйте коммитмент.")
        if description != self.description:
            raise ValueError("Публикуемая часть изменилась; зафиксируйте
коммитмент заново.")
        if self.stage is HerdingStage.RUNNING:
            raise RuntimeError("Поиск связующего блока уже
выполняется.")
        if trials_limit < 1:
            raise ValueError("Лимит поиска должен быть положительным.")
        encode_text_prefix_blocks(description + event_result,
int(self.data["prefix_blocks"]))
        selected = search_method(method)
        if bbht_options is not None:
            bbht_options.validate()
        self.stage = HerdingStage.RUNNING
        try:
            self.controller.start_attack(self.data, description,
event_result, trials_limit, selected.value,
                                         **({"bbht_options":
bbht_options} if bbht_options is not None else {}))
        except Exception:
            self.stage = HerdingStage.ERROR
            raise

    def complete(self, result):
        root = get_diamond_root_state(self.data)
        if result.root != root.h or result.verified_root != root.h or
getattr(result, "verified_sigma", root.sigma) != root.sigma:
            self.stage = HerdingStage.ERROR
            raise ValueError("Независимая проверка не подтвердила
зафиксированное корневое состояние.")
        self.result = result
        self.stage = HerdingStage.COMPLETED

    def cancel(self):
        self.result = None
        self.stage = HerdingStage.COMMITTED if self.description is not
None else HerdingStage.LOADED

    def fail(self):
        self.result = None
        self.stage = HerdingStage.ERROR
```

ПРИЛОЖЕНИЕ Л

Командный интерфейс единого программного комплекса на языке Python

```
"""Командный интерфейс HerdingLab с запуском графического интерфейса
через main.py."""
from __future__ import annotations

import argparse
import json
import os
import subprocess
import sys
import time
import traceback
from dataclasses import asdict
from pathlib import Path

from algorithms.search_methods import BBHTOptions, SearchMethod,
search_method
from integration.backend_adapter import BackendAdapter
from integration.campaign_runner import ROOT, build_to_path,
environment, full_hashes, run_campaign
from integration.experiment_store import ExperimentStore
from integration.result_adapter import ExperimentRecord
from algorithms.statevector_limits import MODEL_REVISION


def common(parser: argparse.ArgumentParser, suppress: bool = False) ->
None:
    default = argparse.SUPPRESS if suppress else False
    flags = parser.add_mutually_exclusive_group()
    flags.add_argument("--quiet", action="store_true", default=default,
help="Suppress progress")
    flags.add_argument("--verbose", action="store_true",
default=default, help="Detailed logs and traceback")
    parser.add_argument("--json", action="store_true", default=default,
help="Print final result as JSON")


def parser() -> argparse.ArgumentParser:
    root = argparse.ArgumentParser(description="HerdingLab: reduced-GOST
diamond/herding experiments")
    common(root)
    commands = root.add_subparsers(dest="command", required=True)
    for name, help_text in (("selftest", "Run every test suite"),
("validate-diamond", "Replay every edge"),
                            ("build-diamond", "Build a diamond"),
("attack", "Find a linking block and replay the path"),
```

```
                            ("campaign", "Run a reproducible
experimental series"), ("analyze", "Create statistical reports and PNG
charts")):
        command = commands.add_parser(name, help=help_text)
        common(command, True)
    commands.choices["validate-diamond"].add_argument("path", type=Path)
    build = commands.choices["build-diamond"]
    build.add_argument("--n", "--t", dest="n", required=True, type=int,
help="Размер части хэш-кода n, бит")
    build.add_argument("--k", required=True, type=int, help="Глубина
алмазной структуры k")
    build.add_argument("--prefix-blocks", type=int, default=1)
    build.add_argument("--rounds", type=int, default=12)
    build.add_argument("--sigma-bits", type=int, default=0)
    build.add_argument('--sigma-seed', type=int, default=0, help='Seed
части контрольной суммы, как в GUI.')
    build.add_argument("--parameter-strategy", choices=("theoretical",
"target_probability"), default="theoretical")
    build.add_argument("--target-probability", type=float, default=0.95)
    build.add_argument("--output", required=True, type=Path)
    attack = commands.choices["attack"]
    attack.add_argument("--diamond", required=True, type=Path)
    attack.add_argument("--description", default="Эксперимент ")
    attack.add_argument("--event-result", default="успешен")
    attack.add_argument("--output", type=Path, help="Save complete
attack transcript")
    for item in (build, attack):
        item.add_argument("--method", choices=("classical", "quantum",
"grover_fixed", "grover_bbht", "bbht", "blackburn",
"quantum_algorithm_2_simulator"), required=True)
        item.add_argument("--bbht-growth", type=float, default=1.2,
help="Коэффициент роста BBHT.")
        item.add_argument("--bbht-max-attempts", type=int, default=128,
help="Максимум попыток BBHT; фактический лимит автоматически
рассчитывается по S.")
        item.add_argument("--bbht-max-queries", type=int,
default=100000, help="Модельных запросов на поиск BBHT.")
        item.add_argument("--bbht-dense-max-states", type=int,
default=8192, help="Порог плотного вектора BBHT.")
        item.add_argument("--bbht-stage-seconds", type=float, default=0,
help="Время этапа BBHT в секундах.")
        item.add_argument("--seed", type=int)
        item.add_argument("--threads", type=int, default=0)
        item.add_argument("--qram-impl", choices=("auto", "generic",
"u64"), default="auto")
        item.add_argument("--rng", choices=("mt19937", "xoshiro256ss"),
default="mt19937")
        item.add_argument("--blackburn-backend", choices=("legacy",
"cpp-layer"), default="legacy")
        item.add_argument("--trials-limit", type=int,
```

```
default=1_000_000_000)
        item.add_argument("--record-db", type=Path, default=ROOT /
"experiments/search_policy_v1.json")
    campaign = commands.choices["campaign"]
    campaign.add_argument("--config", required=True, type=Path)
    campaign.add_argument("--resume", action="store_true")
    campaign.add_argument("--record-db", type=Path)
    analysis = commands.choices["analyze"]
    analysis.add_argument("--database", required=True, type=Path)
    analysis.add_argument("--output", required=True, type=Path)
    analysis.add_argument("--campaign", help="Analyze one campaign
only")
    return root


def main(argv: list[str] | None = None) -> int:
    if hasattr(sys.stdout, "reconfigure"):
        sys.stdout.reconfigure(encoding="utf-8")
        sys.stderr.reconfigure(encoding="utf-8")
    args = parser().parse_args(argv)
    if hasattr(args,"qram_impl"):
        os.environ["NOSTRADAMUS_QRAM_IMPL"] = args.qram_impl
        os.environ["NOSTRADAMUS_RNG"] = args.rng
        os.environ["NOSTRADAMUS_BLACKBURN_BACKEND"] =
args.blackburn_backend
    def log(text):
        if not args.quiet:
            cleaned = str(text).replace("state-vector, без реального
квантового ускорения.", "классический симулятор.").replace(", без
реального квантового ускорения.", "").replace("—", "-")
            print(cleaned, file=sys.stderr if args.json else sys.stdout,
flush=True)
    started = time.perf_counter()
    store = None
    metadata = None
    failure_record = None
    try:
        adapter = BackendAdapter()
        if args.command == "selftest":
            results = []
            for command in ([sys.executable, "-m", "pytest", "-q"],
*([sys.executable, "tests/" + name] for name in
                            ("selftest.py", "selftest_cpp.py",
"test_diamond_builders.py", "test_quantum_algorithms.py",
"test_sigma_aware.py"))):
                completed = subprocess.run(command, cwd=ROOT,
capture_output=True, text=True,
                                           encoding="utf-8",
env={**os.environ, "PYTHONIOENCODING": "utf-8"})
                log(completed.stdout)
```

```
                if completed.stderr:
                    log(completed.stderr)
                results.append({"command": command[1:], "exit_code":
completed.returncode})
                if completed.returncode:
                    raise RuntimeError(f"Test command failed:
{command}")
            output = {"status": "PASS", "suites": results}
        elif args.command == "validate-diamond":
            data = adapter.load_json(args.path)
            output = {"status": "PASS", "path":
str(args.path.resolve()), "leaves": len(data["leaves"]),
                      "edges": sum(len(layer) * 2 for layer in
data["structure"]), "root": adapter.root(data)}
        elif args.command in ("build-diamond", "attack"):
            if args.seed is not None and not 0 <= args.seed < 2**64:
                raise ValueError("seed must be in 0..2^64-1")
            if args.threads < 0:
                raise ValueError("threads must be non-negative")
            selected = search_method(args.method)
            bbht_options = BBHTOptions(args.bbht_growth,
args.bbht_max_attempts, args.bbht_max_queries,
                                       args.bbht_dense_max_states,
args.bbht_stage_seconds)
            if selected is SearchMethod.GROVER_BBHT:
                bbht_options.validate()
            store = ExperimentStore(None, args.record_db)
            metadata = environment(args.seed, args.threads) |
{"builder": args.method, "campaign": None, "repetition": None,
"statevector_simulation": selected.is_quantum}
            if args.command == "build-diamond":
                params = {"t": args.n, "k": args.k, "prefix_blocks":
args.prefix_blocks,
                          "rounds": args.rounds, "sigma_bits":
args.sigma_bits, "trials_limit": args.trials_limit,
                          'sigma_seed': args.sigma_seed,
                          "parameter_strategy": args.parameter_strategy,
"target_probability": args.target_probability,
                          "bbht_options": asdict(bbht_options) if
selected is SearchMethod.GROVER_BBHT else None}
                metadata["parameters"] = params
                failure_record = ExperimentRecord(
                    selected.storage_id,
                    args.n, args.k, args.prefix_blocks, args.rounds,
None, "Ошибка",
                    sigma_bits=args.sigma_bits,
measurement_version=MODEL_REVISION)
                result = build_to_path(adapter, params, args.method,
args.seed, args.output, log, args.threads)
                entry = store.add_completed(result.data,
```

```
result.elapsed_seconds, result.path, metadata)
                output = {"status": "PASS", "path": str(result.path),
"elapsed_seconds": result.elapsed_seconds,
                          "record": entry.record.to_storage_dict(),
"record_id": entry.identity}
            else:
                data = adapter.load_json(args.diamond)
                metadata["parameters"] = {"t": data["trunc_bits"], "k":
data["depth"],
                                          "prefix_blocks":
data["prefix_blocks"], "rounds": data["rounds"],
                                          "sigma_bits":
data.get("sigma_bits", 0), "trials_limit": args.trials_limit}
                failure_record = ExperimentRecord(
                    selected.storage_id,
                    data["trunc_bits"], data["depth"],
data["prefix_blocks"], data["rounds"], None, "Ошибка",
                    sigma_bits=data.get("sigma_bits", 0),
experiment_type="herding", measurement_version=MODEL_REVISION)
                result = adapter.run_attack(data, args.description,
args.event_result, args.trials_limit,
                                            args.method, args.seed,
args.threads,
                                            bbht_options=bbht_options if
selected is SearchMethod.GROVER_BBHT else None)
                output = {"status": "PASS", "elapsed_seconds":
result.elapsed_seconds,
                          "result": asdict(result.result)} |
full_hashes(result.result.forged_hex)
                metadata |= {"full_gost512": output["full_gost512"],
"full_gost256": output["full_gost256"]}
                entry = store.add_attack(data, result.result,
result.elapsed_seconds, args.diamond,
                                         args.trials_limit, metadata)
                output["record_id"] = entry.identity
                if args.output:
                    args.output.parent.mkdir(parents=True,
exist_ok=True)
                    args.output.write_text(json.dumps(output,
ensure_ascii=False, indent=2), encoding="utf-8")
        elif args.command == "campaign":
            output = run_campaign(args.config, resume=args.resume,
database=args.record_db, log=log)
        else:
            from integration.experiment_analysis import analyze_database
            output = analyze_database(args.database, args.output,
args.campaign)
        if args.json:
            print(json.dumps(output, ensure_ascii=False), flush=True)
        elif not args.quiet:
```

```
            print(json.dumps(output, ensure_ascii=False, indent=2),
flush=True)
        return 1 if args.command == "campaign" and
(output.get("failed_campaign", 0) or output.get("skipped_campaign", 0))
else 0
    except (Exception, KeyboardInterrupt) as exc:
        interrupted = isinstance(exc, KeyboardInterrupt)
        message = str(exc) or "Interrupted by Ctrl+C"
        if store is not None and failure_record is not None and
failure_record.n > 0 and failure_record.k > 0:
            from dataclasses import replace
            try:
                store.add_failure(replace(failure_record,
elapsed_seconds=time.perf_counter() - started,
                                          status="Остановлено" if
interrupted else "Ошибка"),
                                  (metadata or {}) | {"error": message,
"interrupted": interrupted})
            except Exception as persistence_error:
                message += f"; cannot save failed attempt:
{persistence_error}"
        if args.json:
            print(json.dumps({"status": "INTERRUPTED" if interrupted
else "ERROR", "error": message}, ensure_ascii=False), flush=True)
        print(f"ERROR: {message}", file=sys.stderr, flush=True)
        if args.verbose and not interrupted:
            traceback.print_exc()
        return 130 if interrupted else 1


if __name__ == "__main__":
    raise SystemExit(main())
```

ПРИЛОЖЕНИЕ М

Сравнение экспериментальных результатов в интерфейсе PySide6 на языке Python

```
from __future__ import annotations


from PySide6.QtCore import Qt, Signal, QSize, QEvent, QVariantAnimation,
QEasingCurve
from PySide6.QtGui import QIcon
from PySide6.QtWidgets import (
    QComboBox, QGridLayout, QHBoxLayout, QHeaderView, QLabel,
QPushButton, QScrollArea,
    QTableWidget, QVBoxLayout, QWidget, QLineEdit, QFrame, QMenu,
QLayout, QSizePolicy,
)
from pathlib import Path
from .comparison_widgets import COLORS, SortableItem,
ExperimentDelegate, action_button
from .scientific_comparison import ScientificComparisonMixin

from algorithms.search_methods import SearchMethod, search_method
from integration.result_adapter import comparison_key, comparable_pairs
from .math_text import math_html
from .math_renderer import MathHeaderView
from .metric_widgets import formatted_number
from .widgets import Card


class ComparisonTab(ScientificComparisonMixin, QWidget):
    QUERIES = "queries"
    RUNTIME = "runtime"
    EVALUATIONS = "evaluations"
    MEMORY = "memory"
    load_requested = Signal()
    delete_requested = Signal(object)
    repeat_requested = Signal(object)
    def __init__(self,parent=None):
        super().__init__(parent)
        self.setObjectName("comparisonTab")
        self.setAccessibleName("Сравнение алгоритмов")
        self._scientific_ready=False

self._sort_column=None;self._sort_order=None;self._experiments_collapsed
=False
        self._experiments_progress=1.;self._expanded_panel_height=260
        self.records=[];self._redrawing=False;self._records_by_id={}
```

```
layout=QVBoxLayout(self);layout.setContentsMargins(15,14,15,14);layout.s
etSpacing(12)
        self.chart=Card();self.chart.setObjectName("comparisonChart")
        layout.addWidget(self.chart,5)

self.experiments_panel=QWidget();self.experiments_panel.setObjectName("c
omparisonExperimentsPanel")

lower=QHBoxLayout(self.experiments_panel);lower.setContentsMargins(0,0,0
,0);lower.setSpacing(12)

self.table_card=Card();self.table_card.setObjectName("comparisonTableCar
d")
        self.table_card.layout_box.setSpacing(0)

self.experiments_heading=QWidget();self.experiments_heading.setFixedHeig
ht(36)

table_heading=QHBoxLayout(self.experiments_heading);table_heading.setCon
tentsMargins(0,0,0,0);table_heading.setSpacing(8)

self.experiments_title=QPushButton("Эксперименты");self.experiments_titl
e.setObjectName("comparisonSectionToggle")

self.experiments_title.setCursor(Qt.CursorShape.PointingHandCursor)
        self.experiments_title.clicked.connect(self.toggle_experiments)

self.experiments_icon=QLabel();self.experiments_icon.setPixmap(QIcon(str
(Path(__file__).with_name("resources")/"comparison_database.svg")).pixma
p(24,24))

table_heading.addWidget(self.experiments_icon);table_heading.addWidget(s
elf.experiments_title);table_heading.addStretch()

self.experiments_tools=QWidget();self.experiments_tools.setMinimumWidth(
0)

tools=QHBoxLayout(self.experiments_tools);tools.setContentsMargins(0,0,0
,0);tools.setSpacing(8);tools.setSizeConstraint(QLayout.SizeConstraint.S
etNoConstraint)

self.search=QLineEdit();self.search.setObjectName("comparisonSearch");se
lf.search.setPlaceholderText("Поиск по экспериментам…")
        self.search.setToolTip("Метод, этап или статус; параметры: n:20
k:3 s:0 N:1024")

self.search.setClearButtonEnabled(True);self.search.setMaximumWidth(260)
;self.search.setMinimumWidth(100)
        tools.addWidget(self.search,1)
```

```
        self.stage_filter=QComboBox()
        self.filter_button=action_button("settings","Фильтры
экспериментов")
        tools.addWidget(self.filter_button)

self.load_button=QPushButton();self.load_button.setObjectName("compariso
nImport")

self.load_button.setIcon(QIcon(str(Path(__file__).with_name("resources")
/"parameter_folder.svg")))
        for button in (self.filter_button,self.load_button):
            button.setFixedSize(36,36);button.setIconSize(QSize(18,18))
        self.load_button.setToolTip("Загрузить базу экспериментов или
JSON структуры")

self.load_button.clicked.connect(self.load_requested);tools.addWidget(se
lf.load_button)
        table_heading.addWidget(self.experiments_tools)
        self.table_card.layout_box.addWidget(self.experiments_heading)

self.experiments_content=QWidget();self.experiments_content.setObjectNam
e("comparisonPanelContent")

content=QVBoxLayout(self.experiments_content);content.setContentsMargins
(0,12,0,0);content.setSpacing(0);content.setSizeConstraint(QLayout.SizeC
onstraint.SetNoConstraint)

self.experiments_content.setMinimumHeight(0);self.experiments_content.se
tSizePolicy(QSizePolicy.Policy.Expanding,QSizePolicy.Policy.Ignored)

self.table=QTableWidget(0,12);self.table.setObjectName("comparisonExperi
ments")
        self.table.setHorizontalHeader(MathHeaderView(self.table))

self.table.setHorizontalHeaderLabels(("Метод","n","k","N","s","Раунды","
Q","g","qRAM","Время","Статус","Этап"))

header=self.table.horizontalHeader();header.setSectionResizeMode(QHeader
View.ResizeMode.Interactive)
        header.setDefaultAlignment(Qt.AlignmentFlag.AlignCenter)

header.setSectionResizeMode(0,QHeaderView.ResizeMode.Interactive)
        self.table.setColumnWidth(0,230)
        self.table.viewport().installEventFilter(self)
        for column,width in
((1,52),(2,52),(3,70),(4,52),(5,65),(6,85),(7,95),(8,85),(9,95),(10,115)
,(11,140)):self.table.setColumnWidth(column,width)

self.table.horizontalHeaderItem(0).setTextAlignment(Qt.AlignmentFlag.Ali
gnLeft|Qt.AlignmentFlag.AlignVCenter)
```

```
        header.moveSection(11,1)
        for column in (3,5,8):self.table.setColumnHidden(column,True)

header.setContextMenuPolicy(Qt.ContextMenuPolicy.CustomContextMenu)
        header.customContextMenuRequested.connect(self._columns_menu)
        self.table.setItemDelegate(ExperimentDelegate(self.table))

self.table.verticalHeader().hide();self.table.verticalHeader().setDefaul
tSectionSize(38)

self.table.setSelectionBehavior(QTableWidget.SelectionBehavior.SelectRow
s)

self.table.setEditTriggers(QTableWidget.EditTrigger.NoEditTriggers)
        self.table.setWordWrap(False);self.table.setShowGrid(False)
        self.table.setToolTip("Q — модельные запросы к оракулу; g —
вычисления функции сжатия; qRAM — пиковое число записей. Остальные
столбцы доступны через меню заголовка.")
        header.setSortIndicator(-1,Qt.SortOrder.AscendingOrder)

header.setSortIndicatorShown(False);header.setSectionsClickable(True)
        header.sectionClicked.connect(self._sort_experiments)
        self.table.setSortingEnabled(False)

content.addWidget(self.table);self.table_card.layout_box.addWidget(self.
experiments_content,1)

self.inspector=Card();self.inspector.setObjectName("comparisonInspector"
)

self.inspector.setMinimumWidth(340);self.inspector.setMaximumWidth(350)
        self.inspector.layout_box.setSpacing(0)

self.inspector_heading=QWidget();self.inspector_heading.setFixedHeight(3
6)

title=QHBoxLayout(self.inspector_heading);title.setContentsMargins(0,0,0
,0);title.setSpacing(8)

self.inspector_icon=QLabel();self.inspector_icon.setPixmap(QIcon(str(Pat
h(__file__).with_name("resources")/"comparison_info.svg")).pixmap(23,23)
)
        self.inspector_title=QPushButton("Выбранный
эксперимент");self.inspector_title.setObjectName("comparisonSectionToggl
e")

self.inspector_title.setCursor(Qt.CursorShape.PointingHandCursor);self.i
nspector_title.clicked.connect(self.toggle_experiments)

title.addWidget(self.inspector_icon);title.addWidget(self.inspector_titl
```

```
e,1)
        self.inspector.layout_box.addWidget(self.inspector_heading)

self.inspector_content=QWidget();self.inspector_content.setObjectName("c
omparisonPanelContent")

inspector_box=QVBoxLayout(self.inspector_content);inspector_box.setConte
ntsMargins(0,12,0,0);inspector_box.setSpacing(12);inspector_box.setSizeC
onstraint(QLayout.SizeConstraint.SetNoConstraint)

self.inspector_content.setMinimumHeight(0);self.inspector_content.setSiz
ePolicy(QSizePolicy.Policy.Expanding,QSizePolicy.Policy.Ignored)

scroll=QScrollArea();scroll.setObjectName("comparisonInspectorScroll");s
croll.viewport().setObjectName("comparisonInspectorViewport")

scroll.setWidgetResizable(True);scroll.setFrameShape(QScrollArea.Shape.N
oFrame);scroll.setHorizontalScrollBarPolicy(Qt.ScrollBarPolicy.ScrollBar
AlwaysOff)

scroll.setMinimumHeight(0);scroll.setSizePolicy(QSizePolicy.Policy.Expan
ding,QSizePolicy.Policy.Ignored)

body=QWidget();body.setObjectName("inspectorBody");body_layout=QVBoxLayo
ut(body);body_layout.setContentsMargins(0,0,2,0);body_layout.setSpacing(
4)
        self.inspector_values={};self.inspector_names={}
        badges=QHBoxLayout()
        for key in ("method","stage"):
            badge=QLabel("Нет
данных");badge.setObjectName("comparisonBadge");badge.setWordWrap(True)
            self.inspector_values[key]=badge;badges.addWidget(badge)
        badges.addStretch();body_layout.addLayout(badges)
        for key,name in (("time","Время выполнения"),("Q","Запросов к
оракулу"),("g","Вычислений функции сжатия")):
            self._inspector_row(body_layout,key,name)

line=QFrame();line.setFixedHeight(1);line.setObjectName("comparisonDivid
er");body_layout.addWidget(line)
        body_layout.addWidget(QLabel("Параметры"))

parameters=QGridLayout();parameters.setHorizontalSpacing(10);parameters.
setVerticalSpacing(3)
        for index,key in enumerate(("n","k","s")):
            pair=QHBoxLayout();pair.setSpacing(4)
            name=QLabel(math_html(("Раунды" if key=="rounds" else key)+"
="));name.setObjectName("inspectorName")
            value=QLabel("-
");value.setObjectName("inspectorValue");value.setTextInteractionFlags(Q
t.TextInteractionFlag.TextSelectableByMouse)
```

```
            pair.addWidget(name);pair.addWidget(value);pair.addStretch()
            parameters.addLayout(pair,index//3,index%3,1,2 if
key=="rounds" else 1)

self.inspector_names[key]=name;self.inspector_values[key]=value
        body_layout.addLayout(parameters)

body_layout.addStretch();scroll.setWidget(body);inspector_box.addWidget(
scroll,1)
        actions=QHBoxLayout()

self.delete_button=QPushButton("Удалить");self.repeat_button=QPushButton
("Повторить")
        self.repeat_button.setObjectName("primaryButton")
        self.delete_button.setObjectName("comparisonDelete")
        for button in (self.delete_button,self.repeat_button):

button.setProperty("comparisonAction",True);button.setFixedHeight(44)

actions.addWidget(self.delete_button);actions.addWidget(self.repeat_butt
on)

inspector_box.addLayout(actions);self.inspector.layout_box.addWidget(sel
f.inspector_content,1)

self.delete_button.clicked.connect(lambda:self.delete_requested.emit(sel
f._selected_record()))

self.repeat_button.clicked.connect(lambda:self.repeat_requested.emit(sel
f._selected_record()))

lower.addWidget(self.table_card,1);lower.addWidget(self.inspector)
        layout.addWidget(self.experiments_panel,4)
        self.search.textChanged.connect(self._controls_changed)
        self.table.itemSelectionChanged.connect(self._selection_changed)
        self._setup_scientific()

self._collapse_animation=QVariantAnimation(self);self._collapse_animatio
n.setDuration(240)

self._collapse_animation.setEasingCurve(QEasingCurve.Type.InOutCubic)

self._collapse_animation.valueChanged.connect(self._reveal_experiments)

self._collapse_animation.finished.connect(self._finish_experiments_trans
ition)
        self._reveal_experiments(1.)

    def _sort_experiments(self,column):
        if column!=self._sort_column:
```

```
self._sort_column=column;self._sort_order=Qt.SortOrder.AscendingOrder
        elif self._sort_order==Qt.SortOrder.AscendingOrder:
            self._sort_order=Qt.SortOrder.DescendingOrder
        else:
            self._sort_column=None;self._sort_order=None
        previous=self.table.blockSignals(True)
        try:self._apply_experiment_sort()
        finally:self.table.blockSignals(previous)

    def _apply_experiment_sort(self):

header=self.table.horizontalHeader();previous=header.blockSignals(True)
        try:
            if self._sort_column is None:
                cells=[self.table.item(row,0) for row in
range(self.table.rowCount())]
                for cell in cells:cell.use_natural_order=True
                try:self.table.sortItems(0,Qt.SortOrder.AscendingOrder)
                finally:
                    for cell in cells:cell.use_natural_order=False
                header.setSortIndicator(-
1,Qt.SortOrder.AscendingOrder);header.setSortIndicatorShown(False)
            else:
                self.table.sortItems(self._sort_column,self._sort_order)

header.setSortIndicator(self._sort_column,self._sort_order);header.setSo
rtIndicatorShown(True)
        finally:header.blockSignals(previous)

    def toggle_experiments(self):
        self._experiments_collapsed=not self._experiments_collapsed
        self._collapse_animation.stop();self.experiments_tools.show()

self._collapse_animation.setStartValue(self._experiments_progress)
        self._collapse_animation.setEndValue(0. if
self._experiments_collapsed else 1.)
        self._collapse_animation.start()

    def _reveal_experiments(self,progress):
        self._experiments_progress=float(progress)
        margins=self.table_card.layout_box.contentsMargins()

heading=margins.top()+self.experiments_heading.height()+margins.bottom()
        height=round(heading+(self._expanded_panel_height-
heading)*self._experiments_progress)
        self.experiments_panel.setFixedHeight(height)
        for body in (self.experiments_content,self.inspector_content):
            body.setMaximumHeight(max(0,height-
heading));body.setVisible(self._experiments_progress>0)
```

```
self.experiments_tools.setMaximumWidth(round(340*self._experiments_progr
ess))

    def _finish_experiments_transition(self):
        self.experiments_tools.setVisible(not
self._experiments_collapsed)


    def eventFilter(self,watched,event):
        if hasattr(self,"table") and watched is self.table.viewport()
and event.type()==QEvent.Type.Resize:
            header=self.table.horizontalHeader()
            others=sum(header.sectionSize(column) for column in
range(1,self.table.columnCount()) if not
self.table.isColumnHidden(column))

self.table.setColumnWidth(0,max(210,self.table.viewport().width()-
others))
        return super().eventFilter(watched,event)


    def resizeEvent(self,event):
        super().resizeEvent(event)
        if hasattr(self,"inspector"):
            self.inspector.setMinimumWidth(340 if self.width()>=1350
else 275)
        if hasattr(self,"_collapse_animation"):

self._expanded_panel_height=max(140,min(360,round(self.height()*.32)))
            self._reveal_experiments(self._experiments_progress)


    def closeEvent(self,event):
        if
hasattr(self,'scientific_chart'):self.scientific_chart.shutdown()
        super().closeEvent(event)


    def _columns_menu(self,point):
        menu=QMenu(self.table)
        for column in range(self.table.columnCount()):

action=menu.addAction(self.table.horizontalHeaderItem(column).text());ac
tion.setCheckable(True)
            action.setChecked(not self.table.isColumnHidden(column))
            action.toggled.connect(lambda
checked,c=column:self.table.setColumnHidden(c,not checked))
        menu.exec(self.table.horizontalHeader().mapToGlobal(point))
```

```
    def _selected_record(self):
        selected=self.table.selectionModel().selectedRows()
        if not selected:return None
        cell=self.table.item(selected[0].row(),0)
        return
self._records_by_id.get(cell.data(Qt.ItemDataRole.UserRole)) if cell
else None


    def _inspector_row(self, layout: QVBoxLayout, key: str, label: str)
-> None:
        line = QHBoxLayout()
        line.setSpacing(8)
        name = QLabel(math_html(label))
        name.setObjectName("inspectorName")
        name.setWordWrap(True)
        name.setMinimumHeight(32)
        value = QLabel("-")
        value.setObjectName("inspectorValue")

value.setTextInteractionFlags(Qt.TextInteractionFlag.TextSelectableByMou
se)
        value.setAlignment(Qt.AlignmentFlag.AlignRight |
Qt.AlignmentFlag.AlignVCenter)
        value.setWordWrap(True)
        value.setMinimumHeight(32)
        value.setMaximumWidth(110)
        line.addWidget(name, 3)
        line.addWidget(value, 1)
        layout.addLayout(line)
        self.inspector_values[key] = value
        self.inspector_names[key] = name


    @staticmethod
    def _counter(record):
        value = record.link_N if record.experiment_type == "herding"
else record.first_level_N
        return "-" if value is None else value


    @staticmethod
    def _number(value: float | int | None, runtime: bool = False) ->
str:
        if value is None:
            return "-"
        if runtime:return f"{value:.3f}".replace(".",",")+" c"
        if isinstance(value,float) and not value.is_integer():
            return f"{value:,.2f}".rstrip("0").rstrip(".").replace(","."
```

```
").replace(".",",")
        return formatted_number(int(value))


    def _fill_table(self) -> None:
        records = self._visible_records()
        selected = self.table.selectionModel().selectedRows()
        previously_selected = None
        if selected and selected[0].row() < self.table.rowCount():
            previously_selected = self.table.item(selected[0].row(), 0)
            previously_selected =
previously_selected.data(Qt.ItemDataRole.UserRole) if
previously_selected else None
        self.table.setUpdatesEnabled(False)
        self.table.blockSignals(True)
        sorting=self.table.isSortingEnabled()
        self.table.setSortingEnabled(False)
        self.table.setRowCount(len(records))
        for row, record in enumerate(records):
            values = (
                "BBHT" if search_method(record.method) is
SearchMethod.GROVER_BBHT else record.method_label,
                record.n, record.k, self._counter(record),
record.sigma_bits,
                record.rounds,
formatted_number(record.algorithmic_queries),
                formatted_number(self._g(record)),
formatted_number(record.quantum_qram_peak_entries),
                self._number(record.elapsed_seconds, True),
record.status,record.stage_label,
            )
            numeric={1:record.n,2:record.k,3:record.link_N if
record.experiment_type=="herding" else
record.first_level_N,4:record.sigma_bits,5:record.rounds,6:record.algori
thmic_queries,7:self._g(record),8:record.quantum_qram_peak_entries,9:rec
ord.elapsed_seconds}
            for col, value in enumerate(values):
                cell = SortableItem(str(value))
                cell.natural_order=row
                cell.setTextAlignment(Qt.AlignmentFlag.AlignLeft |
Qt.AlignmentFlag.AlignVCenter if col == 0 else
                                      Qt.AlignmentFlag.AlignCenter)
                if col == 0:
                    cell.setData(Qt.ItemDataRole.UserRole, id(record))

cell.setData(Qt.ItemDataRole.UserRole+2,COLORS[search_method(record.meth
od)])
                if col in numeric:cell.sort_value=numeric[col] if
numeric[col] is not None else -1
                cell.setToolTip(str(value))
```

```
                self.table.setItem(row, col, cell)
        self.table.setSortingEnabled(sorting)
        self._apply_experiment_sort()
        if previously_selected not in {id(record) for record in records}
and records:
            previously_selected=id(records[0])
        self.table.clearSelection()
        for row in range(self.table.rowCount()):
            if self.table.item(row,0).data(Qt.ItemDataRole.UserRole) ==
previously_selected:
                self.table.selectRow(row)
                break
        self.table.blockSignals(False)
        self.table.setUpdatesEnabled(True)
        self.experiments_title.setText("Эксперименты")
        self.experiments_title.setToolTip(f"Показано {len(records)} из
{len(self.records)} записей")
        self._selection_changed()
```